

THE CURIOUS FAMILY LIBRARY

A First Taste of Algorithms

and the Machines That Learn



Meira Davidson

THE CURIOUS FAMILY LIBRARY

A First Taste of Algorithms

and the Machines That Learn



Meira Davidson

A First Taste of Algorithms — and the Machines That Learn

THE CURIOUS FAMILY LIBRARY

Copyright © 2026 Meira Davidson

All rights reserved.

Set in Iowan Old Style; figures drawn with Matplotlib.

First edition • 2026

For the Jessica family kids — be curious, stay open, be kind.

What's in This Book

This book is an introduction to algorithms — step-by-step methods for solving problems with a computer — and to the ideas behind machine learning. Each chapter takes one idea, shows why it works, and asks what it costs. Here is what each chapter covers:

Chapter 1: Two Questions. How to find something in a list quickly, and the two questions this book asks about every method — does it always work, and how much work does it take?

Chapter 2: The Greedy Idea. Solving a problem by always grabbing the best-looking option right now: when that works (fitting the most events into one day) and when it fails (making change with odd coins).

Chapter 3: The Procedure That Calls Itself. Recursion — solving a problem by having it call a smaller copy of itself. The Towers of Hanoi puzzle, merge sort, and how to count what a recursive method costs.

Chapter 4: Recursion That Remembers. How letting a method remember answers it has already worked out can turn a hopelessly slow procedure into a fast one.

Chapter 5: Maps, Mazes, and Networks. Graphs — dots joined by lines. Finding your way out of a maze, and finding the shortest route across a map.

Interlude: The Other Exponential. A short break: why computers kept getting faster for decades, and what that has to do with this book.

Chapter 6: Order, and Its Price. Putting things in order: why some sorting methods are far faster than others, and a proof that none can ever beat a certain speed limit.

Chapter 7: The Algorithm That Flips Coins. Using chance on purpose — why a method that makes random choices can be both fast and dependable, plus the surprising birthday coincidence.

Chapter 8: Learning as an Algorithm. The first machine-learning ideas: making predictions from examples, and improving a guess by taking small steps downhill.

Chapter 9: Building the Learner. Putting the pieces together to build a small learning machine from scratch, using the same kind of math behind the AI tools you already use.

Epilogue: The Edge of the Map. Two famous questions at the very limits of computing: one problem no computer can ever solve, and one that nobody yet knows how to.

Contents

Chapter 1: Two Questions	1
Problem Set 1	11
Lab 0: Talking to the Machine	15
Lab 1: Counting Steps	19
Chapter 2: The Greedy Idea	25
Problem Set 2	38
Lab 2: The Cost of Confidence	42
Chapter 3: The Procedure That Calls Itself	48
Problem Set 3	56
Lab 3: The Visible Staircase	60
Chapter 4: Recursion That Remembers	67
Problem Set 4	76
Lab 4: The Notebook	81
Chapter 5: Maps, Mazes, and Networks	88
Problem Set 5	98
Lab 5: Dots and Lines	103
Interlude: The Other Exponential	111
Chapter 6: Order, and Its Price	113
Problem Set 6	123
Lab 6: Counted and Clocked	128
Chapter 7: The Algorithm That Flips Coins	135
Problem Set 7	144

Lab 7: The Honest Die	149
Chapter 8: Learning as an Algorithm	157
Problem Set 8	169
Lab 8: Downhill	174
Chapter 9: Building the Learner	182
Problem Set 9	197
Lab 9: The Engine	204
Epilogue: The Edge of the Map	219
Going Deeper	226

Chapter 1: Two Questions

★ ★ ★

In which a guessing game turns out to contain most of this book

The game

Here is a game you have probably played. I think of a whole number between 1 and 100. You ask yes-or-no questions; I answer honestly; you win when you name the number.

You could ask “Is it 1?” — “Is it 2?” — “Is it 3?” and grind your way up. You will win eventually. If you are unlucky, it takes a hundred questions.

Or you could ask: **“Is it bigger than 50?”**

Stop and feel what that question does. Whatever I answer — yes or no — half the numbers die. One question, fifty suspects eliminated, no matter what. The grinding strategy eliminates one suspect per question on a bad day. This question eliminates fifty *on its worst day*.

Ask it again on whatever half survives: “bigger than 75?” or “bigger than 25?” Each question halves the survivors. From 100: down to 50, 25, 13, 7, 4, 2, 1. Seven questions and you have me cornered, *guaranteed*, no luck required.

Play it once with someone before reading on — really. Pick the range 1 to 1000 and see whether ten questions suffice. (They do. By the end of this chapter you will be able to say exactly why, and

exactly how many questions a range of a trillion would need. The answer will be smaller than you think.)

This game, taken seriously, contains the two questions this entire book asks, over and over, of every procedure we meet:

1. **Does it work?** — not “did it work that time,” but: does it *always* work, for every number I might have picked?
2. **What does it cost?** — counted in questions, in steps, in moves; counted, not guessed.

A procedure precise enough to be followed without judgment, together with answers to those two questions, is what this book means by an **algorithm**. The word comes from a person — al-Khwārizmī, a ninth-century scholar in Baghdad whose textbook on calculation was so widely followed that his name became the name for *following a recipe exactly*. We will be choosy, though: a recipe is only interesting to us once we can say what it promises and what it charges.

Why “always” is the hard part

Look at the first demand again: the procedure must work for *every* possible input. This is a strange and strong demand, and it is what makes algorithms mathematics rather than folklore.

Suppose your friend claims a strategy for the guessing game and demonstrates it: she finds your number in six questions. Impressive — and it proves almost nothing. Maybe she was lucky. Maybe your number happened to sit somewhere her strategy reaches quickly. A demonstration is one input. A guarantee is *all* of them, and there are a hundred possible numbers — or a trillion, in the big version. You cannot check a trillion cases by playing a trillion games.

So how can anyone ever promise “always”? The same way mathematics always handles infinity and its little cousins: by an argument

that covers every case at once without visiting any of them. For the halving strategy, the argument is short enough to say in one breath: *after every question, the suspects that remain always form a single unbroken range that contains my number, and the range shrinks by at least half each time; a range under 1 cannot hide anything.* Hold on to the first half of that sentence — “the survivors always form a range containing the answer.” A statement that stays true through every step of a procedure is the basic instrument for proving procedures correct, and it has a name we will meet in the deep water below.

The second demand — counting cost — needs its own honesty rule. When we count, we count the **worst case**: the most steps the procedure could possibly need, over all inputs. Not the lucky case, not the typical case (we’ll earn the right to talk about “typical” in Chapter 7, and it will take real probability to do it honestly). The worst case is the only promise you can make to someone who hasn’t told you their input yet. “At most seven questions” is a contract. “Usually around four” is a mood.

The obvious algorithm

Now put the game in the form computers actually meet ten billion times a day. You have a **list** of items — names, numbers, song titles — and a **target**, and the question is: *is the target in the list, and where?*

The obvious procedure: check the first item. Then the second. Keep going until you find the target or run out of list. This is **linear search**, and you should not look down on it. It is correct on *every* list — sorted, scrambled, repetitive, empty — and its correctness needs no cleverness to believe: if the target is anywhere, the march from front to back must step on it.

The cost: on a list of n items, up to n checks. There is the word from the honesty rule — *up to*. If the target sits in front, one check. If it sits at the back, n . And if the target is **absent**, exactly n , every time, with no luck possible: you cannot declare something missing until

you have looked everywhere. Sit with that for a second, because it is the first genuinely useful insight of the book. *Proving absence is expensive*. Finding your keys takes until you find them; being *sure* they're not in the house takes searching the whole house.

Unless — and this is the turn the whole chapter pivots on — the house is organized.

Structure buys speed

Suppose the list is **sorted**: arranged in order, smallest to largest, A to Z. What did that arrangement buy?

Everything. A sorted list answers the guessing game's magic question. Check the *middle* item. If it equals the target, done. If the target is bigger, then — because the list is sorted — the target cannot live anywhere in the left half. Discard half the list *without looking at it*. If smaller, discard the right half. Repeat on whatever survives.

This is **binary search**, and the phrase to underline is *without looking at it*. Linear search pays one step per item examined and learns about one item. Binary search learns about ten thousand items by examining one, because the sortedness lets a single comparison speak for an entire half. The information was purchased in advance, when the list was sorted, and binary search is how you collect.

Do one by hand, slowly. The sorted list:

2, 5, 8, 13, 21, 34, 55, 89

Target: 21. Keep two fingers on the list — one at each end — marking the region where the target could still be.

- The region is the whole list. The middle (with eight items, take the fourth) is **13**. The target 21 is bigger, so everything from 13 leftward is dead. The region is now 21, 34, 55, 89.

- Middle of the region: **34**. Too big; 34 and everything rightward is dead. The region is just 21.
- Check **21**. Found, three comparisons.

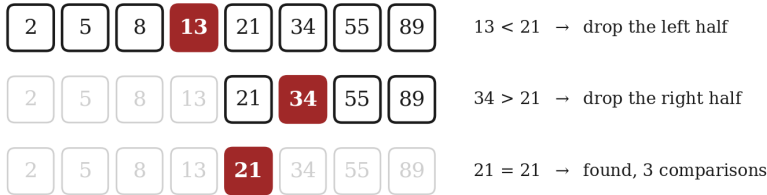


Figure 1.1 — Binary search for 21: the region between the fingers halves at every comparison.

Figure 1.1 — Binary search for 21: the region between the fingers halves at every comparison.

Now the same list, target **20** — a number that *isn't there*. Run the same procedure: 13 (too small, go right), 34 (too big, go left), 21 (too big, go left) — and now the two fingers have crossed. The region where 20 could hide has shrunk to nothing. The procedure doesn't just fail to find 20; it *proves* 20 absent, in three comparisons, where linear search would have needed eight. The expensive thing — certainty of absence — got cheap. That is what structure is for.

Counting the halvings

So binary search costs “however many times you can halve the list before nothing is left.” Give that a name and a table. Start with n items; each comparison cuts the living region at least in half:

items n	comparisons needed (worst case)
8	4
100	7
1,000	10
1,000,000	20
1,000,000,000,000	40

Read the table the way you read the race in Lab 1: the left column is *multiplying* — by 10, by 1000 — and the right column is *adding* a handful each row. A thousand times more items costs ten more comparisons. You met the function that behaves this way in Book 2: the **logarithm**, the function that turns multiplication into addition. The number of halvings from n down to 1 is $\log_2 n$, give or take a rounding we'll pin down in the deep water — and that is why a *trillion*-item phone book for the whole galaxy would surrender any name in forty questions. Forty. You could do it at a kitchen table before the tea cools.

When the cost of a procedure grows like n , we call it **linear**; like $\log n$, **logarithmic**. For now these are just names for shapes you have seen in your own tables — the steady ramp and the nearly-flat curve. We will not dress them in formal notation until Chapter 6, when we'll have enough tables behind us that the notation arrives as bookkeeping instead of vocabulary.

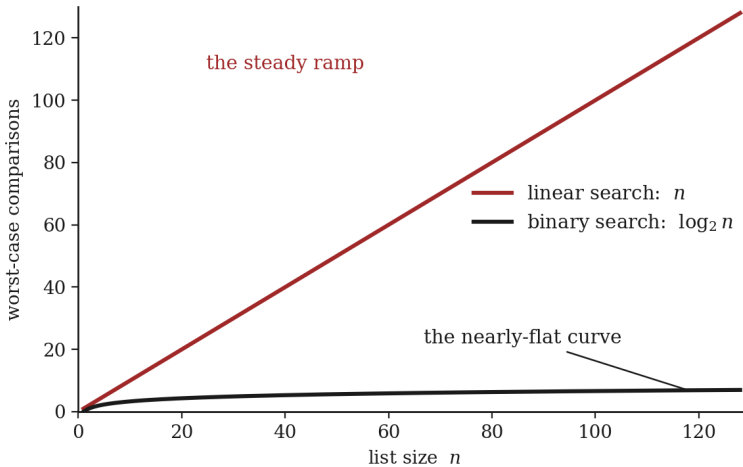


Figure 1.2 — The steady ramp and the nearly-flat curve: worst-case comparisons as the list grows.

Figure 1.2 — The steady ramp and the nearly-flat curve: worst-case comparisons as the list grows.

One more honest note, because this book does not hide its bills. Binary search demands a sorted list, and lists are not born sorted.

Sorting costs something. How much, exactly — and whether anything could ever sort faster — is Chapter 6’s story, and it ends with the most surprising kind of theorem in this book: a proof that something is *impossible*. Whether the toll is worth paying depends on how many searches you’ll run afterward, which is the kind of answer cost questions usually have: not “yes” or “no” but “it depends, and here is precisely on what.”

Deeper water: the invariant

Here is the correctness argument for binary search, done properly. Through the entire procedure, maintain this sentence:

If the target is in the list at all, it lies between the two fingers.

A sentence kept true through every step of a procedure is called a **loop invariant**, and proving one is always the same three moves — the same three moves, you’ll notice, as induction in Book 2, because it *is* induction, marching over steps of a procedure instead of whole numbers:

1. *It starts true.* Before any comparison, the fingers bracket the whole list. If the target is anywhere, it’s in there. ✓
2. *Each step keeps it true.* Suppose it’s true going into a comparison, and the middle item is too small. Sortedness says everything from the middle leftward is also too small, so the target — if present — was already in the right portion. Moving the left finger past the middle discards only dead territory. The sentence survives. (The too-big case is the mirror image.) ✓
3. *It’s true at the end, and that finishes the job.* The procedure stops in one of two ways. If it finds the target — correct, trivially. If the fingers cross, the invariant says: the target,

if present, lies in a region that no longer exists. So it isn't present, and saying so is correct. ✓

Notice what step 3 bought: the proof of *absence*. The invariant is why binary search's "not here" is a theorem, not a shrug. Hold the method itself, too — start true, stays true, finishes the job — because every correctness proof in this book is this argument wearing different clothes.

Deeper water: the exact count

"About $\log_2 n$ " can be made exact. With k yes/no questions, the answers form a sequence like *yes, no, no, yes...* — and there are exactly 2^k such sequences. Each possible position of the target must lead to a different answer-sequence (if two positions produced identical answers, the procedure couldn't tell them apart — it would answer at least one of them wrongly). A list of n items has $n + 1$ distinguishable situations: "it's at position 1," ..., "it's at position n ," and "it's absent." So we need $2^k \geq n + 1$, that is,

$k \geq \log_2(n + 1)$, so $k = \lceil \log_2(n + 1) \rceil$ questions suffice and are required

(the brackets mean "round up"). Check it against the table: $n = 1,000,000$ gives $\log_2(1,000,001) \approx 19.93$, round up to 20. ✓

Read the argument once more, because something remarkable just happened: it doesn't mention binary search. It says *no* questioning strategy whatsoever — however clever, however different from halving — can beat $\lceil \log_2(n + 1) \rceil$ in the worst case. You have just seen your first **lower bound**: a fence around what is possible, proved by counting answer-sequences rather than by trying strategies. Chapter 6 builds its great impossibility theorem with exactly this kind of fence.

A human moment

Binary search was first published in 1946, in the earliest days

of electronic computers. Here is the astonishing part, reported by Jon Bentley and documented by Donald Knuth: the first published version that worked correctly *for every list length* did not appear until **1962**. For sixteen years, the cleverest people alive wrote this twelve-line procedure with small bugs at the edges — fingers crossing wrongly, regions of size one mishandled. And the story isn't over: in 2006 an engineer at Google discovered that the binary search shipped inside Java, used by millions of programmers for two decades, broke on sufficiently huge lists — the innocent line computing the middle could overflow the machine's fixed-size numbers. (Python's numbers grow as needed, so your Lab 1 version is safe; the lesson isn't.) Hold both facts next to the deep water above. The invariant proof is short, and it is the *only* reliable defense humanity has found against sixteen years of off-by-one — which is why Break-it 1 in the lab, where you sabotage $+ 1$ and watch, is not a toy exercise but a reenactment.

What a “step” is — a confession

We have been counting “comparisons” and calling it cost, and you are entitled to ask: who decided a comparison is one step? Comparing two numbers takes the machine some flicker of time; so does moving a finger. Why is each “one”?

The honest answer: counting steps requires choosing what to count, and the choice is a *model* — a deliberate simplification, made in the open. We count comparisons because, for searching, everything else the procedure does (moving fingers, checking whether they've crossed) happens at most once per comparison, so the comparison count tracks the true effort up to a constant factor we openly decline to chase. Different chapters will count different currencies: moves in Chapter 3, additions in Chapter 4, edge-visits in Chapter 5. The discipline is always the same — name your currency, count

honestly in it, and let the *shape* of the growth (steady ramp, near-flat curve) be the conclusion, because the shape survives any reasonable change of currency. Lab 1's last experiment, where counted steps and clocked seconds drew the same curves, is this confession verified.

Where this leaves us

Two questions — *does it always work?* *what does it cost?* — and one chapter's worth of answers:

- Correctness means **all inputs**, and is established by argument, not demonstration. The argument has a standard form: a sentence kept true through every step.
- Cost is counted in a declared currency, and the **worst case** is the only promise available in advance.
- **Absence is expensive** for an unstructured list — and cheap for a structured one. Structure is purchased in advance and collected later; whether the purchase pays depends on how often you collect.
- Multiplying the problem can merely *add* to the cost — the logarithm's signature, and the first hint of how dramatically cleverness can beat effort.

Binary search was clever, but its cleverness was bespoke — a trick fitted to one problem. What we want next is a *strategy*: a way of generating algorithms for problems we haven't met yet. Chapter 2 takes up the most natural strategy there is — at every step, grab the option that looks best right now — and discovers that it is sometimes provably perfect and sometimes confidently, catastrophically wrong. Telling those two outcomes apart will not be a vibe. It will be a theorem.

Problem Set 1

★ ★ ★

The labs are the Computational Lab tier: Lab 0 (Talking to the Machine) and Lab 1 (Counting Steps) accompany this chapter. Do them before the Diamond.

Fluency

1.1 Play the halving strategy by hand on the range 1 to 64, against a friend who picks the number 50. Write down every question and answer. How many questions did you use? How many does the table promise are enough?

1.2 Trace binary search on the sorted list 3, 7, 11, 19, 23, 31, 42, 57, 60, 71, 85, 99 (twelve items) for the target 57. At each step write the region between the fingers and the middle item checked. Repeat for target 4. How many comparisons each?

1.3 For each n , compute the worst-case comparisons for linear search and for binary search: $n = 16$, $n = 1,024$, $n = 32,768$. (Notice anything about these particular numbers?)

1.4 How many times can you halve 1,000,000 before reaching 1? Compute it by repeated halving (a calculator is fine), then check against $\log_2$ from Book 2.

1.5 A sorted list has 500 items. The target is absent. Exactly how many items does linear search examine? At most how many does binary search examine?

1.6 Using the exact count from the deep water, compute

$\lceil \log_2(n+1) \rceil$ for $n = 63, 64, 65$. Explain in one sentence why 64 and 65 differ.

Practice

2.1 Your music app holds 4,000 songs. Sorted by title, how many comparisons might finding one song take, at worst? Unsorted? How many songs would the library need before the sorted search's worst case exceeded 15 comparisons?

2.2 You're guessing a number between 1 and 1,000,000, but today the answerer may *lie once* during the game. Don't solve this fully — it is genuinely hard — but answer two parts: (a) explain in a sentence or two why the plain halving strategy can now fail catastrophically; (b) show that asking every question twice rescues correctness, and count what that costs. (The gap between your repair and the best known repair is real mathematics, and open-ended.)

2.3 A “guess the word” game: I think of a word; you may ask “does it come before ___ in the dictionary?” Roughly how many questions to corner one word among 200,000? Among all $\sim 10^6$ words ever used in English?

2.4 Binary search works on any sorted list — including a list of *moments in time*. You have 10,000 photos sorted by date, and you want the first photo from your trip last July. Describe the search in words, and count its worst case. (You have just invented how photo apps jump to a date.)

2.5 Three runs of linear search on the same 1,000-item list took 3 steps, 998 steps, and 412 steps. A friend concludes “linear search takes about 471 steps on average.” Name two distinct reasons this conclusion is unjustified. (Chapter 7 will earn the right to such sentences; here, identify what's missing.)

Concept & Counterexample

3.1 A classmate claims: “binary search works fine on unsorted lists — I tried it and it found my target.” Construct an unsorted list and a target *that is in the list* for which binary search wrongly reports absence. Make your example as small as possible.

3.2 Another claim: “if binary search finds the target, the list must have been sorted.” Refute it with a three-item example.

3.3 Claim: “in the guessing game, the question ‘is it bigger than 50?’ is the best first question for the range 1–100.” Is the question “is the number even?” just as good — it also eliminates exactly half? Trace what goes wrong (or doesn’t) when you try to build a *whole strategy* out of such questions. Then say precisely what property the survivors must keep for the two-finger method to work, and which deep-water sentence that property comes from.

3.4 Claim: “worst case is too pessimistic; the target is usually somewhere in the middle.” Construct a realistic scenario where the *absent* case — the true worst case — is the overwhelmingly common one. (Hint: think about checking whether a username is taken.)

3.5 Modify binary search so that, when the fingers cross, it reports *where the target would belong* instead of merely “not found.” Why does the invariant guarantee that this position is correct? (This version is what your phone uses to keep a list sorted as items arrive.)

Diamond

4.1 The lower bound, yours now. Write out the exact-count argument from the deep water as a complete proof, in your own words, of the statement: *any strategy that solves the guessing game on 1, ..., n with yes/no questions needs at least $\lceil \log_2(n+1) \rceil$ questions in the worst case.* Your proof must make explicit (a) why two different situations cannot share an answer-sequence, (b) where the “+1” comes from, and (c) why the argument applies to every strategy, including ones no-

body has invented. Then answer: does the bound still hold if the questions need not be of the form “is it bigger than x ?” — and where in your proof is that fact used (or not used)?

Computational Lab

5.1 Labs 0 and 1, completely — including every Break-it prediction *written before running*. Bring your prediction gaps (the ones you got wrong) to the Teach It tier: they are your best material.

Teach It

6.1 Teach the guessing game and the halving strategy to someone at home, with one rule: you may not say “algorithm,” “binary,” or “logarithm.” Win the game on 1–1000 in ten questions in front of them, then get *them* to explain back to you why ten is always enough. Write three sentences on what was hardest for them to believe.

6.2 Explain to the same person why being sure something is *missing* from a messy drawer costs more than finding something that’s there — and why a sorted drawer changes that. If you can make them feel the asymmetry, you understand this chapter.

Lab 0: Talking to the Machine

★ ★ ★

Before Chapter 1. Ten minutes, maybe fifteen.

Every lab in this book asks you to run small programs and watch what they do. This one exists so that the first time is now, when nothing is at stake, rather than later, when something is.

You need a place to run Python. Any of these works: the editor at python.org, an installed copy of Python, or a notebook environment. It does not matter which. The programs in this book use nothing but Python itself — no packages to install, nothing that will stop working in five years. If a program here fits on one screen, it is meant to be typed, not copied. Typing is slower. That is the point.

1. The machine repeats what you say

Type this and run it:

```
print("hello")
```

The machine prints hello. This is the whole transaction: you write an instruction, the machine carries it out, exactly, and nothing more. It does not know what you meant. It knows what you wrote. The entire craft of programming lives in that gap, and this book is partly about learning to feel the gap before the machine shows it to you.

2. The machine does arithmetic

```
print(3 + 4)
print(3 * 4)
print(2 ** 10)
```

The last line is 2 to the 10th power. Run it. Then change it to `2 ** 100` and run it again. That number has thirty-one digits. The machine did not slow down. Hold on to this: the machine is not smart, but it is *fast*, and almost everything in this book comes from asking what fast-but-not-smart is good for.

3. The machine remembers what you name

```
steps = 0
steps = steps + 1
steps = steps + 1
print(steps)
```

A *variable* is a name attached to a value. The line `steps = steps + 1` looks wrong if you read `=` as it is read in mathematics — no number equals itself plus one. In Python, `=` is not a statement of fact. It is an instruction: *compute the thing on the right, then attach the name on the left to it*. Read it as “steps becomes steps plus one” and it is sensible.

This little program is a counter. In Chapter 1 we will count something worth counting.

4. The machine tells you when you’ve lost it

Run this. It is broken on purpose.


```
print("the answer is " + 42)
```

Python stops and prints several lines of complaint ending in something like:

```
TypeError: can only concatenate str (not "int") to str
```

Read the last line. It is not scolding you; it is reporting, precisely, what it could not do: glue a piece of text to a number, because those are different kinds of things. An error message is information. Every programmer who has ever lived reads dozens of them a day. The ones who get good are the ones who read them rather than flinch.

Fix the program by making the number into text:

```
print("the answer is " + str(42))
```

5. The machine is fast

One more, and this one needs a construct we will not explain until Chapter 1 — run it anyway:

```
total = 0
for n in range(1, 1000001):
    total = total + n
print(total)
```

This adds every whole number from 1 to one million. Run it and notice two things. First, the answer appears before your finger leaves the key. A million additions cost the machine nothing you can perceive. Second — check the answer. Gauss, as a schoolboy, is said to have computed this sum (to one hundred, but the trick scales) in seconds, by *thinking* instead of adding: pair 1 with 1,000,000, pair 2 with 999,999, and so on; 500,000 pairs, each summing to 1,000,001.

So there are two ways to be fast: do the dumb thing very quickly, or find a way to not do most of the work at all. The machine gives you the first for free. This book is about the second. An *algorithm* is, more than anything else, a way of not doing most of the work — and the strange, wonderful fact is that the not-doing can be written down, studied, and proved correct, like any other piece of mathematics.

That is where Chapter 1 begins.

Break it

End every lab this way: change one thing, *predict what will happen before you run it*, run it, and if the prediction was wrong, figure out why. The prediction is the exercise. Running the code only grades it.

1. In §3, swap the last two lines, so `print(steps)` comes before the second `steps = steps + 1`. What prints?
2. In §5, change `range(1, 1000001)` to `range(1000001)`. Predict whether the answer changes, then check. (Something here starts at 0. Does it matter?)
3. In §5, change one million to one billion: `range(1, 1000000001)`. Predict first: instant, a few seconds, a minute, or never finishes? Then run it and time it. You have just taken your first measurement of the thing this whole book is about — what computation *costs*.

Lab 1: Counting Steps

★ ★ ★

Accompanies Chapter 1. The chapter asked two questions of every algorithm: does it give the right answer, and what does the answer cost? This lab is about measuring cost — not with a stopwatch, but by counting.

1. Looking through a list is a loop

Chapter 1’s first algorithm was the obvious one: to find a name in an unsorted list, check the first entry, then the second, and keep going until you find it or run out. Written in Python, the algorithm and the sentence describing it are nearly the same thing:

```
names = ["Ada", "Erdos", "Gauss", "Hua", "Noether",  
↪ "Ramanujan"]  
  
for name in names:  
    if name == "Hua":  
        print("found it")
```

The new construct is the for loop. The line `for name in names:` means: take each entry of the list in turn, call it `name`, and run the indented lines below for it. The loop is not a programming trick bolted onto the algorithm. The loop is the algorithm — “check each one in turn” — written in a notation the machine accepts. (The other new construct, `if`, does what it says: run the indented line only when the condition holds. It will earn a fuller treatment in

Chapter 2, where an algorithm's whole character turns on a single `if`.)

2. Make the cost visible

The program above finds Hua but keeps its effort secret. Add the counter from Lab 0:

```
names = ["Ada", "Erdos", "Gauss", "Hua", "Noether",  
        ↪ "Ramanujan"]  
  
steps = 0  
for name in names:  
    steps = steps + 1  
    if name == "Hua":  
        print("found after", steps, "steps")  
        break
```

(`break` means: stop the loop now. Without it, the loop would keep checking names after finding Hua — correct, but wasteful, and we are in the business of counting waste.)

Run it. Then move "Hua" to the front of the list and run it again. Then to the back. Three runs, three costs. The cost of this algorithm is not a number; it depends on where the target sits. When we say *linear search takes up to n steps on a list of n items*, the “up to” is doing honest work — it is the worst case, the only promise the algorithm can keep no matter where the target hides.

3. Searching for what isn't there

Searching real lists, you don't know in advance whether the target is present. Predict before running: what does the program in §2 print if you search for "Euler"?

It prints nothing. The loop ends, no message, silence. The program is not wrong, exactly — but silence is a poor answer to a question. Here is a version that always answers, packaged so we can reuse it:

```
def linear_search(items, target):
    steps = 0
    for item in items:
        steps = steps + 1
        if item == target:
            return steps
    return -steps # negative means: not found, after this
    ↪ many steps
```

Two new things. `def` gives the procedure a name so we can call it on any list and any target without retyping it — that is all `def` means for now, though in Chapter 3 the ability to name a procedure will turn out to be more powerful than it looks. And `return` hands a value back to whoever called. Try it:

```
print(linear_search(["Ada", "Erdos", "Gauss", "Hua"], "Hua"))
print(linear_search(["Ada", "Erdos", "Gauss", "Hua"],
    ↪ "Euler"))
```

Notice what a miss costs: the *full* n steps, every time. You cannot declare something absent until you have looked everywhere. Absence is expensive — unless the list has structure. Which brings us to the chapter's main event.

4. Binary search, measured

Chapter 1's claim: if the list is *sorted*, you can find any target — or certify its absence — by repeatedly checking the middle and discarding half. Here is the claim as a program:

```
def binary_search(items, target):    # items must be sorted
    steps = 0
    lo = 0
    hi = len(items) - 1
    while lo <= hi:
        steps = steps + 1
        mid = (lo + hi) // 2        # // divides and drops the
        ↪ remainder
        if items[mid] == target:
            return steps
        if items[mid] < target:
            lo = mid + 1            # target can only be in
            ↪ the right half
        else:
            hi = mid - 1            # ... or the left half
    return -steps
```

Fifteen lines, and three small novelties: `items[mid]` picks out the entry at position `mid` (positions start at 0); `len(items)` is the length of the list; and `while` is a loop that runs as long as its condition holds — here, as long as the stretch between `lo` and `hi` hasn't shrunk to nothing. The pair `lo`, `hi` marks the region where the target could still be. Every pass through the loop, one of the last two lines cuts that region roughly in half. That sentence is the entire algorithm; the code is just the sentence with its sleeves rolled up.

Before running anything, do one search entirely by hand. Take the sorted list 2, 5, 8, 13, 21, 34, 55, 89 and search for 21, writing down `lo`, `hi`, and `mid` at every step. Then run the code and check that the machine agrees with your hand. If it doesn't, one of you is wrong, and finding out which is the best exercise in this lab.

5. The race

Now the measurement this lab has been building toward. We race the two searches on lists of increasing size, searching each time for the *last* element — the worst case for linear search:

```
for n in [10, 100, 1000, 10000, 100000, 1000000]:
    items = list(range(n))          # the sorted list 0, 1,
    ↪ ..., n-1
    target = n - 1                  # the last element
    print(n, linear_search(items, target),
    ↪ binary_search(items, target))
```

Run it and look at the table. The left column multiplies by 10 each row. The middle column does the same — linear search’s cost grows in step with the list, which is what “linear” means. The right column barely moves. A million items, about twenty steps.

Now sit with the table for a minute, because it is saying something strange. Each tenfold growth in the list adds the same *small, fixed* number of steps to binary search — roughly three or four per row. Multiplying the problem *adds* to the cost. You met a function with exactly this character in Book 2: the logarithm, the function that turns multiplication into addition. Binary search costs about $\log_2 n$ steps because each step halves the region, and “how many halvings from n down to 1” is precisely what $\log_2 n$ counts.

Deeper water. Extend the table in your head: at this rate, how many steps would binary search need on a *trillion* items? Write down your estimate, then test the rule “tenfold the items, add three or four steps” against it. Later, when you can prove the cost is $\lceil \log_2(n+1) \rceil$ exactly, check your estimate again. (And a question to keep: linear search never needed the list sorted. Binary search does. Sorting itself costs something — Chapter 6 asks how much, and whether the toll is worth paying. Spoiler: it depends on how many searches you plan to do, which is the kind of answer cost questions usually

have.)

Break it

Predict first. Run second. Explain any gap third.

1. In `binary_search`, change `lo = mid + 1` to `lo = mid`. Search the list 2, 5, 8, 13 for 21. Predict what happens before you run it — and be patient with your prediction. (When you understand what went wrong, you have understood why the `+ 1` is load-bearing, and you have met the most common way binary search is written incorrectly by professionals.)
2. Run `binary_search` on an *unsorted* list, searching for an element that is genuinely in it. Does it find it? Sometimes? Always? Never? Try several lists before answering. What exactly did sortedness buy?
3. In the race of §5, change the target from the last element to the *first* (target = 0). Predict both columns. Which algorithm's cost changed, and what does that tell you about worst-case versus best-case as ways of describing an algorithm?
4. (Harder.) Modify `linear_search` so that instead of returning step counts it returns the *position* of the target, or `-1` if absent. Then do the same for `binary_search`. You now have the two functions every programmer carries; Python's own `in` and `index` are these, in dress clothes.

Chapter 2: The Greedy Idea

★ ★ ★

In which the obvious move is wrong three times before it is right

Too much Saturday

Here is a problem you have actually had. It is Saturday. More things are happening than you can possibly do, and they overlap, so you have to choose. You want to do *as many as you can* — not the most important, not the most fun, just the largest number of things you can physically fit into the day without being in two places at once.

A real Saturday:

	starts	ends
run with Diego	9:00	10:00
robotics open-build (doors open 8:30)	8:30	1:00
help set up the party	10:00	11:00
soccer match	11:00	12:00
lunch shift	12:00	1:00
study group	1:00	2:00
birthday party	2:00	4:00
movie	4:00	5:00

Two things **conflict** if they overlap in time. (Finishing at 1:00 and starting at 1:00 is fine — you make it across town; call that back-to-back, not a conflict.) The question: what is the most things you can attend?

Try it yourself before reading on. Really — get a pencil, find a combination, then try to beat it. The answer for this particular Saturday is **seven** (everything except the all-morning robotics build), but finding seven is not the point. The point is the *rule* you used to find it — because in a minute we are going to discover that the obvious rule is wrong, that the right rule is not obvious at all, and that the gap between those two facts is one of the most powerful ideas in this book.

The obvious rule, and its humiliation

What did you do first? Most people reach for the same instinct: **do the thing that starts soonest, then the next thing that starts after that one finishes, and so on**. First come, first served. It is how a day naturally feels — handle the morning, then the midday, then the afternoon.

Watch it walk off a cliff. Earliest-starting thing first: the robotics build opens its doors at 8:30, earlier than anything else, so “earliest start” grabs it — and it runs until 1:00. Your whole morning is now gone, because the run, the setup, the match, and the lunch shift all conflict with the build. After it ends at 1:00 you can still catch study group, the party, and the movie. Total: **four things**. The obvious rule got you four. The pencil got you seven.

Sit with that, because it is the chapter in one stroke. The most natural strategy in the world — the one your hands reach for without being asked — is not slightly suboptimal. It misses *almost half the day*, and it does so while feeling completely reasonable at every step. “Do the next thing that’s starting” is wrong, and wrong in a way you would never notice without counting.

So the obvious rule fails. What’s the right one? Before we find it, let’s name what we’re doing, because it has a name and the name is worth more than the Saturday.

The strategy, named

The instinct — *build the answer one piece at a time, and at each step grab whatever looks best right now, and never reconsider* — is called **greedy**. It is not a single algorithm; it is a *strategy for designing algorithms*, a template. Point it at a problem and it proposes a procedure instantly: decide what “looks best” means, then grab repeatedly.

Chapter 1 gave you one clever algorithm, fitted to one problem by hand. Greedy is more ambitious — a *machine for generating algorithms* — and it generates them cheaply: one pass through your options, one decision per step, no memory of roads not taken, no agonizing. Of the two questions this book asks of everything, greedy asks *what does it cost?* before you’ve even checked whether it works. That cheapness is the seduction.

But “grab whatever looks best” is only a strategy once you say what *best* means — and the Saturday just taught you that the choice of “best” is the whole game. “Earliest-starting” is one definition of best, and it failed. So the real question of greedy is never *whether* to be greedy; it is **greedy about what?** Get that right and the strategy is perfect. Get it wrong and it misses half your Saturday with a straight face.

So let’s hunt for the right definition of best — the honest way, by trying the plausible ones and watching them fail until one survives.

Three definitions of “best,” two of them wrong

Best = starts earliest. Already dead. The robotics build starts as early as the run but holds you hostage for four hours. Starting early tells you nothing about when you’re *free* again, and being free again is what lets you do the next thing.

Best = shortest. A sharper idea — short things block less of your day, so grab the shortest available thing each time, then the

shortest of what's left. This *feels* right. It is wrong, and here is the smallest possible proof, just three events:

A runs 1:00–5:00. B runs 4:00–6:00. C runs 5:00–9:00.

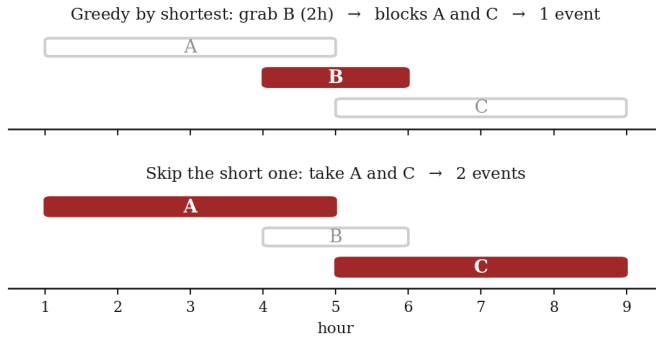


Figure 2.2 — Shortest-first's defeat: the short event B touches both A and C; grabbing it yields one where skipping it yields two.

Figure 2.2 — Shortest-first's defeat: the short event B touches both A and C; grabbing it yields one where skipping it yields two.

B is the shortest (two hours). Grab it greedily and you've knocked out *both* A and C — B overlaps the tail of A and the head of C — so you attend **one** thing. But skip the tempting short one and take A and C, which don't overlap each other, and you attend **two**. Shortest-first measured the event itself and ignored what the event *touches*. A short thing wedged between two others can be more destructive than a long thing off in the corner.

Best = finishes earliest. Now think about what actually went wrong in both failures. The robotics build was bad because it kept you busy *late*. B was bad because it straddled two slots. In both cases the damage was about when you become *available again*. So make availability the whole criterion: **always do the thing that finishes soonest among the things you can still attend**. Finishing early keeps the most future open — of everything you could do next, the earliest-finisher leaves the largest runway for everything after.

Run it on the Saturday. The run finishes first (10:00) — take it. Among things starting at 10:00 or later, setup finishes first (11:00)

— take it. Then the match (12:00), the lunch shift (1:00), study group (2:00), the party (4:00), the movie (5:00). **Seven things.** The pencil’s answer, produced by a rule.

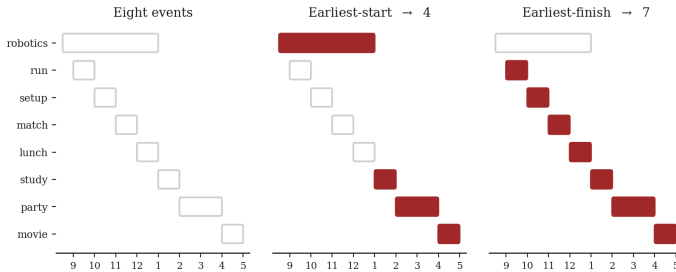


Figure 2.1 — The overbooked Saturday three ways: the eight events; earliest-start grabbing the all-morning build and salvaging four; earliest-finish fitting seven.

Figure 2.1 — The overbooked Saturday three ways: the eight events; earliest-start grabbing the all-morning build and salvaging four; earliest-finish fitting seven.

Earliest-finish-first. It is not what anyone reaches for — it feels almost backward, choosing by when things *end* rather than when they start or how long they take — and that is exactly why it had to be discovered rather than assumed. The obvious rule was obvious and wrong. The right rule is strange and right. That is the texture of greedy: the strategy is simple, but *which* greedy is the entire problem, and the answer is often counterintuitive.

(One more plausible rule deserves a mention because it’s so tempting: *best = fewest conflicts*, grab the thing that bumps into the fewest others, avoid the troublemakers. It also fails — Problem 3.3 has a verified case where it gets 2 where earliest-finish gets 4 — and for the same reason: “how many things you conflict with” still isn’t “when you free up.”)

But does it always work?

Earliest-finish-first got seven on *this* Saturday. But one Saturday is one input, and Chapter 1 taught the difference between a demonstration and a guarantee. Shortest-first also looked fine until the

three-event trap. How do we know earliest-finish doesn't have its own trap — some devious calendar where it quietly loses?

We need a proof that it works on *every* possible schedule, and the proof is a thing of real beauty — the standard shape of all greedy correctness arguments — so it is worth seeing in full.

Suppose a rival hands you the *best possible* schedule for some day — found by magic, by trying every combination, by whatever means; call it perfect. Line up the rival's schedule and greedy's, both in time order, and walk along them until the *first* place they differ. There, greedy attends some event G and the rival attends a different event R. Now the key fact: at that moment greedy chose the earliest-finishing event among everything compatible with the (identical) shared start — and R was one of the options greedy was choosing among. So **G finishes no later than R.**

Here is the move. In the rival's "perfect" schedule, cross out R and write in G. Does anything break? Everything before the swap was identical — no conflict there. Everything the rival did *after* R started after R finished, which is after G finished (G finishes no later) — no conflict there either. The rival's schedule is still valid, still has exactly as many events, and now agrees with greedy one step longer.

Repeat. Each swap is free and pushes the agreement forward one more event. When you run out of disagreements, the rival's schedule has *become* greedy's — without ever losing a single event. So greedy's schedule has just as many events as the perfect one. Greedy **is** perfect. This is the **exchange argument**: you don't prove greedy is brilliant; you prove that *any other answer can be massaged into greedy's without getting worse*, so greedy's can't be beaten.

That is what was missing from the failed rules. There is no exchange argument for shortest-first — try to swap a shortest-first choice into a better schedule and it breaks, because "shortest" guaranteed nothing about timing. The presence or absence of a working exchange argument is the exact, provable line between a greedy rule you can trust forever and one that fails on the third example. "Does greedy work here?" is never a vibe. It is a question about whether

the swap goes through, settled at a desk.

Here is what makes this particular problem a favorite of the people who study these things. You could solve it by **brute force** — try every possible combination of compatible events and keep the largest — and that always works, at ruinous cost (Chapter 1’s exponential wall). You could solve it by the **table method** of Chapter 4, which also always works, at reasonable cost. And then there is this: a one-line greedy rule, the cheapest possible procedure, that *also* gets the exact right answer — provably, on every input. Three methods, three costs, one answer. The marvel is not the algorithm; the algorithm is almost too simple to be impressed by. The marvel is the *proof* — that a choice made with no thought for the future, grabbing the earliest finisher and never looking back, can be shown to equal the best that any amount of foresight could achieve. When a reckless local rule provably produces a global optimum, something structural is going on underneath, and the exchange argument is how you see it.

A human moment

The first greedy algorithm ever *proved* correct was invented before computers existed, to wire a country. In 1926 the electric company of western Moravia asked the mathematician Otakar Borůvka for the cheapest way to connect dozens of towns with power lines. Borůvka saw the real problem under the engineering one — towns and possible lines form an abstract web of dots and connections, the graph waiting for you in Chapter 5 — invented a greedy procedure for it, and proved, with an argument of exactly the swap-and-nothing-breaks kind you just read, that greed could not be beaten. No machine ran his algorithm in his lifetime’s main work; he published it in Czech, in an electrotechnical journal, and the world’s computer scientists rediscovered it decades later. The order of events is the lesson: the *proof* is the technology. Machines came later and merely

made it fast.

Deeper water: greedy stays ahead

The swap story, formalized, is an induction. List greedy's chosen events $g_1, g_2, \dots, g_k$ and any rival's $s_1, s_2, \dots, s_m$, both in finish-time order. Claim: for every i up to the shorter length, **$\text{finish}(g_i) \leq \text{finish}(s_i)$** — greedy's i -th event ends no later than anyone's i -th.

Base: g_1 is the earliest-finishing event of all, so $\text{finish}(g_1) \leq \text{finish}(s_1)$. *Step:* assume $\text{finish}(g_i) \leq \text{finish}(s_i)$. The rival's next event s_{i+1} starts after $\text{finish}(s_i) \geq \text{finish}(g_i)$, so s_{i+1} was *available* to greedy when it picked g_{i+1} — and greedy picks the earliest finisher among the available, so $\text{finish}(g_{i+1}) \leq \text{finish}(s_{i+1})$. ■

Greedy is never behind at any position. The conclusion that the rival can't have *more* events than greedy is now one observation away — the Diamond asks you for it, and for an honest account of where back-to-back events (the $\geq$ versus $>$ of conflict) enter. Compare this induction's skeleton — true at the start, preserved each step, decisive at the end — with Chapter 1's loop invariant. It is the same correctness trick, second appearance. There will not be a third.

Deeper water: the best greedy algorithms are theorems in disguise

Interval scheduling is the gateway, not the summit. The greedy algorithms that working mathematicians actually find beautiful are the ones where the “grab the best-looking thing” rule looks not just non-obvious but *reckless* — and is exactly optimal anyway, because a deep structural fact about the problem is secretly guaranteeing it. The crown jewel waits for you in Chapter 5, once you have graphs. The problem: connect a set of towns with the least total length of cable (this is Borůvka's problem, from the human-moment above — the first greedy algorithm ever proved correct). The rule, called **Kruskal's algorithm**: repeatedly take the *cheapest remaining cable that doesn't*

form a loop, and stop when everything is connected. It feels like it can't possibly work — surely grabbing cheap cables greedily paints you into a corner — yet it produces the provably cheapest network, every time.

Why? Because of a fact called the **cut property**: split the towns into any two groups, look at all the cables crossing between the groups, and the *cheapest* such cable must belong to the best network. The proof is the exchange argument once more — if the best network didn't use that cheapest crossing cable, it used a pricier one to bridge the two groups, and you could swap the cheap one in and save money, a contradiction. The same swap-and-nothing-breaks move that tamed your Saturday, now revealing a structural law about graphs.

So here is a rough ranking of the classic greedy algorithms by how much the greedy step exceeds mere common sense — by how deep the hidden theorem is. Minimum spanning trees (the cut property) sit at the top; interval scheduling (the exchange argument you just saw) next; then Huffman coding (the optimal way to compress a message, which you could nearly discover yourself with the tools of Chapter 3); then Dijkstra's shortest paths, whose greedy step is safe only because distances can't be negative — which you *will* meet in Chapter 5, where it is this very exchange argument wearing weights. If any of this is irresistible, you do not have to wait for the chapters; the names are above, and the internet is patient. The point of this book is to make you the kind of reader who can chase them.

The same instinct, somewhere it betrays you

So greedy, pointed correctly, is provably perfect on scheduling. Lulled by that, you might start to trust the *instinct* — grab the locally best thing — as if it were generally safe. It is not, and the

cleanest way to feel the danger is to watch the exact same instinct fail on a problem where it looks even *more* obviously fine than “earliest start” did.

A problem from a world before your time, which is part of the point. Long ago, people paid for things with metal coins of fixed values, and a cashier making change had to hand back a given amount using as *few coins* as possible. The greedy rule is the one every old cashier used without thinking: hand over the biggest coin that doesn’t overshoot, then the biggest that fits in what’s left, and so on. For the coins those societies actually used, this works perfectly — it always produces the fewest coins.

Now invent a country. Its mint, for whatever strange reason, makes coins worth exactly **1, 7, and 10**. You owe someone **14**. Apply the cashier’s instinct: the biggest coin that fits is 10, so hand it over; now you owe 4, and only 1-coins fit, so four of them. **Five coins:** $10 + 1 + 1 + 1 + 1$.

The right answer is **two coins:** $7 + 7$.

Perform the autopsy, because it is the same disease as earliest-start in a new body. The greedy choice was not stupid — 10 *was* the biggest coin that fit. But grabbing it destroyed a better future greedy never even priced: the $7 + 7$ plan was sitting right there, worth less per coin at the first step and worth more in the end. Greedy optimized the step and mortgaged the rest. It is blind in exactly one direction — forward — and it never looks back to notice what it threw away.

Two lessons, and the second is the deep one:

1. *The same instinct succeeds on one problem and fails on another.* Earliest-finish-first was greedy and perfect; grab-the-biggest-coin is greedy and, in this country, wrong by 150%. So “greedy” is never the answer by itself.
2. *Whether greedy works is a property of the problem, not of greed.* And you already know the test: does the exchange argument go through? For scheduling it did. Try it here — swap greedy’s 10

into the $7 + 7$ plan — and it *breaks*: $10 + 7$ isn't 14, and patching with 1-coins makes the answer worse. The swap has a cost. That broken swap is the precise, structural reason greedy fails on these coins and succeeds on the Saturday. Not a vibe. A swap that does or doesn't go through.

File this country's coins in your head — **14, paid badly with five coins when two would do** — because this wound does not heal in this chapter. It is the standing reminder that *locally best can mortgage the global future*, and it returns, by name, twice more: in Chapter 4, where a method that refuses to commit early finds the $7 + 7$ by *pricing every option instead of grabbing one*; and in Chapter 8, in continuous disguise, when an algorithm walking downhill on a landscape commits its way into the wrong valley for exactly this reason. The coins are not really about coins. They are about the price of not looking back.

Deeper water: how the coins are caught, and how honest coins are certified

“Greedy fails for *some* amount” looks like a claim needing an infinite search — infinitely many amounts. It isn't, by a small theorem of Kozen and Zaks: **if greedy change ever fails, it already fails at some amount below the sum of the two largest coins**. For $\{1, 7, 10\}$ that bound is 17, and the smallest failure, 14, sits under it. Read the contrapositive, the useful direction: *audit every amount below the bound, and if greedy survives all of them, it is correct for all infinitely many amounts forever*. Your Lab 2 audit loop on $\{1, 5, 10, 25\}$ up to 34 is therefore not evidence — it is a **proof**, a finite certificate of an infinite claim, run by machine. Problem 3.5 has you write it down. (Notice the *kind* of result: infinity fenced into a finite check — Chapter 1's lower bound fenced what's possible; this fences where failure can hide.)

Deeper water: a door, ajar

Scheduling is not a lucky one-off. There is a whole theory — of structures called *matroids* — characterizing a large family of problems on which greedy is *guaranteed* optimal, all by essentially the exchange argument you saw. That a single proof shape could be bottled and poured over infinitely many problems is the kind of thing that happens in mathematics more often than seems fair. The door stays ajar; this book does not walk through it.

Where this leaves us

Greedy is a strategy, not a truth. The obvious version of it failed your Saturday by half; a subtler version — earliest-finish-first — was provably perfect; the very same instinct, on a strange country's coins, failed again. What separated success from failure was never how clever the rule sounded. It was whether the exchange argument went through: whether any other answer could be massaged into greedy's at no cost. When it can, greedy is unbeatable. When it can't, greedy lies with total confidence.

And the failures taught more than the success. *Which* greedy is the whole problem, the right choice is often the counterintuitive one, and “locally best” can quietly bankrupt the future — the coins' lesson, filed for two later chapters.

And there is a hint of the next chapter hiding in the exchange argument, if you look. After greedy picks the first event — the earliest finisher — what's left is *the same problem on the events that remain*: attend the most non-overlapping things from a smaller pile. Pick the earliest finisher of *those*, and you face a smaller pile again. The problem contains a smaller copy of itself, and greedy is really solving that smaller copy, and the smaller copy inside that, all the way down. That structure — a problem defined in terms of a smaller version of itself — is the engine of the next chapter, and it is stranger and deeper than greedy. Instead of committing to one option at

each step and never looking back, you solve a problem by handing a smaller copy of it to *yourself*.

Problem Set 2

★ ★ ★

Lab 2 (The Cost of Confidence) is the Computational Lab tier. The audit loop you build there is required equipment for problems 3.5 and 2.5.

Fluency

1.1 On the chapter's Saturday, execute earliest-finish-first by hand, writing at each step which events are still compatible and which gets chosen. Confirm you reach seven.

1.2 Execute earliest-start-first on the same Saturday by hand and confirm it reaches only four. Which specific events did grabbing the robotics build cost you?

1.3 Draw the three-event shortest-first counterexample (A 1–5, B 4–6, C 5–9) as bars on a timeline. Mark which event shortest-first grabs and which two it kills.

1.4 Run greedy change by hand in the country of $\{1, 7, 10\}$ for the amounts 14, 20, 21, and 13. For each, also find the true minimum by inspection. Where does greedy lose, and by how much?

1.5 Run greedy change for the *same* amounts using ordinary US-style coins $\{1, 5, 10, 25\}$. Does greedy ever lose here? (The chapter's deep water explains why not.)

Practice

2.1 Design your *own* overbooked day — at least eight events, at least three genuine conflicts — and find the maximum attendable set by earliest-finish-first. Then verify by hand that no larger set exists. (Trading days with a classmate and checking each other’s answers is the real exercise.)

2.2 A single video-editing computer can render one project at a time; each queued project has a fixed start and end time it’s scheduled for. From ten projects you invent, find the largest number that can run without overlap. Is this the same problem as the Saturday, in disguise? Say exactly what maps to what.

2.3 A streaming service can show one program in its 8 p.m. slot per night, and each pitched show wants a run of consecutive nights. Recast “air the most shows this month” as the Saturday problem (nights instead of hours) and describe the winning rule in one sentence.

2.4 The country of squares mints coins worth 1, 4, and 9. Compute greedy’s answer and the true best for amounts 12 and 13. Then, using your Lab 2 audit loop, list every amount up to 30 where greedy loses.

2.5 Using your Lab 2 audit loop, design a three-coin system $\{1, a, b\}$ whose *smallest* greedy failure is as large as you can make it. Report the system and the failure. (Class competition: largest smallest-failure wins.)

Concept & Counterexample

3.1 Claim: “for scheduling, *longest*-first can’t be optimal, but it’s a decent rule.” Refute it with the smallest counterexample you can build, and explain in one sentence what longest-first ignores. (You are re-running the chapter’s hunt with a new wrong rule.)

3.2 Claim: “earliest-finish-first works only because the events

have different lengths; if all events were exactly one hour long, earliest-start-first would tie it.” Decide, then prove or refute. (If you believe it, find the exchange argument; if not, the counterexample needs care precisely *because* all bars are equal length.)

3.3 Fewest-conflicts, refuted. The rule “attend the event that conflicts with the fewest others, then repeat” feels wise. Verify it fails on this exact instance — events (start, finish): chain (1,3), (3,5), (5,7), (7,9); blockers (2,4), (2,4), (2,4), (6,8), (6,8), (6,8). Count each event’s conflicts, run fewest-conflicts-first, run earliest-finish-first, and report both scores. Then say in one sentence why “fewest conflicts” was the wrong measure of trouble.

3.4 Claim: “greedy change fails on {1, 7, 10} because 7 doesn’t divide 10.” Test the divisibility theory: find a coin system where no coin divides the next yet greedy still succeeds on every amount (audit it!), *or* a system with clean divisibility that fails. Then state carefully what divisibility actually guarantees — show that when each coin divides the next ({1, 5, 25}, say), greedy is optimal, and find where your argument uses the divisibility.

3.5 The finite certificate. Run your audit loop on US coins {1, 5, 10, 25} for every amount from 1 to 34. Combined with the Kozen–Zaks theorem (deep water; quote it, don’t prove it), write a five-sentence proof that American greedy change is optimal for **every** amount, forever. Be precise about what the computation established and what the theorem added — neither alone suffices.

Diamond

4.1 Greedy stays ahead, completed. Starting from the deep-water lemma $\text{finish}(g_i) \leq \text{finish}(s_i)$: (a) Prove the lemma’s base case and inductive step in your own words, drawing the timeline picture that makes the step obvious. (b) Derive the conclusion: greedy’s schedule is at least as large as any valid schedule. (Suppose the rival has $m > k$ events; apply the lemma at position k and contradict greedy’s

stopping rule — what *exactly* does greedy’s having stopped after g_k assert about the events that remain?) (c) The boundary audit: our convention says finishing at 1:00 and starting at 1:00 don’t conflict. Locate every place in (a) and (b) where $\leq$ versus $<$ matters, and state whether the theorem survives if back-to-back events *are* counted as conflicts. (d) One sentence: where would this proof break if you tried to run it on the $\{1, 7, 10\}$ coins instead of on intervals?

Computational Lab

5.1 Lab 2 in full, predictions written before runs. The $\{1,7,10\}$ failure pattern you conjecture in Break-it 2 should be stated as a precise sentence and tested on the range 100–130 before running it.

Teach It

6.1 Give someone the overbooked-Saturday table and ask them to attend as many things as possible, watching what rule they use. They will almost certainly reach for “earliest start” or “the ones I like most.” Let them commit to an answer, then reveal earliest-finish-first and have *them* check it beats their rule. Three sentences: what was their first instinct, and what convinced them it was wrong?

6.2 Teach the $\{1,7,10\}$ disaster with actual objects — coins, beans, Lego of two sizes. Rules: they must make the greedy mistake *themselves* first (set up the 14 so their hands grab the 10), and you may not say “greedy” or “algorithm” until after the $7 + 7$ reveal. Write three sentences: what did their face do, and what explanation finally landed?

Lab 2: The Cost of Confidence

★ ★ ★

Accompanies Chapter 2. The chapter introduced a strategy — at every step, take the option that looks best right now — and claimed it is sometimes perfect and sometimes confidently wrong. In this lab you build both outcomes with your own hands.

1. The cashier's algorithm

Chapter 2 opened behind a cash register: owe someone 63 cents, and your hands know the algorithm — give a quarter, another quarter, a dime, three pennies. Take the biggest coin that fits; repeat. Here it is in Python:

```
def greedy_change(coins, amount):    # coins listed largest
    ↪ first
    used = []
    for coin in coins:
        while amount >= coin:
            amount = amount - coin
            used.append(coin)
    return used

print(greedy_change([25, 10, 5, 1], 63))
```

One new tool: `used.append(coin)` adds an item to the end of the list `used` — the program is keeping a record of its decisions, which is what `used = []` (an empty list, a blank ledger) is for. Everything else you own already: the `for` walks the coin types from largest down; the `while` spends each coin type as long as it still fits.

Run it on a few amounts. Six coins for 63. Count by hand and check: can you beat six?

You can't — with these coins, the greedy answer is the best answer, for 63 and for every other amount. (That this is *true* is believable; *why* it's true is genuinely subtle, and the chapter was honest that we are not proving it. We are about to see why such a proof would have to exist: because the same instinct, on different coins, fails.)

2. The kingdom of Sevenia

Chapter 2's kingdom mints three coins: 1, 7, and 10. You owe 14. Before running anything, do it greedily in your head — and then do it *well* in your head.

```
print(greedy_change([10, 7, 1], 14))
```

Five coins, says your program, with complete confidence. The right answer is two: $7 + 7$. The code is not buggy. It did exactly what greedy means — and greedy is wrong here. The grab-the-biggest instinct took the 10 because the 10 looked best *at that moment*, and taking it destroyed the $7 + 7$ future that was quietly available. Greedy never looks back, and never apologizes.

3. Catching every failure

One failure could be a fluke. Let's audit Sevenia completely. For three-coin systems we can find the *true* best answer by sheer brute

force — trying every possible combination. This is “doing all the work,” the thing algorithms exist to avoid, and we do it here deliberately, as a measuring stick:

```
def best_change_3(c1, c2, c3, amount):    # c3 must be 1
    best = amount                        # amount pennies
    ↪ always works
    for a in range(amount // c1 + 1):
        for b in range(amount // c2 + 1):
            for c in range(amount // c3 + 1):
                if a*c1 + b*c2 + c*c3 == amount:
                    if a + b + c < best:
                        best = a + b + c
    return best

print(best_change_3(10, 7, 1, 14))
```

Three loops, one inside another: every count of 10s, against every count of 7s, against every count of 1s. Nothing escapes. Now the audit — every amount from 1 to 30, greedy versus truth:

```
for amount in range(1, 31):
    g = len(greedy_change([10, 7, 1], amount))
    b = best_change_3(10, 7, 1, amount)
    if g != b:
        print(amount, "greedy:", g, "best:", b)
```

Run it and read the list of amounts where greedy loses. Look for a pattern in them before the Break-it section asks you about it. This little audit loop — confident method versus exhaustive truth on every small case — is a tool you will use for the rest of your life, and not only on coins.

A note for later: the brute force was affordable here because the kingdom was small. Watch what `best_change_3` costs as

amounts grow — three nested loops multiply. Chapter 4 will find the best answer *without* trying everything, and will do it using the failure you just catalogued. The wound stays open until then.

4. Scheduling a day

The chapter's second arena: from a pile of events that may overlap, attend as many as possible. The winning rule was *earliest finish first* — always take the event that ends soonest among those you can still attend. To build it we store each event as a two-item list `[finish, start]`, finish *first*, for one sneaky reason: Python's `.sort()` arranges a list of pairs by their first entries, so putting the finish time first makes plain sorting line the events up exactly the way the rule wants. (`.sort()` itself is a sealed instrument for now — Chapter 6 opens the box and asks what it costs.)

```
def schedule(events):                                # each event is
    ↪ [finish, start]
    events.sort()                                    # earliest finish now
    ↪ comes first
    chosen = []
    free_at = 0
    for e in events:
        finish = e[0]
        start = e[1]
        if start >= free_at:                          # does it begin after
            ↪ I'm free?
            chosen.append([start, finish])
            free_at = finish
    return chosen
```

Try it on the chapter's example day — times are hours on a 24-hour clock, and each event is `[finish, start]`:

```
day = [[10, 9], [11, 9], [12, 11], [13, 10], [14, 12], [16,  
↪ 13], [15, 14]]  
print(schedule(day))
```

Trace the result against a drawing of the seven events on a timeline (really draw it — the timeline is the proof’s natural habitat, and you’ll want it for the Diamond).

Now the real assignment: write out *your own actual week* — classes, practices, the things you do — as [finish, start] pairs, and run the scheduler on it. Then deliberately overbook a fantasy Saturday with twelve overlapping events and see what the rule salvages.

Break it

Predict first. Run second. Explain any gap third.

1. In §1, hand `greedy_change` the coins *smallest first*: `greedy_change([1, 5, 10, 25], 63)`. Predict the exact output before running. What does the result say about the phrase “take the best-looking option” — best-looking *according to whom*?

2. From your §3 audit of Sevenia: extend the range to 100 and collect every amount where greedy loses. State a pattern (“greedy fails exactly when the amount looks like ___”) and then *test your pattern* by predicting the failures between 100 and 130 before running that range.

3. US coins minus the nickel: a country with only [25, 10, 1]. Predict — does greedy stay perfect, or fail somewhere? Commit to an answer and a reason, then audit amounts 1 to 100. (If it fails, find the smallest failure and explain, in the style of Sevenia’s 14, exactly which future the greedy choice destroyed.)

4. In `schedule`, change `start >= free_at` to `start > free_at` and rerun your week. Predict what kind of event pair gets wrongly rejected. One character — what real-world scheduling assumption does it encode?

5. Sabotage the rule itself: store events as `[start, finish]` so that `.sort()` lines them up by *start* time, adjust the two indexing lines to match, and rerun the chapter's example day. Predict the output first. You have just built, in working code, one of the counterexamples from the chapter — say which one.

Chapter 3: The Procedure That Calls Itself

★ ★ ★

In which a puzzle is solved by handing it to a smaller you

Three pegs, a rule, and a tower

The puzzle: three pegs stand in a row. On the left peg sits a tower of disks, largest on the bottom, smallest on top. Move the whole tower to the right peg, under two laws: move one disk at a time, and never place a larger disk on a smaller one.

Get three coins of different sizes and play — this chapter is useless watched and easy played.

One disk: move it. One move.

Two disks: the small one is in the way, so park it on the middle peg, move the big one to the right, drop the small one back on top. Three moves.

Three disks. Fumble for a minute — everyone does — and then catch yourself doing something remarkable. To free the big disk, you must first move the *two-disk tower* above it out of the way, onto the middle peg. But you just learned how to move a two-disk tower — you do it again, aiming at the middle. Then the big disk crosses. Then you move the two-disk tower once more, middle to right. Done in $3 + 1 + 3 = 7$ moves, and the three-disk solution turned

out to contain the two-disk solution, *twice*, as a part.

That observation, taken seriously, solves the puzzle for every tower at once. To move n disks from here to there:

move the top $n - 1$ disks to the spare peg — *somehow*; carry disk n across; move the $n - 1$ disks from the spare onto it — *somehow*.

And the scandal is what fills in “somehow”: **the same instructions, for a smaller tower**. The procedure delegates its hard part to itself. A four-disk tower asks the three-disk procedure for help; the three-disk procedure asks the two-disk; the two asks the one; and the one-disk case needs no help at all — pick it up, put it down.

This is **recursion**: a procedure defined partly in terms of itself, on smaller cases. It is the deepest idea in this book, and your first reaction to it should be suspicion.

Why this is not cheating

“To move a tower, move a tower” sounds like the dictionary that defines a word using the word. The suspicion is correct in general and wrong here, and the difference is worth nailing down, because it is the difference between recursion and nonsense.

Two things rescue the definition. First, every self-call is *strictly smaller*: n hands its work to $n - 1$, never to n or upward. Second, the descent has a floor: at $n = 1$ the procedure does its work directly, asking nobody. Smaller-every-time plus a floor means the chain of delegations cannot go on forever — $n, n - 1, n - 2, \dots$ must hit 1, the floor acts, and the answers come climbing back up the chain. A recursion with both properties is as legitimate as any procedure in this book; a recursion missing either is an infinite regress, and in Lab 3 you will build one on purpose and watch the machine refuse it.

If “smaller every time, with a floor” sounds familiar, it should. It

is **induction**, from Book 2, read in the opposite direction. Induction *climbs up*: the statement holds at 1, and each rung passes truth to the next. Recursion *climbs down*: the case at n hands its work toward 1, where work becomes possible. They are the same staircase. And that means proving a recursive procedure correct is induction with no disguise at all: *the procedure is correct for $n = 1$ (look at it); and if it is correct for $n - 1$, then the three-line plan — clear, cross, restack — is legal and correct for n* . Notice your old friend: true at the floor, preserved at each level, decisive at the top. Same trick as Chapter 1's invariant and Chapter 2's stays-ahead. Still the only trick.

The bill

Two questions, always. Correctness is settled. Now the cost, in the declared currency of *moves*. Let $M(n)$ be the moves our procedure makes on n disks. Read the three-line plan and the bill writes itself:

$$M(n) = 2 \cdot M(n - 1) + 1, \text{ with } M(1) = 1.$$

A cost equation that refers to itself — of course it does; the procedure refers to itself. Such an equation is called a **recurrence**, and the cheapest way to understand one is to make it produce numbers:

n	1	2	3	4	5	6	7	8
$M(n)$	1	3	7	15	31	63	127	255

Stare until the pattern declares itself: each entry is one less than a power of two. $M(n) = 2^n - 1$. (Conjectured from the table; *proved*, in two lines of induction, in Fluency 1.3 — by now you know that a table is a conjecture machine, not a proof.)

Sit with what this kind of growth means, because it is the opposite of Chapter 1's logarithm. There, multiplying the problem *added* to the cost. Here, adding one disk *doubles* the cost. The logarithm and the exponential are the two ends of the book's scale — the curve you want to be on, and the wall. Ten disks, about a thousand moves:

an evening. Twenty disks, about a million: a sleepless month at one move per second. Sixty-four disks — hold that thought.

A human moment

The Tower of Hanoi is not ancient. It was invented in 1883 by Édouard Lucas — a French number theorist with serious theorems to his name — and sold in toy shops, credited to a mysterious “Professor N. Claus (de Siam), mandarin of the College of Li-Sou-Stian.” Read those names again slowly: *N. Claus de Siam* is an anagram of *Lucas d’Amiens* — Lucas of Amiens, his hometown — and “Li-Sou-Stian” rearranges the Lycée Saint-Louis, where he taught. The toy shipped wrapped in a legend, also written by Lucas: in a temple at Benares, priests labor over a tower of 64 golden disks, and when the last disk falls, the world ends.

The legend is marketing. Its *arithmetic* is real, and it is the whole reason to tell the story. By your own formula the priests need $2^{64} - 1$ moves — 18,446,744,073,709,551,615 of them. At one move per second, day and night, that is about 585 *billion* years: more than forty times the age of the universe. Lucas knew exactly what he was doing — he dressed the scariest curve in mathematics as a children’s toy, signed it with a puzzle, and let the customers discover the apocalypse by themselves. The pseudonym was solved; the curve never softened. When someone tells you a process is “just like Hanoi but bigger,” check which axis got bigger before you agree to wait.

Two halves are better than one smaller

Hanoi’s recursion shrank the problem by *one* — n hands work to $n - 1$ — and paid an exponential bill. Watch what happens to the bill when a recursion shrinks the problem by *half*.

The arena is sorting, and the lab equipment is a deck of cards.

Take twenty, shuffled. The recursive plan, called **merge sort**:

split the pile into two halves; sort each half — *somehow* (you already know what fills in “somehow”); **merge** the two sorted halves into one sorted pile.

The floor: a pile of one card is already sorted.

Everything rests on the merge, so do it with your hands until it is boring. Two sorted piles face-up before you. The smallest card of all must be one of the two top cards — *compare just those two*, take the smaller, repeat. The merge never searches; it only ever looks at two cards and takes one, so combining piles of total size n costs about n cheap steps. Sortedness, once again, is purchased information — each pile’s order is exactly what lets one glance speak for the whole pile. (Chapter 1 said structure is bought in advance and collected later. Merge sort is a *pyramid scheme of that idea that actually works*: each level’s sortedness is collected from the level below, all the way down to single cards, which are sorted for free.)

Adding up the tree

Now the bill. Currency declared: element-steps, one for each time the merge touches a card. The cost $C(n)$ of sorting n cards:

$C(n) = 2 \cdot C(n/2) + n$ — two half-size sorts, plus a merge that touches everything once.

Don’t solve it with algebra. *Draw it*. The full job sits at the top: one box of size n . It opens into two boxes of size $n/2$; each opens into two of size $n/4$; and so on down to single cards. Now add up the merging cost **level by level**:

- Top level: one merge touching n cards $\rightarrow n$.
- Next: two merges, each touching $n/2 \rightarrow n$.
- Next: four merges of $n/4$ each $\rightarrow n$.

- ... every level totals exactly n .

So the whole bill is n per level, times the number of levels — and the number of levels is how many times n halves down to 1, which Chapter 1 taught you to name: $\log_2 n$. Total:

$$C(n) \approx n \log_2 n.$$

A 7th grader's arithmetic — sizes halve, level-sums stay flat, count the levels — and it tames one of the most important recurrences in computer science. Feel the number: a million cards cost about $1,000,000 \times 20 =$ twenty million element-steps. Your laptop does that between two frames of a video. The obvious ways of sorting (Chapter 6 examines them, and people, sorting cards) cost around n^2 — for a million cards, a *trillion* steps, fifty thousand times more. Same problem, same answer, and the entire difference is that one procedure halves and the other plods. Hanoi shrank by one and hit the exponential wall; merge sort shrank by half and got the logarithm's discount. *Where a recursion sends its work is the whole game.*

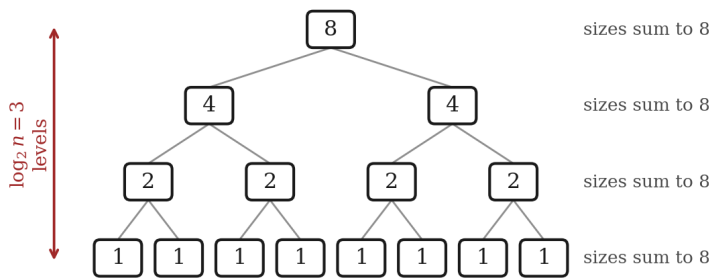


Figure 3.1 — The merge sort tree: every level's merges touch n cards, and there are $\log_2 n$ levels.

Figure 3.1 — The merge sort tree: every level's merges touch n cards, and there are $\log_2 n$ levels.

Deeper water: the master theorem

The level-by-level tree settles any recurrence of the shape $T(n) = a \cdot T(n/b) + n^c$ — a subproblems, each a b -th of the size, plus n^c of local work — and the tree always tells one of exactly three stories. Compute the level sums: each level down, the work is multiplied by a/b^c .

- $a/b^c < 1$: level sums *shrink* geometrically; the top level dominates; $T(n) \approx n^c$.
- $a/b^c = 1$: level sums *stay flat* (merge sort's case, $a = b = 2$, $c = 1$); total = work-per-level $\times$ levels; $T(n) \approx n^c \log n$.
- $a/b^c > 1$: level sums *grow*; the bottom — the swarm of tiny cases — dominates; $T(n) \approx n^{\log_b a}$.

That is the master theorem, minus rounding ceremony: every divide-and-conquer bill is a geometric series, and a geometric series is ruled by its ratio. (Book 2's geometric series chapter was for this, among other things.) Test yourself: binary search is $T(n) = 1 \cdot T(n/2) + 1$ — which story, and does it agree with Chapter 1?

Deeper water: the stack is a physical object

When a procedure calls itself, the machine must remember every unfinished outer job while the inner one runs — Hanoi at $n = 5$ is paused, mid-sentence, while $n = 4$ works, which is paused while $n = 3$ works... The machine keeps these suspended jobs in a pile called the **call stack**: each call goes on top, finishes, comes off. (A pile where only the top is touched will get a name and a starring role in Chapter 5.) In Lab 3 you print each call indented by its depth, and the stack stops being a metaphor — you watch it breathe in and out on the screen.

The stack is physical: it occupies real memory, and it is finite. Python ships with roughly a thousand levels of patience, after which it raises `RecursionError` rather than drown — you will trigger this on purpose in the lab. And the floor-and-shrink rule from this chapter has a thunderclap hiding behind it: checking that *your* recursion terminates is usually easy, but could a procedure exist that inspects **any** program and decides whether it terminates? Hold the question; it is the Epilogue's, and the answer is the strangest theorem you will ever meet.

Where this leaves us

A procedure may legitimately call itself, provided it always calls *downward* and the descent has a *floor* — and its correctness proof is then induction wearing work clothes. Its cost obeys a recurrence, and the recurrence is read from a tree: Hanoi's tree doubles forever sideways and bills $2^n - 1$; merge sort's tree halves quickly downward and bills $n \log_2 n$. Same idea, opposite fates, decided entirely by where the recursion sends its work.

But there is a third fate, stranger than either, and Chapter 4 walks into it. Some recursions send their work *to the same place twice* — ask a perfectly honest question, and then, having forgotten the answer, ask it again, and again, millions of times. Such a recursion is correct, exponential, and *wasting almost all of its effort on repetition* — which means the exponential wall, for once, is made of paper. The fix is one idea: write the answers down. And the first thing the fix will do, in that chapter, is walk back into Sevenia and make change for 14 — correctly.

Problem Set 3

★ ★ ★

Lab 3 (The Visible Staircase) is the Computational Lab tier. The Diamond wants the lab's counting program as a check on part (a).

Fluency

1.1 With three coins, perform the 7-move solution for three disks, writing each move as “disk d: peg $\rightarrow$ peg.” Then, without playing, use the three-line plan to *write out* all 15 moves for four disks. Check your list's move count against the table.

1.2 Extend the $M(n)$ table to $n = 12$ using the recurrence (not the formula). Then verify $M(12)$ against $2^{12} - 1$.

1.3 Prove $M(n) = 2^n - 1$ by induction from the recurrence $M(n) = 2M(n-1) + 1$, $M(1) = 1$. Two lines suffice; write them.

1.4 Merge these two sorted piles by hand, recording every comparison: $[2, 9, 11, 30]$ and $[4, 5, 12, 28]$. How many comparisons did you make? What is the *most* comparisons merging two piles of four could ever take — and design the two piles that achieve it.

1.5 Draw the full merge sort tree for the 8-item list $[7, 3, 9, 1, 8, 2, 6, 4]$: every box, every split, and the sorted contents flowing back up. Verify that each level's merge work totals 8.

1.6 Evaluate, by drawing the level sums of the tree: $T(n) = 3 \cdot T(n/3) + n$. Which of the deep water's three stories is it?

Practice

2.1 Rewrite Chapter 1's binary search as a recursion in words: "to search a region: if empty, absent; check the middle; otherwise search the ___ half." What is the floor? What does the recursion's cost recurrence look like, and which master-theorem story does it tell?

2.2 You have 2 parents, 4 grandparents, 8 great-grandparents. (a) How many ancestors n generations back does the doubling rule predict? (b) Thirty generations is roughly 800 years; compute the rule's prediction and compare it with the entire population of medieval Earth (~ 400 million). (c) The rule's conclusion is absurd, so a hypothesis fails. Find it — what does the failure tell you about your family tree's actual shape? (Recursions, like all arguments, are only as true as their assumptions about the smaller case.)

2.3 A sheet of paper is 0.1 mm thick; folding doubles the thickness. The Moon is 384,400 km away. How many folds reach it? Solve using the $M(n)$ -table style first (make the doubling table), then with one logarithm. Why does the answer feel impossible, and what (physically) actually stops it?

2.4 A tournament: 64 players, single elimination. How many *matches* are played in total? Find the answer two ways: by the recurrence/tree, and by a one-sentence counting argument with no recursion at all. (The second way is the kind of argument that makes mathematicians grin.)

2.5 Phone chain: news must reach 1,000 people; each person who knows it calls one person per minute. Model the spread as a recurrence for $K(t)$, the number who know after t minutes, and find how many minutes are needed. Then the variant: each informed person makes only *three calls total*, then stops. Does the news still reach everyone? What changed in the tree?

Concept & Counterexample

3.1 Claim: “any procedure that calls itself on a smaller input terminates.” Refute with the smallest broken procedure you can write or describe — then state the *second* condition the claim forgot, and reconcile with the chapter’s rule.

3.2 Claim: “recursion requires shrinking by at least 1, so calling yourself on $n/2$ is extra safe.” Find the loophole: a procedure that calls itself on $n // 2$ (whole-number division) and never terminates for some starting n . What is the floor it forgot?

3.3 In merge, what happens if one of the two input piles is *not* sorted? Construct a 3-and-3 example where the merge completes without complaint and outputs an unsorted list. Moral, in one sentence: what does merge trust, and what happens to procedures built on trust?

3.4 Claim: “merge sort makes exactly $n \cdot \log_2 n$ element-steps.” Test at $n = 6$: draw the tree (the halves are 3 and 3, then 2-and-1) and count exactly. Is the claim exact, an overcount, or an undercount? Repair the claim’s wording so it is true and still useful.

3.5 Claim: “Hanoi’s exponential cost comes from recursion; an iterative (loop-based) solution would be faster.” What does the Diamond say about this claim before any iterative solution is even examined?

Diamond

4.1 The wall is real: $2^n - 1$ is optimal. Two directions, both by induction. (a) **Achievability:** prove the three-line plan makes exactly $2^n - 1$ moves. (You did this in 1.3; restate it as the first half of a theorem.) (b) **The lower bound:** prove that *every* legal solution, however clever, makes at least $2^n - 1$ moves. Scaffold: at some moment, the largest disk must move for the first time (why must it move at all?). What must be true of the other $n - 1$ disks at that

exact moment — where can they legally be? Conclude that before that moment, a full $(n-1)$ -tower transfer has already happened, and after the largest disk's *last* move, another one must still happen. Apply the induction hypothesis twice and add. (c) Compare this lower bound's *style* with Chapter 1's answer-sequence bound. One counts information; this one finds an unavoidable *event*. Both fence in every strategy, including uninvented ones — point to the sentence in your proof of (b) that does the fencing.

Computational Lab

5.1 Lab 3 in full, predictions before runs. Bring the Break-it where you starve the recursion of its base case: the error message you receive is the deep water's "physical stack" made visible, and the Teach It tier wants it.

Teach It

6.1 Teach someone the Tower of Hanoi with real objects, in this strict order: let them struggle with 3 disks; have them master 2 disks until automatic; then ask only this question — "can you see a 2-disk job hiding inside the 3-disk job?" — and stay silent until they find it. Write three sentences about the moment the recursion became visible to them (it is visible on a face).

6.2 Retell Lucas's legend of the 64 golden disks to someone, ending with the 585-billion-year computation done together on a phone calculator from 2^{64} alone. Then ask them: the priests work for free and never err — why does the world still get to keep existing? Report their answer and yours.

Lab 3: The Visible Staircase

★ ★ ★

Accompanies Chapter 3. The chapter legitimized a strange object — a procedure that calls itself — and read its cost off a tree. In this lab the strange object runs on a real machine, the hidden staircase of calls becomes visible on the screen, and the $n \log_2 n$ bill gets audited.

1. A first self-call

Before towers, the smallest possible recursion — a countdown:

```
def countdown(n):
    if n == 0:
        print("liftoff")
        return
    print(n)
    countdown(n - 1)

countdown(5)
```

Read it against the chapter's two rescue conditions. The self-call goes strictly *downward* ($n - 1$). The descent has a *floor* ($n == 0$, where the procedure acts alone and returns). Both present: legitimate. Run it, then trace it in your head the slow way: `countdown(5)` prints 5 and asks `countdown(4)` to do the rest, which prints 4 and asks `countdown(3)`... Every call trusts the smaller call completely. That trust is the whole technique.

2. Hanoi in seven lines

The chapter's three-line plan — clear, cross, restack — translated nearly word for word:

```
def hanoi(n, source, target, spare):
    if n == 1:
        print("disk 1:", source, "->", target)
        return
    hanoi(n - 1, source, spare, target)      # clear the way
    ↪ (note: spare is the goal)
    print("disk", n, ":", source, "->", target) # the big
    ↪ crossing
    hanoi(n - 1, spare, target, source)      # restack on top

hanoi(3, "A", "C", "B")
```

Run it and check the seven moves against the solution your hands found with coins. Then the only part of this program that ever confuses anyone: the *peg arguments swap roles* at each call. In the first self-call, the spare peg is the destination — clearing the way means parking the small tower *out of* the way. Trace `hanoi(2, "A", "B", "C")` (just that call) on paper and confirm the pegs land where the plan demands.

How big a tower dares the machine? First predict from the chapter's table how many moves `hanoi(20, ...)` would print, then *don't print them* — counting wants a different version:

```
def hanoi_moves(n):
    if n == 1:
        return 1
    return hanoi_moves(n - 1) + 1 + hanoi_moves(n - 1)

print(hanoi_moves(10))
```

```
print(hanoi_moves(20))
print(hanoi_moves(30))
```

Check all three against $2^n - 1$. Then read the function once more: it is the recurrence $M(n) = 2M(n-1) + 1$, alive. The chapter's *cost equation* and this *cost program* are the same sentence in two grammars.

3. Making the staircase visible

Every call in §2 ran at some *depth* — Hanoi-for-3 paused while Hanoi-for-2 worked, which paused for Hanoi-for-1. The machine keeps the paused calls on its stack; let's print the stack's shape directly. Pass the depth along, and indent by it:

```
def hanoi_trace(n, source, target, spare, depth):
    print(" " * depth, "want", n, "disks:", source, "->",
          ↪ target)
    if n == 1:
        print(" " * depth, "move disk 1")
        return
    hanoi_trace(n - 1, source, spare, target, depth + 1)
    print(" " * depth, "move disk", n)
    hanoi_trace(n - 1, spare, target, source, depth + 1)

hanoi_trace(3, "A", "C", "B", 0)
```

(" " * depth is a small Python kindness: a string times a number repeats it.) Run it and *read the left margin*: the indentation breathes in and out, deeper as work is delegated, shallower as delegations finish. That margin **is** the call stack — the deep-water box made of whitespace. Draw the same shape by hand for `hanoi_trace(2, ...)` before believing the machine's.

4. Merge sort, audited

The chapter sorted cards: split, sort each half somehow, merge. The merge first — it is where all real work happens:

```
def merge(left, right):
    result = []
    i = 0                # finger on left pile
    j = 0                # finger on right pile
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i = i + 1
        else:
            result.append(right[j])
            j = j + 1
    while i < len(left):    # one pile ran dry; sweep the
        ↪ other in
        result.append(left[i])
        i = i + 1
    while j < len(right):
        result.append(right[j])
        j = j + 1
    return result

print(merge([2, 9, 11, 30], [4, 5, 12, 28]))
```

Two fingers, two piles, always take the smaller top card — the code is your hands from the chapter. Now the recursion around it, with one new piece of notation: `items[:mid]` is the list *up to* position `mid`, and `items[mid:]` is the rest. Splitting is the idea; the colon is just its spelling.

```
def merge_sort(items):
```

```
if len(items) <= 1:           # the floor: tiny piles are
    ↪ born sorted
    return items
mid = len(items) // 2
left = merge_sort(items[:mid])
right = merge_sort(items[mid:])
return merge(left, right)

print(merge_sort([7, 3, 9, 1, 8, 2, 6, 4]))
```

Eight lines around a merge. Test it on your own lists — include an empty list, a one-item list, a list with repeats, a list already sorted backward. (Testing the *edges* first is a habit worth stealing from professionals; Chapter 1’s human moment is the reason the habit exists.)

5. The bill, audited

The chapter promised $C(n) \approx n \log_2 n$ in the currency of element-steps — one step per card the merge touches. Send the count climbing back up the recursion alongside the sorted list; the function now returns a two-item list, [sorted, steps]:

```
def merge_sort_count(items):
    if len(items) <= 1:
        return [items, 0]
    mid = len(items) // 2
    left = merge_sort_count(items[:mid])
    right = merge_sort_count(items[mid:])
    merged = merge(left[0], right[0])
    steps = left[1] + right[1] + len(merged)    # this merge
    ↪ touches every card once
    return [merged, steps]
```

Now the audit, in the style of Lab 1’s race — and watch the *per-*

item column:

```
for n in [1000, 10000, 100000, 1000000]:
    items = list(range(n, 0, -1))          # n, n-1, ..., 1:
    ↪ sorted backward
    result = merge_sort_count(items)
    steps = result[1]
    print(n, steps, "steps;", steps / n, "per item")
```

The left column multiplies by 10; the per-item column climbs by a little over 3 each row. You have seen that gait before — it is the logarithm's, from Chapter 1's table, now appearing as the *per-item* price of sorting. Total steps $\approx n \times (\log_2 n\text{-ish per item})$: the tree's level-count, measured in the wild. Check one row against arithmetic: for $n = 1,000,000$, is steps close to 20 million?

Break it

Predict first. Run second. Explain any gap third.

1. Starve countdown of its floor: delete the `if n == 0:` block (keep the rest) and call `countdown(5)`. Predict precisely what the machine will do — forever? crash? something else? — then run it and *read the error to its last line*. The number it complains about (Python's recursion limit, near a thousand) is the deep-water box's claim that the stack is physical, demonstrated. How is this failure different in kind from Sevenia's in Lab 2 — which lab's program was *wrong*, and which was *exhausted*?

2. In `hanoi`, swap the two recursive calls (clear-the-way and restack trade places). Predict: will the printed moves still be *legal* — no large disk onto a small one? Then run it for $n = 3$ and *audit the move list with your coins*, move by move. Find the first illegal moment and describe which trust was betrayed.

3. In `merge_sort_count`, change `mid = len(items) // 2` to `mid = 1`: every split now peels a single card off the front instead of halving.

Two predictions before running: is the output still *correct*? and what happens to the cost — per-item column flat-ish like the logarithm, or growing with n ? Audit the lopsided version on sizes 200, 400, 800 (not the big sizes — you'll discover why in a moment) and look at the *total* steps as n doubles. Doubling the input does what to the bill? Connect it to the chapter's closing sentence: *where a recursion sends its work is the whole game*. You have just met, in your own code, the difference between $n \log n$ and n^2 — Chapter 6 makes it official. Then, deliberately, run size 2,000 and read the error to its last line: the lopsided tree is n calls deep where the balanced tree was $\log_2 n$ deep, and the machine's stack — Break-it 1's discovery — just refused the architecture, not the arithmetic.

4. In merge, change `<=` to `<`. Predict whether any list of *numbers* can ever come out unsorted. Run your tests; they should all pass. So the character changed nothing? To find out, sort things that can genuinely *tie*. Make each item a [score, name] pair and — one more edit — make the merge compare **scores only**: change the comparison line to `if left[i][0] <= right[j][0]:`. Now sort `[[3, "Ada"], [1, "Hua"], [3, "Noether"]]` with the `<=` version and the `<` version and compare the outputs. What happened to the two score-3 students, and which version keeps tied items in their original order? (Chapter 6 names this property; you have discovered it early. And a side puzzle: why did the experiment *need* the scores-only edit — what was Python quietly doing with the pairs before it?)

Chapter 4: Recursion That Remembers

★ ★ ★

In which an honest procedure is caught wasting its life, and a wound from Chapter 2 is healed

The staircase

A staircase has n steps. You climb it in hops of one step or two, as you please. Question: how many *different* ways are there to climb it? (Two ways are different if the sequence of hops differs: 1-then-2 is not the same as 2-then-1.)

Small cases by hand. One step: one way. Two steps: two ways (1+1, or 2). Three steps: 1+1+1, 1+2, 2+1 — three ways. Four steps: enumerate carefully — five ways.

1, 2, 3, 5... Before conjecturing from the pattern, *derive* the law, Chapter 3 style. Stand at the bottom of n steps and look at your first decision. It is a hop of 1 or a hop of 2 — there is no third kind — and the two choices lead to different futures that can never produce the same climb. Hop 1, and what remains is the staircase problem for $n - 1$. Hop 2, and it is the problem for $n - 2$. So:

$\text{ways}(n) = \text{ways}(n - 1) + \text{ways}(n - 2)$, with the floor ways(1) = 1, ways(2) = 2.

A recursion that calls itself twice, on two different smaller cases.

Chapter 3's two rescue conditions hold — strictly downward, solid floor — so the procedure is *correct*, by the staircase of induction (on an actual staircase, for once). The sequence it generates — 1, 2, 3, 5, 8, 13, 21, 34 — is one of the most famous in mathematics: the **Fibonacci numbers**, which have a habit of appearing wherever something grows by combining its two most recent pasts. You met some of its members in Chapter 1's sorted list; now you know what they were doing there.

Correctness settled. Now the second question, and this time the answer is a scandal.

The procedure that forgets

Run the recursion in your head for `ways(6)` and *draw the tree* of calls, Chapter 3 style. The top box, `ways(6)`, opens into `ways(5)` and `ways(4)`. The `ways(5)` box opens into `ways(4)` and `ways(3)`. Stop right there — **`ways(4)` appears twice**. Keep drawing and count appearances: `ways(3)` is computed three times, `ways(2)` five times, `ways(1)` three times. The full tree for `ways(6)` contains fifteen boxes to answer what is really six questions.

15 boxes for 6 distinct questions: $\text{ways}(6) \times 1$ $\text{ways}(5) \times 1$ $\text{ways}(4) \times 2$ $\text{ways}(3) \times 3$ $\text{ways}(2) \times 5$ $\text{ways}(1) \times 3$

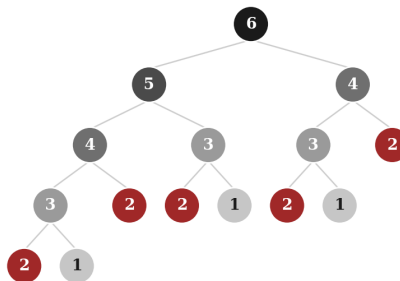


Figure 4.1 — The amnesiac call tree for `ways(6)`: same shade, same question — fifteen boxes for six questions.

Figure 4.1 — The amnesiac call tree for `ways(6)`: same shade, same question — fifteen boxes for six questions.

Nothing is *wrong*. Every box computes a true answer by a valid method. But the procedure has no memory: each time it needs `ways(4)`, it rebuilds the entire `ways(4)` sub-tree from nothing, like a scholar who burns each page upon finishing it and re-derives the page whenever it's cited. And the waste compounds on itself: the tree for n contains the *whole tree* for $n - 1$ and the *whole tree* for $n - 2$, so the call count grows by the staircase's own law — the cost of computing Fibonacci numbers is itself Fibonacci, which grows like the exponential wall of Chapter 3. In Lab 4 you will count the calls: `ways(30)`, a number you could extend with a pencil in a minute, costs the amnesiac recursion about **1.7 million** calls. At $n = 50$ the pencil still needs its minute; the recursion needs twenty-five *billion* calls — an hour or more of a modern laptop's full attention, spent re-answering fifty questions.

Here is the diagnosis to hold on to, because it locates the exponential's weak point. Hanoi's $2^n - 1$ was a wall of *necessary work* — the Diamond of Chapter 3 proved every move unavoidable. The staircase's exponential is a wall of **repetition**: only n distinct questions exist, asked exponentially many times. A wall made of the same brick photocopied is a wall made of paper.

Write it down

The fix is one idea, and it is the entire chapter: **the first time you answer a question, write the answer down; ever after, look it up.**

The notebook for answers is a new tool, Python's **dictionary** — a list, roughly, that you index by *question* instead of by position; Lab 4 introduces it properly. The memoized procedure (the term of art is **memoization** — “memo,” not “memor”: leaving yourself a memo) does this:

asked $\text{ways}(n)$: if n is in the notebook, read it out; otherwise compute $\text{ways}(n - 1) + \text{ways}(n - 2)$ — by this same procedure — write the result next to n in the notebook, then answer.

Now count the cost. Each of the n distinct questions is *computed* at most once — every later asking is a lookup. So the total computing work is about n . The audit in Lab 4 makes it brutal: $\text{ways}(30)$ falls from 1.7 million calls to **57**. Same recursion, same answers, same correctness proof. One notebook, and the exponential collapses to a straight line. There is no other moment in this book where so much is bought for so little.

And once the notebook is in view, a small liberation follows. The recursion fills the notebook from the top down — asked about 30, it scrambles down to the floor and climbs back. But you could just as well fill the notebook **bottom-up**: write $\text{ways}(1)$, $\text{ways}(2)$, then march upward, each entry a one-line sum of two earlier entries, no recursion at all, just a table and a loop. Top-down and bottom-up produce *the identical table*; they differ only in the order the blanks get filled (and bottom-up, having no call stack, never meets Chapter 3’s recursion limit). The table is the real object. The method that builds answers to a problem out of stored answers to its subproblems — by either route — is called **dynamic programming**, a name so unhelpful that it has its own story below. Think “recursion that remembers,” or just: *the table method*.

A human moment

“Dynamic programming” contains no dynamics and no programming in the modern sense. The name was chosen around 1950 by Richard Bellman at the RAND Corporation — and by his own telling, chosen as *camouflage*. The Secretary of Defense then funding RAND, Bellman wrote in his autobiography, “had a pathological fear and hatred of the word *research*”; mathemat-

ics was even worse. So Bellman picked a name no one could object to or even quite parse: “it’s impossible to use the word *dynamic* in a pejorative sense,” he explained, and “programming” then meant planning — schedules, logistics, tables — not code. Historians note the dates in his story don’t all line up, and Bellman, a famous raconteur, may have polished it. But the lesson survives the embellishment, and it is the second vignette in a row about a mathematician choosing a disguise: Lucas dressed an exponential as a toy from Siam; Bellman dressed a method as bureaucratic nothing to keep it funded. Names are not neutral. The thing itself — the table method — outlived the Secretary, the camouflage, and very possibly the anecdote.

Sevenia, healed

Now the debt this book has carried since Chapter 2. Sevenia’s coins $\{1, 7, 10\}$; you owe 14; greedy grabbed the 10 and paid five coins for a two-coin debt. Time to make change *correctly* — for every amount, in any coin system whatsoever.

Ask the question in the staircase’s grammar. Let $\text{best}(a)$ be the fewest coins paying amount a . Stand at amount a and look at your first decision: the first coin handed over is a 1, a 7, or a 10 — no fourth kind — and each choice leaves a smaller debt:

$$\text{best}(a) = 1 + \min(\text{best}(a - 1), \text{best}(a - 7), \text{best}(a - 10))$$

(only the options with $a - c \geq 0$ compete, and the floor is $\text{best}(0) = 0$: no debt, no coins).

Put this next to greedy and the difference in *character* is the chapter’s real lesson. Greedy, at amount 14, looked at the three coins, judged the 10 best-looking, and **committed** — the 7-branch and the 1-branch were never priced. The recurrence refuses to commit: it prices **every** branch — what would the future cost after a 1, after a 7, after a 10 — and only then takes the min. Greedy is a verdict before the trial; dynamic programming insists on hearing every future

testify. That refusal would be exponentially expensive done naively (three branches, branching again, the staircase’s tree all over). The notebook makes it cheap: only the amounts 0 through a exist as distinct questions, so the whole trial costs about $a \times (\text{number of coin types})$ table entries.

Run the table bottom-up by hand for Sevenia to 14 — this is the wound actually closing, so do it slowly:

a	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
best(a)	0	1	2	3	4	5	6	1	2	3	1	2	3	4	2

Watch the entry at 7 snap to 1, the entry at 10 snap to 1, and then the entry at 14: $\min(\text{best}(13), \text{best}(7), \text{best}(4)) + 1 = \min(4, 1, 4) + 1 = 2$. The table found $7 + 7$ not by insight but by *bookkeeping* — the 7-branch was priced like every other branch, and the price was right there. In Lab 4 your DP changer faces your own Chapter 2 greedy across every amount to 100; greedy loses six times and DP never loses once, because DP cannot lose: the Diamond below has you prove its recurrence is not a heuristic but a *theorem*.

When the table method works — and the spell-checker

Two ingredients made all of this go, and they are worth naming because together they are a *diagnostic* you can run on new problems:

1. **Optimal substructure:** the best answer to the whole is assembled from best answers to subproblems. (The first hop of an optimal climb leaves an optimal climb; the first coin of an optimal payment leaves an optimal payment. The Diamond makes this precise — there is a cut-and-paste argument hiding in “leaves.”)

2. **Overlapping subproblems:** the recursion meets the *same* subproblem many times — otherwise the notebook records answers nobody asks for twice. Merge sort has perfect substructure and *zero* overlap (its two halves are different lists); memoizing it buys nothing. The notebook pays exactly where the tree repeats.

The diagnostic passes on a problem that runs, today, inside every phone you have ever typed on. How does a spell-checker know that when you typed “graet” you meant “great”? It measures the **edit distance** between words: the fewest one-letter edits — insert a letter, delete a letter, or replace a letter — turning one word into the other. Both questions of the book apply: a correct procedure, at what cost? — and the table method answers, because the problem has the grammar of the staircase. Look at the *last letters* of the two words. If they match, they cost nothing: the distance is whatever the rest costs. If they differ, your last edit was one of exactly three kinds — replace one with the other, delete one, or insert one — and each choice leaves a smaller two-word problem. Three branches, a min, a floor (the distance from any word to the empty word is its length: delete everything). Subproblems — “first part of word one versus first part of word two” — overlap massively, so the notebook is a two-dimensional table, one row per prefix of the first word, one column per prefix of the second.

Small enough to do at a desk: “cap” to “tape” has distance 2 (replace c→t, then insert the e), and the 4×5 table verifying it is Fluency 1.5. Big enough to matter: the same table, grown, compares DNA sequences, suggests your search-bar corrections, and shows lawyers what changed between contract drafts. One grammar — *last decision, three branches, min, floor, notebook* — and you should feel the chapter consolidating into a method you could point at a problem neither of us has seen.

Deeper water: the craft is the state

The table method's honest limitation: ingredient 1 can *fail*, and when it fails it fails quietly. Try planning the best two-week trip as “the best first week, then the best second week from wherever it ended” — and watch the assembly break: the two weeks *interact* (one budget, one energy, can't see the same city twice), so the best whole is not built from the best parts. Subproblems that share a resource are not really sub-problems; they are one coupled problem wearing two name tags.

The professional response is not to abandon the method but to **enrich the state**: don't tabulate `best(week)`, tabulate `best(week, money remaining)` — fold the shared resource into the question itself, and substructure is restored, at the price of a bigger table. Choosing *what the table's questions are* — the state — is the actual craft of dynamic programming; the recurrences write themselves once the state is right. The staircase's state was one number; the coins', one number; edit distance needed two. Problem 3.4 hands you a coupled problem and asks you to design the state that uncouples it. And a forward pointer: in Chapter 9, a famous algorithm will silently memoize *shared subexpressions in the chain rule* — the notebook idea, working inside calculus — so file this chapter somewhere permanent.

Where this leaves us

The exponential wall, it turns out, comes in two materials. Hanoi's was stone — every move proved necessary. The staircase's was paper: few questions, asked absurdly often, and one notebook tore it down. Dynamic programming is the systematic tearing: find the grammar (*last decision, few branches, min or sum, floor*), name the state, fill the table — top-down with a memo or bottom-up with a loop, same table either way. Greedy commits before pricing; the table

prices before committing; Sevenia's 14 costs two coins forever now, and you can prove it.

The tables themselves are trying to tell you something, if you look at them sideways. Entry 14 depended on entries 13, 7, and 4; those depended on others; the whole computation is a web of *questions pointing at the questions they need* — dots and arrows, a territory with a map. Chapter 5 stops solving problems long enough to study the maps themselves: things connected to things, the shape behind staircases, subway lines, friendships, and the web — and the two or three ways of walking such a territory that answer almost every question about it.

Problem Set 4

★ ★ ★

Lab 4 (The Notebook) is the Computational Lab tier. Problems 2.5 and 3.2 want your lab code as equipment.

Fluency

1.1 Extend the staircase table by hand to $n = 12$ using the recurrence. Then answer from the table, no new work: a 12-step staircase where the first hop was a 2 — how many ways remain?

1.2 For ways(7), draw the full amnesiac call tree and count how many times each of ways(1) through ways(6) is computed. Then list, in order, the calls the *memoized* version actually computes (not looks up) when asked ways(7) cold.

1.3 Build the Sevenia table bottom-up from 15 to 22, continuing the chapter’s table. At which amounts in that range does greedy (from your Chapter 2 audit) disagree, and by how much?

1.4 US coins {1, 5, 10, 25}: build the best(a) table to $a = 30$ and verify it agrees with greedy at every entry. (Chapter 2’s deep water explains why this agreement, continued to 34, is a certificate.)

1.5 Fill in the complete 4×5 edit-distance table for “cap” $\rightarrow$ “tape” (rows: prefixes of “cap” including empty; columns: prefixes of “tape” including empty). Confirm the corner says 2, and trace one optimal edit sequence back through the table.

1.6 Edit distance of “kitten” $\rightarrow$ “sitting” is famously 3. Name three edits that achieve it; the table (optional, 7×8) certifies noth-

ing beats them.

Practice

2.1 Triple staircase: hops of 1, 2, or 3 allowed. Derive the recurrence and floor, build the table to $n = 10$, and find how many ways to climb 10 steps.

2.2 City grid: you stand at the corner of 1st & 1st and walk to 5th & 7th moving only north or east one block at a time. Set up ways(street, avenue) as a table-method problem — what is the recurrence, what is the floor (think: edges of the grid), and how many routes exist? Then the variant: the corner at 3rd & 4th is closed. One table entry changes its rule; recount.

2.3 Toll staircase: step k charges a toll $t(k)$, and you pay the toll of every step you *land on*. For tolls (steps 1 through 8) = 3, 1, 4, 1, 5, 9, 2, 6, find the cheapest climb to the top of step 8 with hops of 1 or 2. State the recurrence (min now, not sum), build the table, and recover *which steps* the cheapest climb lands on.

2.4 Counting versus optimizing: the Sevenia recurrence used min; the staircase used sum. Write the recurrence for “how many *different ways* can Sevenia’s coins pay amount a , where order of handing over matters” and compute the table to $a = 12$. Why does the same skeleton serve both questions?

2.5 Use your lab’s DP changer on the stamp system $\{1, 4, 9\}$ (Problem 2.3 of Chapter 2) for all amounts to 50: list every amount where greedy overpays and the worst overpayment in the range. Then design a three-coin system (with a 1) where greedy’s worst overpayment on amounts up to 100 is as large as you can make it — beat your Chapter 2 entry now that you own the truth-machine.

Concept & Counterexample

3.1 Claim: “memoization speeds up any recursion.” Use merge sort to refute it precisely: say what the memo’s keys would be, why almost no key is ever asked twice, and what the notebook costs while earning nothing. State the missing ingredient by name.

3.2 Claim: “the table method always beats greedy.” Cheap shot, but make it precise: on US coins both are optimal — so compare their *costs* (table entries built vs. greedy’s steps) for amount 10,000. When greedy is *right*, what is it, in the table method’s terms? (Look at what greedy refuses to price, and what that refusal saves.)

3.3 Claim: “ways(n) counts climbs, so by symmetry, climbs that *start* with a 1-hop and climbs that *end* with a 1-hop are equally numerous.” Decide by computing both counts for $n = 5$ from the table and an argument, not enumeration. Then give the one-sentence reason (a staircase climbed backward is...).

3.4 *The coupled trip, uncoupled.* You have 7 days and \$400. Each city i offers joy(i) but costs days $d(i)$ and dollars $m(i)$, and no city twice. “Best trip = best 3-day trip + best 4-day trip” fails — exhibit a tiny 3-city instance where it fails. Then design the state that restores substructure (what *two* quantities must a subproblem’s question carry?) and write the recurrence. Do not fill the table; designing it is the problem.

3.5 Claim: “bottom-up and top-down always do the same work.” Find the crack: design a coin system and amount where top-down *never asks* about most amounts (which ones can it skip and why?), while bottom-up dutifully fills every entry. (Hint: coins {1000, 2000}, amount 10,000 — wait, where did the 1 go? What does best(a) even mean for unpayable a , and how must the recurrence’s min handle it? Two design decisions hide in this innocent claim.)

Diamond

4.1 *The recurrence is a theorem.* Prove: for any coin system C containing 1 and any amount $a \geq 1$, $\text{best}(a) = 1 + \min \text{ over } c \text{ in } C, c \leq a, \text{ of } \text{best}(a - c)$. Two inequalities, two cut-and-paste arguments: (a) $\leq$ (the recurrence can be achieved): for the minimizing coin c , take any optimal payment of $a - c$ and add one c . Verify this pays a legally and count it. (b) $\geq$ (nothing beats the recurrence): take any optimal payment of a ; it contains *some* first coin c . Remove it. Prove what remains must be an **optimal** payment of $a - c$ — suppose it weren't, paste in a better one, and exhibit the contradiction with the optimality you started from. (This paste is the heart; write it slowly.) (c) One sentence each: where exactly does (b)'s paste fail for the two-week-trip problem of the deep water, and where does the *greedy* argument — which also picks “some first coin” — part ways with yours?

Computational Lab

5.1 Lab 4 in full, predictions before runs. The shared-notebook Break-it (one memo, two coin systems) is required preparation for Teach It 6.2 — bring the wrong answer it produced.

Teach It

6.1 Teach someone the staircase count with real stairs or taped floor squares, in this order: have them enumerate 3 and 4 steps by walking; show them the first-hop argument; then ask them to predict 5 steps *before* enumerating, and let the staircase grade them. Three sentences on what convinced them — the walking or the argument.

6.2 Tell the two-disguise story — Lucas's Siamese professor, Bellman's bureaucrat-proof name — to someone, and then explain what the *method* behind the boring name actually does, using Sevenia's

14 with coins on the table: greedy's grab, then the notebook's trial of every branch. End with the lab's shared-notebook bug and ask them: whose fault was the wrong answer — the method's, or the notebook-keeper's? Report their verdict.

Lab 4: The Notebook

★ ★ ★

Accompanies Chapter 4. The chapter caught a correct recursion wasting its life on repetition, and tore the exponential down with a notebook. In this lab the waste is counted call by call, the notebook is built, and your own Chapter 2 greedy is put on trial against a changer that cannot lose.

1. The amnesiac, counted

The staircase recursion, plus the counting trick from Lab 3 — each call reports the *total number of calls* it caused, alongside its answer, as a two-item list [answer, calls]:

```
def ways_count(n):
    if n <= 2:
        return [n, 1]                # ways(1)=1,
        ↪ ways(2)=2; one call
    a = ways_count(n - 1)
    b = ways_count(n - 2)
    return [a[0] + b[0], a[1] + b[1] + 1]

print(ways_count(6))
print(ways_count(10))
```

Check ways(6) against the chapter's hand count — and check the call count against your drawing of the tree (the chapter said fifteen boxes; does the machine agree?). Now the audit, in the house style:

```
for n in [10, 15, 20, 25, 30]:
    result = ways_count(n)
    print(n, "ways:", result[0], "  calls:", result[1])
```

Read the calls column the way you read every table in this book: n goes up by 5 each row — what happens to the calls? This is not Chapter 1’s logarithm and not even Chapter 3’s doubling-per-step; measure its gait (each row is roughly *how many times* the previous?). Run $n = 34$ and *feel* the pause before the answer — the first time in this book the machine has kept you waiting. The answer is a seven-digit number you could extend by hand in seconds. All that waiting was repetition.

2. The notebook

Python’s notebook is the **dictionary**: like a list, but indexed by *question* rather than by position. Three moves to learn, all visible here:

```
memo = {}                # an empty notebook
memo[7] = 13             # write: next to question 7, answer
↪ 13
print(7 in memo)         # ask: is question 7 already
↪ answered?
print(memo[7])           # read the answer
```

Now the chapter’s fix, verbatim — look it up, or compute-and-write:

```
memo = {1: 1, 2: 2}      # the floor, pre-written

def ways_memo(n):
    if n in memo:
        return memo[n]
```

```
    answer = ways_memo(n - 1) + ways_memo(n - 2)
    memo[n] = answer
    return answer

print(ways_memo(30))
```

Same recursion, same floor, same answers — verify a few against §1. The only new behavior: no question is ever *computed* twice.

3. The collapse, measured

Count the memoized version's calls (the counter rides along in a one-item list — a small trick: numbers passed to functions don't come back changed, but a list's *contents* do, so we keep the count in a box and pass the box):

```
memo = {1: 1, 2: 2}
calls = [0]

def ways_memo_count(n):
    calls[0] = calls[0] + 1
    if n in memo:
        return memo[n]
    answer = ways_memo_count(n - 1) + ways_memo_count(n - 2)
    memo[n] = answer
    return answer

ways_memo_count(30)
print("memoized calls for n=30:", calls[0])
```

Set it beside §1's number for $n = 30$. Millions against dozens, identical answers. Now the gait check: reset the notebook (rerun the cell after changing 30 to 40, but reset `memo` and `calls` first — why must you reset? predict what the count reports if you *don't*) and confirm the memoized call count grows like n itself: the straight

line that replaced the explosion.

4. Sevenia on trial

The chapter's recurrence, as a program — for *any* coin system. The notebook here must be reset per problem, so we wrap the whole trial in one function that builds a fresh memo, then defines the recursive worker inside (a function defined inside a function: new sight, no new idea — the inner worker simply gets to use the outer function's memo and coins):

```
def fewest_coins(coins, amount):
    memo = {0: 0}                    # the floor: no debt,
    ↪ no coins

    def best(a):
        if a in memo:
            return memo[a]
        options = []
        for c in coins:
            if c <= a:
                options.append(best(a - c))
        answer = 1 + min(options)    # min: the cheapest
        ↪ priced future
        memo[a] = answer
        return answer

    return best(amount)

print(fewest_coins([10, 7, 1], 14))
```

Two. The wound closes on your screen. (min of a list is what it says; it has been waiting in Python all along for this chapter.)

Now the full trial — your Chapter 2 greedy versus the table method, every amount to 100. Retype your greedy from Lab 2 (or

paste it from your notebook there):

```
def greedy_change(coins, amount):
    used = []
    for coin in coins:
        while amount >= coin:
            amount = amount - coin
            used.append(coin)
    return used

for amount in range(1, 101):
    g = len(greedy_change([10, 7, 1], amount))
    d = fewest_coins([10, 7, 1], amount)
    if g != d:
        print(amount, "greedy:", g, " table:", d)
```

Compare the verdict list with your Lab 2 audit (which used brute force as the truth). Same failures, same truth — but notice what changed *about the truth-machine*: Lab 2’s three nested loops tried everything; this one prices each amount once. The notebook replaced the brute force. Confirm the new machine’s reach: `fewest_coins([10, 7, 1], 900)` — instant, where three nested loops would grind.

5. The same table, bottom-up

The chapter promised the notebook can be filled by a plain loop, floor upward, no recursion at all:

```
def fewest_coins_table(coins, amount):
    best = [0]                                # best[0] = 0; the
    ↪ table grows rightward
    for a in range(1, amount + 1):
        options = []
```

```
    for c in coins:
        if c <= a:
            options.append(best[a - c])
        best.append(1 + min(options))
    return best

table = fewest_coins_table([10, 7, 1], 14)
print(table)
print("best(14) =", table[14])
```

Read the printed list against the chapter’s hand-built table — entry by entry, they must agree (the table *is* the method; the two routes merely fill it in different orders). This version climbs to any height without a call stack: try amount 5,000, then try the same in §4’s recursive version and witness which one meets Lab 3’s old enemy.

Break it

Predict first. Run second. Explain any gap third.

1. In §2’s `ways_memo`, delete only the line `memo[n] = answer` (keep everything else). Predict: still correct? still fast? one or the other? Run §3’s count with the line gone. Which single line of this lab was the entire chapter?

2. Sabotage by sharing: move §4’s `memo = {0: 0}` *outside* the function (one notebook for all trials), then run `fewest_coins([10, 7, 1], 14)` followed by `fewest_coins([25, 10, 5, 1], 14)`. Predict both outputs first. The second answer is wrong — wrong *how*, and in whose favor? State the moral as one sentence about what a notebook entry silently assumes. (This bug, in grown-up systems, is called cache poisoning, and it has caused real outages; you have now caused one in eleven lines.)

3. Hops of {2, 3} only: adapt `ways_memo` for a staircase where you may hop 2 or 3 steps. The floor needs care — decide what `ways(0)` and `ways(1)` should *be* (is “do nothing” a way to climb zero steps?

is 1 climbable at all?), write your floor, and test against hand counts for $n = 2, 3, 4, 5$. Then break a classmate's floor: find the n where a plausible-but-wrong floor (say, $\text{ways}(0) = 0$) first gives a wrong count. Floors are where recursions are won and lost.

4. In §5's bottom-up changer, feed it a kingdom without the 1-coin: `fewest_coins_table([10, 7], 11)`. Predict the failure mode precisely — wrong answer, error, or silence? — and read what actually happens to its last line. Then repair the design: unpayable amounts need an answer that survives `min` and `+ 1` honestly (the chapter's Problem 3.5 is circling this same hole). Propose a fix, implement it, and verify amount 11 reports "impossible" while 17 reports 2.

Chapter 5: Maps, Mazes, and Networks

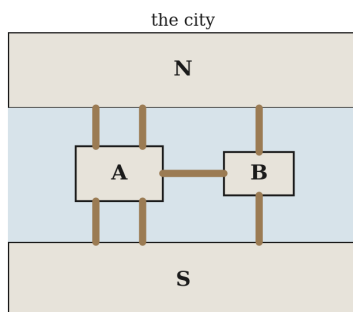
★ ★ ★

In which a city's bridges become dots and lines, and three ways of walking answer almost everything

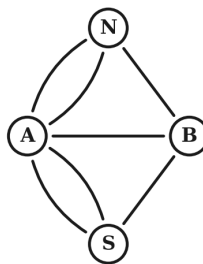
Königsberg, 1736

The city of Königsberg sat on both banks of a river, plus two islands, the four land masses joined by seven bridges. The townspeople had a standing puzzle: can you take a walk that crosses every bridge exactly once?

They asked Leonhard Euler — already the most prolific mathematician alive — and his first reaction, in a letter, was that the problem “has little connection to mathematics.” Then he solved it, and in solving it he *made* the connection, founding a field. His move is the reason this chapter exists, so watch it closely. Euler threw the city away. The size of the islands, the length of the bridges, the bends of the river — none of it matters to the question. What matters is only *what is connected to what*. Each land mass becomes a **dot**; each bridge a **line** between two dots. Four dots, seven lines, and the famous old city has become a diagram you can draw on a napkin.



the napkin: 4 dots, 7 lines



every vertex odd $\rightarrow$ no such walk

Figure 5.1 — Königsberg and Euler's napkin: the city, then the same city with everything inessential burned away.

Figure 5.1 — Königsberg and Euler's napkin: the city, then the same city with everything inessential burned away.

The diagram is called a **graph**: dots (call them *vertices*) and connecting lines (*edges*). And Euler's answer came straight out of the napkin: such a walk is impossible — every one of the four vertices has an *odd* number of bridge-ends touching it, and (you will prove this yourself in Practice 2.1, with a counting argument worthy of the name) a walk that crosses every edge once can tolerate at most two odd vertices: its start and its end. Anyone passing *through* a place must arrive and leave in pairs. The townspeople had been searching for years for something a parity check kills in a minute. Throwing away detail is not laziness; done right, it is the most powerful act in mathematics.

The same picture, everywhere

Once you own the abstraction, you see it daily, because “things, connected to things” is most of the world's structure:

- your school: rooms as vertices, hallways as edges;
- the subway map (which already is Euler's napkin — the designers threw away geography for you);

- friendships: people as vertices, an edge per friendship;
- the web: pages as vertices, links as edges — note these edges have a *direction*;
- and Chapter 4’s closing promise: every dynamic programming table you built was secretly a graph — entries as vertices, an edge from each entry to the entries it consulted. `best(14)` pointed at `best(13)`, `best(7)`, `best(4)`. You have been drawing graphs since last chapter without the name.

A graph enters Python the way Chapter 4 taught things to enter Python — as a dictionary. One entry per vertex; next to it, the list of its neighbors:

```
hallways = {  
    "gym":    ["lobby"],  
    "lobby":  ["gym", "library", "cafeteria"],  
    "library": ["lobby", "lab"],  
    "cafeteria": ["lobby", "lab"],  
    "lab":    ["library", "cafeteria"],  
}
```

That is the entire data structure. The questions, though, are many: *can* you get from here to there? what is the *fewest* hallways? the *quickest* route when hallways have lengths? Remarkably, three walking disciplines — two of them one word apart — answer nearly all of it.

Ripples: breadth-first search

Drop a stone in a pond and watch the rings: everything one hop away gets wet first, then everything two hops away, then three. **Breadth-first search (BFS)** is that ripple, run on a graph: starting somewhere, visit all vertices 1 edge away, then all at 2 edges, then 3 — never touching a far vertex before every nearer one is done.

The discipline that enforces ripple-order is a waiting line. Keep a **queue** of vertices to process; always serve the *front* of the line; when you serve a vertex, its not-yet-seen neighbors join the *back*. First in, first out — exactly a bakery line — and fairness of the line is what makes the rings expand evenly: everything at distance d entered the line before anything at distance $d + 1$, so it gets served first, so *its* discoveries (distance $d + 1$) enter before theirs ($d + 2$). The ripple is bureaucracy, perfectly administered. One more piece of bookkeeping makes it terminate: a **visited** set, so no vertex joins the line twice. (Chapter 1’s lesson about structure: the visited set is what makes “have I seen this?” cheap.)

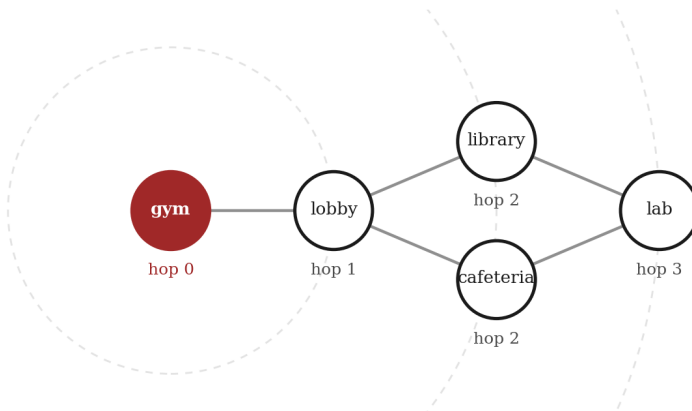


Figure 5.2 — The school as a graph, and the ripple from the gym: ring numbers are hop distances.

Figure 5.2 — The school as a graph, and the ripple from the gym: ring numbers are hop distances.

What you get for this discipline is the chapter’s central theorem: **BFS finds shortest paths** — when BFS first reaches a vertex, it has reached it by a fewest-possible-edges route, and the ring number is the distance in hops. The proof is an induction on the rings, and it is the Diamond: ring d is correct because ring $d - 1$ was, true at the floor (the start is its own ring zero), preserved ring to ring. Fourth appearance of the only trick.

And note what BFS says when the ripple dies out with some vertex never wetted: that vertex is **unreachable** — not “probably not connected,” but provably, because the ripple visited everything touchable. Chapter 1’s expensive certainty of absence, again purchased by exhaustiveness, again organized by structure.

A human moment

Nobody invented breadth-first search; at least three people did, separately. Konrad Zuse — who built programmable computers in his parents’ Berlin living room during the war — described it in his 1945 thesis, which sat unpublished until 1972. E. F. Moore at Bell Labs reinvented it in 1959 as “the shortest path through a maze,” wanting to route telephone calls through switching networks. C. Y. Lee reinvented it again in 1961 to route *wires* across circuit boards — chip designers call grid-BFS the “Lee algorithm” to this day, and some version of it has helped lay out nearly every chip in every device you own. Triple invention is not embarrassment; it is evidence. When the world builds networks — telephone, electronic, social — the ripple gets discovered by whoever is standing closest to the need. Ideas have infrastructure the way plants have seasons.

The thread: depth-first search

The opposite temperament: instead of expanding evenly, *commit*. Walk down one corridor as far as it goes; only when stuck, back up to the last junction with an untried option and commit again. This is **depth-first search (DFS)** — the instinct of every maze-runner in history, including the first one: Greek myth sends Theseus into the Labyrinth trailing Ariadne’s thread, and the thread *is* the algorithm — it is the record of the current path, paid out going forward, rewound when backing up.

The first person to state the maze method precisely enough to de-

serve the word algorithm was Charles Trémaux, a French engineer, around 1880 — and his work reached the world in the *Récréations Mathématiques* of one Édouard Lucas. Yes: the Hanoi man, again, in his other role as mathematics’ great impresario of puzzles. And Lucas’s two appearances in this book are secretly one appearance, because **DFS is recursion** — Chapter 3 in a new costume. “Explore a vertex” means: mark it visited; for each unseen neighbor, *explore that neighbor* — a procedure calling itself — and the backing-up is nothing but the call stack unwinding. Ariadne’s thread, Trémaux’s pencil marks, and Chapter 3’s visible staircase are the same object. In the lab you will see it plainly: written iteratively, DFS and BFS differ by *one word* — serve the line from the back instead of the front, a stack instead of a queue, and the patient ripple becomes the headlong thread.

What DFS does *not* promise is shortest paths — the thread plunges past near rooms to reach far ones, and Problem 3.1 has you build the maze that proves it. What it does superbly: visit everything reachable, cheaply, while carrying a *path* in its hand at all times — which makes it the tool of choice when the question is “is there a way at all,” “find me any route,” or, later in your life, a dozen subtler questions about structure.

When hops lie: Dijkstra

BFS’s currency is hops — every edge costs 1. But hallways have lengths, roads have minutes, flights have prices, and the fewest-edges route may be the slowest one. Put a number, a **weight**, on each edge, and ask for the route of least total weight. Hops now lie; we need a new discipline.

Here it is as a tabletop protocol — play it with coins before reading any code. Every vertex gets a tentative price tag: the start reads 0, all others read ∞ (“no known way yet”). Then repeat:

among vertices not yet *settled*, pick the one with the **cheapest** tag and settle it — its tag is now the truth; for each neighbor of the newly settled vertex, check whether traveling *through* it beats the neighbor’s current tag: settled-tag + edge-weight, and if that is cheaper, write it over the old tag.

That overwrite — “is the path through me better than your best so far?” — is called *relaxing* an edge, and the whole algorithm is just: settle the cheapest, relax its edges, repeat until done. Run it by hand on this map (it reappears in the lab; distances are minutes):

Home–Park 2, Park–School 5, Home–School 9, School–Library 1, Park–Library 9, Library–Café 4, School–Café 7, Park–Gym 11, Gym–Café 3.

From Home: Park settles at 2; School at 7 — *through the park*, beating the direct 9; Library at 8 (through School, beating Park’s 11); Café at 12 (through Library, beating School’s 14); Gym at 13 (through Park, beating Café’s 15). Every single answer is a detour beating a direct claim, which is the algorithm earning its keep: the cheap-looking edge is not the cheap path, and only patient relaxation finds out.

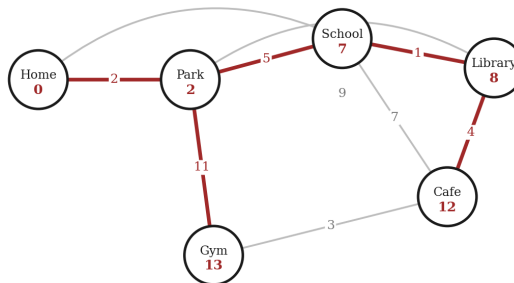


Figure 5.3 — The town, then Dijkstra’s settled tags: the red tree is the cheapest-routes answer; and every tag beats a direct edge.

Figure 5.3 — The town, then Dijkstra’s settled tags: the red tree is the

cheapest-routes answer, and every tag beats a direct edge.

Now name what you just did. *Repeatedly commit to the cheapest-looking option.* This is Chapter 2’s instinct — **Dijkstra’s algorithm is a greedy algorithm** — and after Sevenia you are right to demand the theorem before trusting it. It exists, and it is the stays-ahead argument in new clothes: when the cheapest unsettled vertex v is settled at price p , no future discovery can undercut it, because any other route to v must *exit the settled region somewhere*, and its first step outside already costs at least p — v was the cheapest exit, and edges only add. Read that twice; the deep water writes it carefully, and shows the one assumption it leans its whole weight on: **edges only add** — weights are not negative. Hand the argument a negative edge and it collapses like greedy in Sevenia, with a three-vertex counterexample waiting in Problem 3.3.

A human moment

In 1956, Edsger Dijkstra — twenty-six, employed as the Netherlands’ first professional “programmer,” a job title so new his marriage license wouldn’t accept it — was shopping in Amsterdam with his fiancée. Tired, they sat on a café terrace, and in about twenty minutes, over coffee, without pencil or paper, he designed the algorithm you just executed with coins. The constraint was the method, he later said: designing without paper “you are almost forced to avoid all avoidable complexities.” He thought so little of the result that he didn’t publish for *three years*, and then only as a three-page note. That note now guides essentially every navigation app on Earth; some appreciable fraction of all routed packets, delivered parcels, and turned-left cars are downstream of one tired afternoon. Two lessons, both real: simplicity is a discipline, not an accident — and the inventor is reliably the worst judge of which twenty minutes mattered.

Deeper water: why the cheapest exit is safe

The invariant: *every settled vertex's tag equals its true shortest distance*. True at the start (Home, tag 0). Preserved: suppose it holds, and v is the cheapest unsettled tag, value p , about to settle. Any path from the start to v begins inside the settled region and at some point crosses out — let its first unsettled vertex be u . The path's length up to u is a true settled distance plus one relaxed edge, which is exactly what u 's tag records or better, so the path already costs $\geq \text{tag}(u) \geq \text{tag}(v) = p$ by v 's minimality — *and the remaining leg from u onward cannot bring the total back down, because the remaining edges only add*. So no path to v beats p ; settling is safe. Decisive at the end: all settled, all true. ■ (Fifth appearance of the trick.)

The italicized clause is the load-bearing one. With a negative edge, “remaining edges only add” is false and the proof dies at that exact word — and so does the algorithm, as your three-vertex counterexample will show. A theorem's hypotheses are not decoration; they are the places it can be killed.

One honesty note on cost: our protocol finds the cheapest unsettled vertex by *scanning everyone*, so a map with n places and m roads costs about n^2 — fine for towns, sloppy for continents. Real implementations keep the tags in a structure that surrenders its minimum quickly (a *heap*); the algorithm is unchanged, only the line-keeping improves. Chapter 6 is entirely about what good line-keeping is worth.

Where this leaves us

One abstraction — keep the connections, burn the geography — and three disciplines for walking it. The ripple (a queue): shortest paths in hops, and certified absence when the ripple dies. The thread (a stack, or Chapter 3's recursion undisguised): any path, all of reachability, cheaply. The price tags (greedy, with a theorem): cheapest paths under honest, nonnegative weights. Between them

sits one of the quietest facts in the chapter: BFS and DFS are *one word apart*. Which end of the line you serve — that is the entire difference between the patient cartographer and the plunging maze-runner. Small hinges; large doors.

You now hold the book's first half: two questions, two strategies (greedy and the table), recursion beneath both, and the territory — graphs — where they all live. Before Chapter 6 takes up the toll booth this book has been promising since Chapter 1 (sorting: its price, and the astonishing proof that the price has a floor), there is a short interlude. It concerns a rumor you have heard all your life — that computers got fast — and the half of the story the rumor leaves out: most of the speed, measured honestly, was never in the machines. It was in chapters like the five you have just read.

Problem Set 5

★ ★ ★

Lab 5 (Dots and Lines) is the Computational Lab tier. Problems 2.3 and 3.3 expect its code as equipment.

Fluency

1.1 Draw the Königsberg graph from the description (four land masses A, B, C, D; you may invent which bridges join which, keeping seven total and every vertex odd). Label each vertex's *degree* — its count of edge-ends.

1.2 Write the hallways school as a drawing, then write this drawing back as a dictionary: vertices P, Q, R, S, T; edges PQ, PR, QR, QS, RS, ST. (Round trips between picture and dictionary should feel free by the end.)

1.3 Run BFS by hand from P on the graph of 1.2: list the rings (ring 0, ring 1, ...) and give each vertex its hop-distance. Then rerun from T and watch the rings reorganize.

1.4 Run DFS by hand from P on the same graph, exploring neighbors in alphabetical order; write the sequence of visits and, separately, the moments of backing up. Compare its visit order with BFS's.

1.5 Execute Dijkstra's protocol on the chapter's town map starting from **School** (not Home): full table of tags, with each overwrite shown struck through. Check your finished tags give the Home distance as 7.

1.6 In the town map, the city closes the School–Library path (construction). Re-run Dijkstra from Home. Which destinations got more expensive, and by how much?

Practice

2.1 *Euler’s parity argument, yours.* Prove: in any walk that crosses every edge exactly once, every vertex except possibly the first and last must have even degree. Scaffold: count each intermediate visit’s consumption of edge-ends (arrive + depart = ?). Conclude Königsberg’s stroll is impossible. Then the modern victory lap: the city (now Kaliningrad) lost two bridges to war and history — with the right two gone, does a walk exist? Choose which two to delete from your 1.1 drawing and exhibit the walk.

2.2 Word ladders: CAT $\rightarrow$ COT $\rightarrow$ DOG-style puzzles, one letter changing at a time, every rung a real word. Recast as a graph problem: what are the vertices, what are the edges, and what does BFS from CAT *guarantee* about the ladder it finds to DOG? Find by hand a shortest CAT $\rightarrow$ DOG ladder (there is one of length 3 — three edges).

2.3 “Six degrees of separation” claims any two people are ≤ 6 friendship-hops apart. Say precisely what BFS from you, on the friendship graph of Earth, would compute, ring by ring. If each person had exactly 30 *new* acquaintances per ring, how many people would ring 6 contain? Compare with Earth’s population, and state what that arithmetic does and does not prove about the claim. (Chapter 3’s exponential, now working *for* the rumor.)

2.4 Your town’s subway charges per *transfer*, not per stop: riding any single line any distance is one token. Model “fewest tokens from station X to station Y” as a shortest-path problem — the trick is choosing what the vertices and edges are (hint: lines, not just stations, may deserve to be vertices). Set it up; don’t solve.

2.5 A knight stands on a chessboard corner. “Fewest knight-

moves to the opposite corner” is a BFS problem: say what the graph is (vertices? edges? how many of each?) and run the first three rings by hand from the corner. (Lab Break-it 4 lets the machine finish; predict the full answer now and write it down.)

Concept & Counterexample

3.1 Claim: “DFS, like BFS, finds shortest paths — it explores systematically, after all.” Build the smallest maze or graph where DFS from S (neighbors explored in alphabetical order) reaches T by a path strictly longer than the shortest, and state in one sentence what the thread does that the ripple never does.

3.2 Claim: “BFS handles weighted graphs: just treat the weights as distances.” Three vertices suffice for the refutation — exhibit weights where the fewest-hops route is not the cheapest route, and say which ring BFS puts the destination in versus what Dijkstra’s tags say.

3.3 Claim: “Dijkstra works fine with a few negative weights as long as the total stays positive.” Build the three-vertex counterexample the chapter promised: Home→B direct weight 2, Home→C weight 3, C→B weight −2. Trace the protocol — at what moment does it *settle a lie*? Point to the sentence in the deep-water proof that your example kills, quoting its load-bearing clause.

3.4 Claim: “adding an edge to a graph never makes any shortest path longer.” True or false for nonnegative weights? Prove or refute. Then the sneaky cousin: “adding an edge never makes any *DFS* route longer.” (Careful: what does DFS’s route even depend on?)

3.5 Claim: “if a graph is connected, BFS from any vertex visits all vertices; therefore, to *check* connectivity, you must run BFS from every vertex.” Keep the first half, demolish the second: how many BFS runs does connectivity-checking actually need, and why?

Diamond

4.1 *The rings are the distances.* Prove: when BFS runs from vertex s , every vertex v it visits is assigned to ring $d(v)$, where $d(v)$ is the true minimum number of edges from s to v — and vertices never visited have no path from s at all. Structure it as induction on rings: (a) Floor: ring 0. (b) Step, two halves that students always conflate, so keep them apart: (i) everything BFS puts in ring $k + 1$ truly has distance $\leq k + 1$ (exhibit the path — where does it come from?); (ii) nothing of true distance $k + 1$ gets missed or placed later (its neighbor on a shortest path lives in ring k by hypothesis — finish the sentence, and find the precise line of the queue’s discipline that this half leans on). (c) The absence clause: prove the never-visited claim by contradiction, walking a hypothetical path from s and locating its first never-visited vertex. (d) One sentence: which half of (b) fails for DFS, and which for BFS-with-weights? (Your counterexamples from 3.1 and 3.2 are the witnesses; the proof tells you *where* they strike.)

Computational Lab

5.1 Lab 5 in full, predictions before runs. Your hand-run of Dijkstra (Fluency 1.5) must be checked against the machine’s tags before you trust either.

Teach It

6.1 Tell someone the Königsberg story with a napkin: draw the city, then perform Euler’s deletion — redraw as four dots and seven lines while they watch — and let them hunt for the walk before you show the parity argument. Three sentences: did the abstraction step or the parity step impress them more, and why do you think that is?

6.2 Run the Dijkstra coin protocol with someone on a map of your actual neighborhood (six or seven places, walking-minutes as weights, estimated honestly). Let them be the bureaucrat — they move the coins, you only enforce the two rules. Report the first moment a detour beat a direct edge on *your* streets, and whether they trusted it.

Lab 5: Dots and Lines

★ ★ ★

Accompanies Chapter 5. The chapter walked graphs three ways with coins and napkins; here the same three walks run on the machine. Two small tools arrive — each because the graph demands it — and a maze of your own design gets solved by ripple, by thread, and then, with one word changed, by each pretending to be the other.

1. A graph in the machine

The chapter's school, verbatim — a dictionary with one entry per vertex, holding its list of neighbors:

```
hallways = {
    "gym":      ["lobby"],
    "lobby":    ["gym", "library", "cafeteria"],
    "library":  ["lobby", "lab"],
    "cafeteria": ["lobby", "lab"],
    "lab":      ["library", "cafeteria"],
}

print(hallways["lobby"])
```

Notice the convention: every edge appears *twice* (gym lists lobby, lobby lists gym), because a hallway can be walked both ways. Check the dictionary against a drawing of the school — then deliberately desynchronize it (delete "gym" from lobby's list but keep lobby in

gym's) and hold that thought for Break-it 1.

2. The ripple

BFS, with the queue kept honest in plain sight: the waiting line is a list, and a marker head points at who gets served next. Serving means moving the marker; joining means appending. Nothing is ever deleted — the line keeps its history, which will turn out to be a gift.

One new tool, arriving with its debt: the **set**. The visited-check must be fast and duplicate-proof; a set is a bag that holds each thing at most once and answers in instantly. Three moves: `seen = set()`, `seen.add(x)`, `x in seen`.

```
def bfs(graph, start):
    queue = [start]
    head = 0
    seen = {start}                # a set with one thing
    ↪ in it
    dist = {start: 0}             # the notebook: vertex
    ↪ -> hops
    while head < len(queue):
        here = queue[head]
        head = head + 1           # serve the front of
        ↪ the line
        for neighbor in graph[here]:
            if neighbor not in seen:
                seen.add(neighbor)
                dist[neighbor] = dist[here] + 1
                queue.append(neighbor) # join the back of the
                ↪ line
    return dist

print(bfs(hallways, "gym"))
```


Run it and check every distance against your drawing. Then print queue itself after the run (add a print, or return it too): the line's history is the rings, in order — read it aloud as “ring 0, ring 1, ring 2” and confirm the bureaucracy did what the chapter promised.

3. A maze is a graph wearing a costume

Build a maze as a list of strings — # wall, . floor, S start, E exit:

```
maze = [  
    "#####",  
    "#S..#...#",  
    "#.#.#.#.#",  
    "#.#...#.#",  
    "#.###.#.#",  
    "#.....#E#",  
    "#####",  
]
```

Where is the graph? Every floor cell is a vertex; two floor cells side-by-side share an edge. We never build the dictionary — the maze *answers neighbor queries on demand* (look up, down, left, right; a graph is anywhere that can answer “who are your neighbors?”).

A cell's name is its (row, column) position — and here the second tool arrives with its debt. Positions must live in the seen set and the dist notebook, and Python's rule is that set members and notebook keys must be *unchangeable* things; a list, which can be appended to and edited, is not allowed. The unchangeable list is called a **tuple**: written (5, 7) with round brackets, indexed like a list, frozen forever at birth. Positions are tuples because positions must be look-upable.

```
def maze_bfs(maze, start):  
    queue = [start]
```

```
head = 0
seen = {start}
dist = {start: 0}
while head < len(queue):
    here = queue[head]
    head = head + 1
    r = here[0]
    c = here[1]
    for next_pos in [(r-1, c), (r+1, c), (r, c-1), (r,
↪ c+1)]:
        if maze[next_pos[0]][next_pos[1]] != "#":
            if next_pos not in seen:
                seen.add(next_pos)
                dist[next_pos] = dist[here] + 1
                queue.append(next_pos)

    return dist

distances = maze_bfs(maze, (1, 1))      # S is at row 1,
↪ column 1
print(distances[(5, 7)])                # E is at row 5,
↪ column 7
```

The ripple's answer is the shortest path length in steps. Verify it by tracing the maze with a pencil — then design your *own* maze (keep the # border, it is what spares the code from falling off the edge) with at least two routes from S to E of different lengths, and confirm BFS reports the shorter.

4. The one word

The chapter claimed BFS and DFS differ by which end of the line you serve. Test the claim with the smallest possible edit — serve the *back*:

```
def dfs(graph, start):
    line = [start]
    seen = {start}
    order = []
    while len(line) > 0:
        here = line.pop()          # .pop(): remove and
        ↪ return the LAST item
        order.append(here)
        for neighbor in graph[here]:
            if neighbor not in seen:
                seen.add(neighbor)
                line.append(neighbor)
    return order

print(dfs(hallways, "gym"))
print(bfs(hallways, "gym"))
```

(`.pop()`) is the one new move: take the last item off a list. A line served from the back is called a **stack** — the call stack of Chapter 3 is one, which is why the chapter could say DFS is recursion.) Compare the two visit orders on the school, then on your maze (adapt as in §3): the ripple sweeps level by level; the thread plunges. Same dots, same lines, opposite temperaments, one word.

5. Price tags on the town

The chapter's town, with weights — a dictionary of neighbor lists where each neighbor comes with a price, as a (place, minutes) tuple:

```
town = {
    "Home":    [("Park", 2), ("School", 9), ],
    "Park":    [("Home", 2), ("School", 5), ("Library", 9),
    ↪ ("Gym", 11)],
```

```

    "School": [("Park", 5), ("Home", 9), ("Library", 1),
↪      ("Cafe", 7)],
    "Library": [("School", 1), ("Park", 9), ("Cafe", 4)],
    "Cafe":    [("Library", 4), ("School", 7), ("Gym", 3)],
    "Gym":     [("Park", 11), ("Cafe", 3)],
}

```

Dijkstra's protocol, exactly as you played it with coins — tags, settle the cheapest, relax its edges:

```

def dijkstra(graph, start):
    tags = {start: 0}                                # tentative prices;
    ↪ absent = infinity
    settled = set()
    while len(settled) < len(graph):
        cheapest = None                               # scan for the
        ↪ cheapest unsettled tag
        for place in tags:
            if place not in settled:
                if cheapest is None or tags[place] <
                ↪ tags[cheapest]:
                    cheapest = place
        if cheapest is None:
            break                                     # the rest is
            ↪ unreachable
        settled.add(cheapest)
        for edge in graph[cheapest]:
            place = edge[0]
            cost = tags[cheapest] + edge[1]
            if place not in tags or cost < tags[place]:
                tags[place] = cost                    # relax: the detour
                ↪ wins
    return tags

print(dijkstra(town, "Home"))

```

(None is Python's word for "nothing yet" — the scan starts with no champion and crowns the first candidate.) Check every tag against the chapter's hand-run: Park 2, School 7, Library 8, Café 12, Gym 13 — five answers, five detours. Then rebuild town as **your actual neighborhood**, six or seven places with honest walking-minutes, and find something true you didn't know about your own streets.

Break it

Predict first. Run second. Explain any gap third.

1. The desynchronized graph from §1: delete "gym" from lobby's neighbor list (keep lobby in gym's). Predict `bfs(hallways, "gym")` and then `bfs(hallways, "lab")` — same reachability or different? You have built a *one-way door*. Real graphs (the web, who-follows-whom) are full of them; what single sentence of the §1 convention did you break, and which of the two BFS runs is the one that can no longer find everything?

2. In `maze_bfs`, remove the border: change your maze's top row to open floor and run from S. Predict the failure before running — wrong answer, error, or infinite loop? — then read the error's last line and explain which innocent-looking expression walked off the world. (Two repairs exist: rebuild the wall, or teach the neighbor-check to refuse positions outside the maze. Implement the second; the first you already had.)

3. *The road that pays you*. First, make Dijkstra's promises audible: add one line right after `settled.add(cheapest)` — `print("settled:", cheapest, "at", tags[cheapest])` — and rerun the honest town. Compare the announcements with the final notebook: they agree perfectly, every time, on every honest map (that agreement is the deep-water theorem talking). Now build the chapter's Problem 3.3 graph using Break-it 1's one-way doors — `{"Home": [("B", 2), ("C", 3)], "B": [], "C": [("B", -2)]}` — predict the announcements, and run it. Catch the exact moment the algorithm publicly

settles a lie (which vertex, announced at what price, versus the truth you can see by eye?). Then two follow-ups: (a) point to the clause of the deep-water proof that this graph kills, quoting it; (b) the *final* notebook actually shows the right number anyway — find the line of our code that quietly forgave the lie after the announcement, and explain why you still must not trust this on bigger graphs (what if the lying vertex’s wrong tag had been used to relax *its* neighbors before the correction arrived?). One warning while experimenting: keep negative edges one-way here. A negative *two*-way road lets a walker pace back and forth earning money forever, and “cheapest route” stops meaning anything — the question itself, not just the algorithm, dies. (Restore the town when done; it has suffered enough.)

4. *The knight’s ripple* (finishing Problem 2.5). A chessboard is rows and columns 0–7; a knight at (r, c) can jump to eight positions: $(r \pm 1, c \pm 2)$ and $(r \pm 2, c \pm 1)$, board permitting. Adapt `maze_bfs` — same skeleton, different neighbor list, and the “board permitting” check is Break-it 2’s repair earning rent. Compute the fewest moves from $(0, 0)$ to $(7, 7)$, compare with the prediction you wrote in the problem set, and then print the full `dist` notebook and find the squares that are *hardest* for a corner knight to reach. Are they where you’d have guessed?

Interlude: The Other Exponential

★ ★ ★

Between Chapters 5 and 6. No problems; ten minutes; one rumor, corrected.

You have heard, all your life, that computers got fast. The rumor is true and even understated: for half a century, the number of transistors that fit on a chip doubled roughly every two years — Moore’s law, the most famous exponential in the world — and machines rode the curve from filling rooms to fitting in pockets. (The honest footnote: the ride has been slowing. Chips stopped getting *faster* in the simple clock-speed sense around 2005, and even the transistor doubling has labored in recent years. Exponentials in the physical world are always a phase, never a law.)

Here is the half of the story the rumor leaves out. Over the same decades, for problem after problem, the *algorithms* got faster too — and where anyone has carefully measured both, the ideas have often beaten the hardware.

The best-documented case concerns linear programming — the workhorse optimization method behind airline schedules, factory plans, and shipping routes. Robert Bixby, who spent a career building the leading solvers, ran the comparison directly: take the same problems, and ask how much of the speedup between the late 1980s and the early 2000s came from better machines and how much from better mathematics. His answer, for that fifteen-year window: machines contributed a factor in the *thousands* — and algorithms con-

tributed a factor in the *tens of thousands*. The same planning problem that ran overnight in 1990 runs in seconds today, and most of the difference is not silicon. It is people refusing, more and more skillfully, to do most of the work.

You do not have to take Bixby's word for it, because you have already measured the other exponential yourself, in Lab 1, on your own machine. Searching a million-item list: a million steps the obvious way, twenty steps the sorted way. That is a factor of fifty thousand — from an *idea*, costing zero transistors, discovered before electronic computers existed. Every chapter so far has handed you another such factor. The notebook of Chapter 4 turned twenty-five billion calls into fifty. The ripple of Chapter 5 routes a continent. None of these speedups will ever flatten the way Moore's law is flattening, because they were never physical to begin with. A proof does not decay.

So when someone gestures at a data center and says "look how fast computers got," you may now silently append the correction: *look how fast the refusals accumulated*. The machines made the dumb thing quick; the chapters made most of the dumb thing unnecessary; the world you live in is the product of the two factors, and the larger factor is the quieter one.

One more thing, and then back to work. The other exponential has no fab, no supply chain, no minimum age. Its entire manufacturing base is people who have learned to ask the two questions and to count honestly — which, five chapters in, includes you. The curve runs through the reader. Chapter 6 resumes with a toll booth this book has owed you since page one, and with something rarer than a fast algorithm: a proof that no one, ever, with any idea, will beat a certain price. Even the other exponential has walls — and knowing exactly where they stand is the most valuable knowledge in the subject.

Chapter 6: Order, and Its Price

★ ★ ★

*In which the toll booth opens its books, and a fence is built
around the possible*

The bill comes due

Chapter 1 sold you a miracle on credit. Binary search answers in twenty steps what costs a million — *provided the list is sorted* — and the chapter admitted, in fine print, that sortedness is purchased in advance. Five chapters later, the invoice: what does sorting actually cost? Is the toll worth paying? And — the question that makes this chapter different from every chapter before it — could anyone, ever, sort *fast*? Not “has anyone” but “could anyone”: is there a floor under the price, built into the universe of the problem itself?

By the end you will hold a proof that there is. It is this book’s first full **impossibility theorem**, and impossibility theorems are the strangest goods mathematics sells: a fence around every algorithm that exists and every algorithm that ever will, including ones no mind has conceived. Chapter 1’s deep water built a small fence of this kind around the guessing game. Today we build the famous one.

But first, honest accounting of how sorting is actually done.

How humans sort

Pick up a dozen dealt cards the way every card player on Earth does: take them one at a time, and slide each new card leftward through the cards already in your hand until it sits where it belongs. The cards in your hand are always sorted; each arrival walks left past everything bigger than itself.

That is an algorithm — **insertion sort** — and your hands have known it for years. Its correctness needs one sentence (the hand is sorted before each arrival and sorted after — the invariant trick, which you should greet like a relative by now). The cost, in the declared currency of comparisons: the arriving card is compared against each card it walks past. Worst case — the deck dealt in perfectly reversed order — every card walks past *everything* already in hand: the first arrival makes 1 comparison, the next 2, then 3, and the bill is

$$1 + 2 + 3 + \dots + (n - 1) = n(n - 1)/2$$

— Gauss’s schoolroom sum from Lab 0, sitting in the middle of sorting, presenting the bill instead of the discount. For $n = 2,048$ cards: 2,096,128 comparisons. And the *gait*: double the deck and the bill quadruples — the n^2 signature you met in Lab 3’s lopsided tree, now arising honestly rather than by sabotage. (One virtue, recorded now and cashed later: deal the cards *already sorted* and each arrival walks past nothing — $n - 1$ comparisons total. Insertion sort notices order that is already there. Hold that.)

A human moment

Sorting was an industrial crisis before it was a subject. The United States counts its people every ten years, and the 1880 census — tallied by hand, by clerks, with paper and patience — took about eight years to process. Projections for 1890 ran *past* 1900: the country would be counting one census when the next arrived. A young engineer who had worked on the 1880 count,

Herman Hollerith, proposed heresy: punch each person's answers as holes in a card, and build machines to sort and tally the cards electrically. His tabulators processed the 1890 census in a fraction of the time, sorting millions of cards by trade, by age, by state — and the company he founded to rent out the machines became, after mergers and a renaming, International Business Machines. IBM. The computer industry is, by lineage, a *sorting* industry: before machines computed much of anything, they sorted, because sorting was where civilization's paperwork actually drowned. When this chapter asks what sorting costs, remember that for Hollerith's clerks the answer was measured in *years of a nation's attention* — and that the gap between n^2 and $n \log n$, at census scale, is the gap between drowning and not.

The halver returns

Chapter 3 already built the alternative: **merge sort** — split, sort each half, merge — with its bill read off the tree: every level's merges touch n cards, $\log_2 n$ levels, about $n \log_2 n$ element-steps. (A lineage note worth one sentence: when John von Neumann sketched programs for the first stored-program computer in 1945, among the very first things he wrote out was merge sort. The halver is as old as the machines themselves.)

Now race them in the counted currency, worst case against worst case, and read the gap as it opens:

n	insertion (comparisons)	merge (element-steps)
8	28	24
32	496	160
128	8,128	896
512	130,816	4,608
2,048	2,096,128	22,528

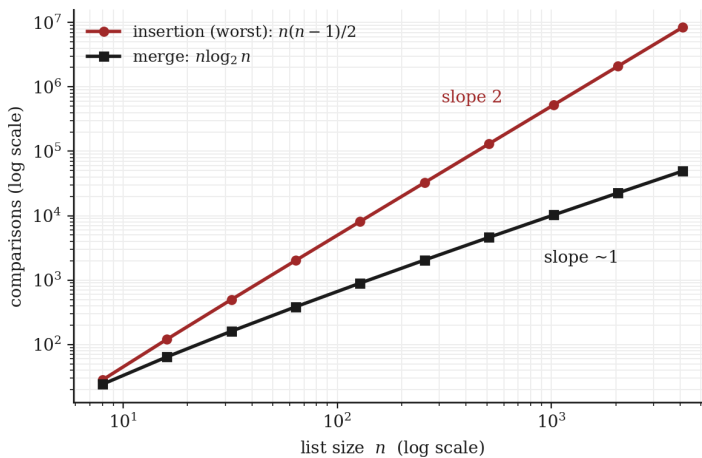


Figure 6.1 — The counted race on log-log axes: two nearly straight lines, slopes 2 and ~ 1 , diverging forever.

Figure 6.1 — The counted race on log-log axes: two nearly straight lines, slopes 2 and ~ 1 , diverging forever. In this currency the halver never loses.

At a deck of 8 the two are neighbors. At 2,048 the plodder pays *ninety times* more, and every doubling of the deck doubles the ratio again. In the counted currency, the halver never loses and never looks back — Lab 6 plots this table and the gap becomes a pair of diverging straight lines. (Then why does insertion sort exist in every real library on Earth? Because clocks are not counts. Lab 6's second experiment times both on your machine and finds a neighborhood — small n — where the plodder's low overhead *wins the clock* while losing the count. Constants, the thing our counting openly declines to chase, own the small- n neighborhood; the count owns everywhere else. This is Chapter 1's confession, audited and confirmed.)

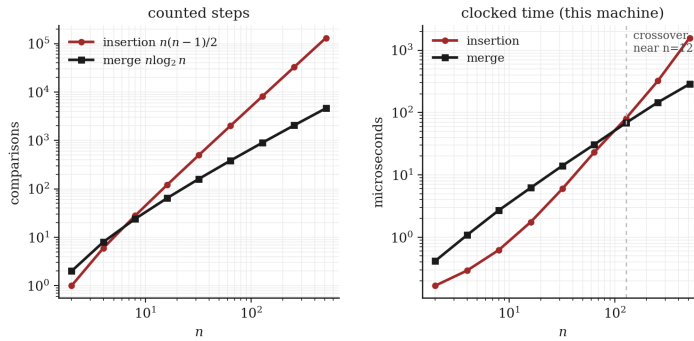


Figure 6.2 — Counted steps beside clocked seconds: the count predicts the shape everywhere; the constants own the small- n neighborhood, where the plodder’s low overhead wins the clock.

Figure 6.2 — Counted steps beside clocked seconds: the count predicts the shape everywhere; the constants own the small- n neighborhood, where the plodder’s low overhead wins the clock.

Names for the shapes, at last

Six chapters of tables have earned you the right to some professional shorthand. You have met, repeatedly, costs that grow like $\log n$, like n , like $n \log n$, like n^2 , like 2^n — and what mattered every single time was the *shape*, never the constant: not “496 comparisons” but “quadruples when n doubles.”

The notation: a cost grows as $O(f(n))$ — read “big-oh of f ” — if, for large n , it is at most a constant multiple of $f(n)$. It grows as $\Omega(f(n))$ if it is at *least* a constant multiple. $\Theta(f(n))$ claims both: the shape, exactly. So insertion sort’s worst case is $\Theta(n^2)$; merge sort’s is $\Theta(n \log n)$; binary search is $\Theta(\log n)$; the amnesiac staircase was exponential and the notebook made it $\Theta(n)$.

Three honest remarks, because notation is where understanding goes to hide:

1. *O is a ceiling, not a destination.* “Insertion sort is $O(n^2)$ ” promises it never exceeds that shape; it does not promise to reach it. On an already-sorted deck, insertion runs in $n - 1$ steps. Saying “ $O(n^2)$, so it takes about n^2 steps” is a misreading you will now hear weekly for the rest of your life; correct it gently.

2. *The dropped constants are real.* Θ tells you who wins the race eventually; the clocked crossover tells you the constants pick the winner at small n . Both are facts; the notation simply chooses, openly, to track the first.
3. *We waited on purpose.* This notation in Chapter 1 would have been vocabulary; in Chapter 6 it is bookkeeping for tables you built yourself. Notation should always arrive in arrears.

The fence

Now the chapter's summit. Merge sort pays about $n \log_2 n$. Could something pay less — n , say, or $n\sqrt{(\log n)}$, some uninvented cleverness? For algorithms that sort by **comparing** items (asking “is a before b ?” and branching on the answer — every method in this chapter, and nearly every sort ever run), the answer is *no*, and here is the proof.

Watch any comparison sort run on three items a , b , c , and write down only the questions it asks and the answers it receives. The algorithm's entire experience of the input is that sequence of yes/no answers — it never sees the items, only the comparisons. So the algorithm *is* a tree of questions: each internal node a comparison, each branch an answer, and at each leaf the algorithm must stop and announce the sorted order.

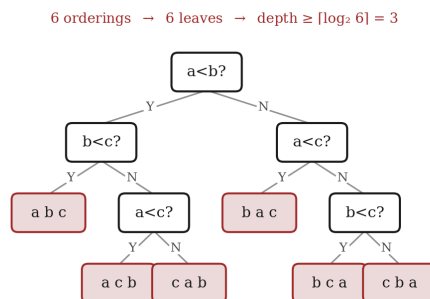


Figure 6.3 — The decision tree for sorting three items: six orderings demand six leaves; a tree of questions with six leaves cannot be shallower than three.

Figure 6.3 — The decision tree for sorting three items: six orderings demand six leaves, and a tree of questions with six leaves cannot be shallower than three.

Two facts about this tree, and the theorem falls out:

Fact 1: the tree needs $n!$ leaves. Three items can arrive in $3! = 6$ different orderings; n items in $n! = n \times (n-1) \times \dots \times 1$ orderings. Two *different* orderings must reach *different* leaves — because if they followed identical answer-paths to the same leaf, the algorithm would announce the same rearrangement for both, and the announcement would be wrong for at least one of them (the Diamond makes you nail this, and it is the proof’s load-bearing wall). Distinct situations, distinct leaves: where have you heard this? Chapter 1’s deep water, counting answer-sequences in the guessing game. Same architecture, larger building.

Fact 2: a tree of depth d has at most 2^d leaves. Each question at most doubles the possibilities — count them level by level, the tree’s geometry permits no more.

Therefore $2^d \geq n!$, that is, $d \geq \log_2(n!)$ — some input forces at least $\log_2(n!)$ comparisons, *whatever the strategy*. And $\log_2(n!)$ is genuinely of the $n \log n$ shape: even without fancy estimates, the first $n/2$ factors of $n!$ are each at least $n/2$, so $n! \geq (n/2)^{n/2}$ and $\log_2(n!) \geq (n/2) \cdot \log_2(n/2)$ — the $n \log n$ shape with a humble constant, by arithmetic a 7th grader does. (The deep water sharpens the constant with Stirling; for $n = 1,024$, $\log_2(n!) \approx 8,769$ against merge sort’s $\approx 10,240$ — the fence and the algorithm nearly touching.)

Sit with what was just proved, because it is a different *kind* of sentence than this book has mostly produced. Not “this algorithm costs $n \log n$ ” but: **no comparison sort, invented or uninvented, run on any machine, ever beats $\Theta(n \log n)$ in the worst case.** The proof never examined an algorithm. It counted *distinguishable situations* against *information per question* and fenced in every strategy at once — the move Chapter 1 made on a guessing game, now closing one of the great questions of the subject. Merge sort, the recursion

from Chapter 3 you traced with playing cards, is standing essentially *against the fence*. There was almost nothing left to find, and now you know why.

Dodging the fence

A theorem’s hypotheses, Chapter 5 taught, are where it can be killed. They are also where it can be *legally dodged* — and this fence has a gate marked clearly in its own wording: *sorts by comparing*.

Suppose you must sort a million exam scores, each a whole number from 0 to 100. Don’t compare anything. Set out 101 labeled buckets; drop each score in its bucket (one touch per score); read the buckets out in order. Total cost about $n + 101$ — **linear**, serenely under the fence. This is **counting sort**, and it contradicts nothing: it never asks “is a before b?” It exploits something the comparison model deliberately ignores — that these keys are small integers whose *values* can be used as addresses. Change what you’re allowed to know, and the impossible becomes Tuesday. (The dodge has a price tag of its own: a bucket per possible value. Scores 0–100, fine; arbitrary words or unbounded decimals, no. Problem 3.4 probes exactly where the gate closes.)

The toll-booth question that opened the chapter now closes with real numbers. Sorting costs $\Theta(n \log n)$ once; each later search costs $\log n$ instead of n . One search? Sorting wasted your time. Many searches — and “many” means roughly *more than $\log n$ of them*, a number Problem 2.1 has you make precise — and the toll repays itself forever after. It depends, and now you know exactly on what.

Deeper water: Stirling, and the fence’s exact height

The crude bound $n! \geq (n/2)^{n/2}$ proved the shape. Stirling’s approximation gives the height: $\log_2(n!) \approx n \log_2 n - n \cdot \log_2 e \approx n \log_2 n - 1.443n$. So the fence stands at $n \log_2 n$

minus a linear correction — and merge sort's $\approx n \log_2 n$ sits just above it. The remaining sliver (that $1.443n$, and the gap between element-steps and comparisons) is real, and shaving it occupied serious people for decades; *merge-insertion sort* and its descendants chase the exact minimum comparison count, which is still not known exactly for all n . Even finished questions have unfinished decimals.

Deeper water: stability, and the sort in your pocket

Lab 3's Break-it 4 made you discover, before it had a name, that merge's $\leq$ versus $<$ decides whether tied items keep their original order. The property is called **stability**, and it matters the moment items carry more than their key: sort emails by date, then stably by sender, and each sender's mail stays date-ordered inside — two sorts composing into one organization. (Unstable sorts shred the inner order; Problem 3.5 exhibits the shredding.)

And the sort actually running on your machine? Python's built-in is **Timsort** — written for Python in 2002 by Tim Peters, since adopted by Java and others — and it is this chapter folded into one algorithm: merge sort's frame, insertion sort deployed on small runs (the clocked crossover, productized), an explicit hunt for stretches that arrive already ordered (insertion's recorded virtue, generalized), stable throughout, $\Theta(n \log n)$ worst case against the fence. Lab 6's last experiment times it against your handmade merge sort; the margin you'll measure is the constants again — the same mathematics, engineered within an inch of its life, plus the small fact that it doesn't run in Python at all but in C underneath. The shapes are yours now; the constants are an industry.

Where this leaves us

The toll booth's books are open. Humans sort in n^2 ; the halver sorts in $n \log n$; the gap at scale is the gap between Hollerith's drowning clerks and a census that finishes. The shapes got their professional names — O , Ω , Θ — as receipts for tables already built. And above everything stands the fence: $n \log n$ is not the record, it is the *law*, proved by counting situations against questions, binding every comparison sort that will ever be written — with a gate, clearly marked, for those willing to know more about their keys than comparisons confess.

One thread was left deliberately loose. The fence is a *worst-case* fence; insertion sort's kindness to already-sorted decks hints that other cases have other stories. Chapter 7 walks into the most useful of them: an algorithm that *gambles* — that lets a coin flip choose its pivot — and is fast not always, not in the worst case, but *on average*, in a sense that can be made precise, trusted, and measured in your own lab. The strange comfort of the average is the last big idea of the book's first act, and the doorway to its second.

Problem Set 6

★ ★ ★

Lab 6 (Counted and Clocked) is the Computational Lab tier; problems 2.1 and 3.2 use its measurements.

Fluency

1.1 Sort the hand 7, 3, 9, 1, 8 by insertion sort on paper, drawing the hand after each arrival and counting every comparison. Then feed the same five cards in reverse-sorted order and count again; check against $n(n-1)/2$.

1.2 Compute $n(n-1)/2$ and $n \cdot \log_2 n$ (round freely) for $n = 16, 256, 4,096$. At each n , how many times more does the plodder pay?

1.3 Compute $\log_2(n!)$ exactly for $n = 3, 4, 5$ (small factorials by hand), and compare with merge sort's element-steps on the same n from Lab 3's counter. Is the fence below the algorithm at every n ?

1.4 Classify each cost into its Θ shape, citing which chapter's table you met it in: $2^n - 1$; $\lceil \log_2(n+1) \rceil$; $n(n-1)/2$; calls of the memoized staircase; element-steps of merge sort; steps of linear search.

1.5 True positions: “ f is $O(g)$ ” is a ceiling claim. For each pair, say which of O, Ω, Θ honestly relate f to g : $f = 3n^2, g = n^2$; $f = n \log n, g = n^2$; $f = 100n, g = n^2$; $f = n^2, g = n \log n$.

1.6 Using the crude bound $n! \geq (n/2)^{(n/2)}$, show by hand that $\log_2(8!) \geq 8$ — then compute $\log_2(8!)$ exactly ($8! = 40,320$) and report how loose the crude bound was. Loose but load-bearing: what did the bound's *shape* establish that its constant didn't need

to?

Practice

2.1 *The toll, exactly.* A list of n items will be searched s times. Unsorted: each search costs n . Sorted once ($n \log_2 n$) then searched: each costs $\log_2 n$. For $n = 1,000,000$, find the smallest s for which sorting first is the cheaper total. Then derive the rule of thumb in symbols and compare with the chapter’s “more than about $\log n$ searches.”

2.2 A librarian receives returned books all day and must keep the shelf ordered. Argue that her natural method is insertion sort run in slow motion — and explain why her n^2 is nothing to fix (what is her n per insertion, and what would she pay to “batch” with merge sort at closing time instead?). Sometimes the plodder is the right professional.

2.3 Design counting sort, in full sentences or code, for sorting a million dates-of-birth by *year* (years 1900–2030). State the exact cost in touches and buckets. Then the follow-up: your output groups by year but you also want each year’s people ordered by month — compose two counting sorts and state the property from the deep water your composition silently requires.

2.4 You have k sorted piles totaling n cards (the ends of k merge sorts, say). Two plans to combine them: merge pile 1 with 2, the result with 3, the result with 4, ...; or merge them as a balanced tournament, pairs of pairs. Count both plans’ element-steps for $k = 8$ equal piles of m cards each, and explain the difference using Chapter 3’s tree-of-levels picture. (You are re-deriving why the halver halves.)

2.5 “Is it already sorted?” is itself an algorithm question. Give the obvious checker and its exact cost. Then prove a small fence: any checker must, in the worst case, examine every adjacent pair — exhibit what an adversary does with any pair you skip. (A fence

argument with no tree in it; fences come in many materials.)

Concept & Counterexample

3.1 Claim: “merge sort is $O(n \log n)$ and insertion sort is $O(n^2)$, so merge sort is faster.” Produce two refutations from your own Lab 6 data: one about small n and clocks, one about a *specific input family* where insertion sort beats merge sort at every size in the counted currency. State what the claim should have said.

3.2 Claim: “to speed up insertion sort, find each card’s landing spot by binary search — comparisons drop to $O(n \log n)$, so insertion sort becomes as good as merge sort.” The comparison count is right; the conclusion is wrong. Find the cost the claim forgot to count (watch your own hands slide cards), and state insertion sort’s Θ with the repair in place. Currencies, always currencies.

3.3 Claim: “the lower bound proves sorting takes $\Omega(n \log n)$, so counting sort must have a hidden flaw.” Resolve the apparent contradiction in three sentences, quoting the exact phrase of the theorem that counting sort steps around. Then state what counting sort *cannot* do: exhibit a sorting task where no finite bucket scheme works and the fence reasserts itself.

3.4 Claim: “counting sort is $O(n)$, so it’s always the best choice when keys are integers.” The hidden variable is the universe size k . Construct a scenario (real numbers welcome: ID numbers, timestamps) where k is so much larger than n that counting sort loses to merge sort by any honest accounting — and give the rule for when the dodge pays.

3.5 Take the score-name pairs from Lab 3’s Break-it 4: $[[3, \text{“Ada”}], [1, \text{“Hua”}], [3, \text{“Noether”}]]$. Show, step by step, how sorting by score with an *unstable* method can shred a previous sort by name, while the stable method composes. Then explain why stability is free for insertion sort as humans deal cards (where does the tie naturally land?).

Diamond

4.1 *The fence, built by your hands.* Prove: every comparison-based sorting algorithm makes at least $\log_2(n!)$ comparisons on some input of n distinct items. (a) Model: argue that a deterministic comparison sort is a binary tree — what exactly are its nodes, branches, and leaves, and why does determinism matter to the modeling? (b) The load-bearing wall: prove distinct input orderings cannot share a leaf. Suppose orderings $\pi \neq \sigma$ follow identical answer-paths; the algorithm at that leaf applies one fixed rearrangement to the input; show it cannot correctly sort both, *exhibiting* the failure (where do π and σ first differ, and what does the fixed rearrangement do to that position?). (c) Geometry: prove a binary tree of depth d has at most 2^d leaves, by induction on d (the only trick, appearance the sixth). (d) Conclude $d \geq \log_2(n!)$, and finish the shape with the crude bound: $\log_2(n!) \geq (n/2)\log_2(n/2)$. (e) Two sentences of architecture: set this proof beside Chapter 1's answer-sequence bound and Chapter 3's unavoidable-event bound for Hanoi. Which of the three is this one's sibling, and what single word names what both of them count?

Computational Lab

5.1 Lab 6 in full, predictions before runs. Your clocked crossover point (or your environment's refusal to produce one) is data for problems 2.1 and 3.1 — record it.

Teach It

6.1 Deal someone twelve cards and ask them to sort their hand while you watch their hands, not the cards. Then show them they were running an algorithm with a name, count its worst case together on a reversed deal, and demonstrate the halver on the table with the

same cards. Three sentences: which fact surprised them more — that they already knew an algorithm, or that the table method beat their hands?

6.2 Tell the census story, ending not with IBM but with the fence: explain to your listener, in plain words, what it means that *nobody* — no company, no genius, no machine — will ever sort by comparisons faster than the $n \log n$ shape, and how anyone could possibly know such a thing about algorithms not yet invented. If you can convey “we counted situations, not strategies,” you own this chapter. Report the sentence that finally worked.

Lab 6: Counted and Clocked

★ ★ ★

Accompanies Chapter 6. Until now this book has counted and refused to clock. Today both instruments run side by side: the count predicts the shapes, the clock reveals the constants, and a plot — your first — shows the two sorts diverging exactly as the chapter promised. Two instruments arrive, each with its debt: the clock because constants are real, the plot because six chapters of tables have earned a picture.

1. The plodder, counted

Insertion sort, exactly as hands do it — each arrival walks left past everything bigger:

```
def insertion_sort_count(a):
    comparisons = 0
    for i in range(1, len(a)):
        arriving = a[i]
        j = i - 1
        while j >= 0 and a[j] > arriving:
            comparisons = comparisons + 1
            a[j + 1] = a[j]          # shift the bigger card
            ↪ right
            j = j - 1
        if j >= 0:
            comparisons = comparisons + 1  # the comparison
            ↪ that said "stop"
```



```

        a[j + 1] = arriving
    return comparisons

hand = [7, 3, 9, 1, 8]
print(insertion_sort_count(hand), hand)

```

One behavior to notice and remember: this sorter works **in place** — it rearranges the very list you hand it, the way your hands rearrange the actual cards (print hand after the call: it changed). Merge sort, by contrast, built a *new* sorted list and left its input alone. The difference seems cosmetic today; in §3 it becomes load-bearing. Check the count against your paper run from Fluency 1.1. Then the gait audit, worst case (reverse-sorted), in the house style:

```

for n in [8, 32, 128, 512, 2048]:
    print(n, insertion_sort_count(list(range(n, 0, -1))))

```

Verify each row equals $n(n - 1)/2$ exactly, and read the gait: n quadruples each row — the count multiplies by *sixteen*. The n^2 signature, measured.

2. The race, counted

Retype merge sort and its step counter from Lab 3 (merge, merge_sort_count) — retyping is review — then race the two on the same worst-case inputs:

```

for n in [8, 32, 128, 512, 2048]:
    ins = insertion_sort_count(list(range(n, 0, -1)))    #
    ↪ fresh worst case each -
    mrg = merge_sort_count(list(range(n, 0, -1)))[1]    #
    ↪ remember: insertion mutates
    print(n, " insertion:", ins, " merge:", mrg, " ratio:",
          ↪ round(ins / mrg, 1))

```

Read the ratio column: about 1 at the top, about 90 at the bottom, roughly doubling with each doubling of n . In the counted currency the halver wins at *every* size on this table — even $n = 8$ — and the gap only opens. Hold that fact; the clock is about to complicate it.

3. The clock arrives — with a protocol

Python's stopwatch is `time.perf_counter()`: call it, do work, call it again, subtract. But a computer is a busy place — your timing shares the machine with everything else running — so single measurements lie. The professional protocol: **repeat many times, take the median** (the middle value when sorted — outliers from a busy moment fall off the ends).

```
import time

def clock(sorter, items, repeats):
    times = []
    for _ in range(repeats):
        copy = list(items)
        start = time.perf_counter()
        sorter(copy)
        times.append(time.perf_counter() - start)
    times.sort()                                # the built-in sort; §6
    ↪ turns the clock on it
    return times[len(times) // 2]              # the median
```

(Note the line `copy = list(items)`: each repeat sorts a **fresh copy**. For an in-place sorter this is not tidiness, it is the experiment's integrity — predict what the second repeat would be measuring without it, then hold the thought for Break-it 2.)

4. The clocked race, and the neighborhood

Now both sorts on the clock. Plain versions without counters (re-type `merge_sort` from Lab 3; write `insertion_sort` by deleting the counter from §1), shuffled inputs this time — the clock’s question is “typical day,” and `random.shuffle` deals one:

```
import random
random.seed(7)                                # same shuffles every
↪ run: experiments should repeat

for n in [4, 8, 16, 32, 64, 128, 256]:
    items = list(range(n))
    random.shuffle(items)
    t_ins = clock(insertion_sort, items, 51)
    t_mrg = clock(merge_sort, items, 51)
    winner = "insertion" if t_ins < t_mrg else "merge"
    print(n, "  insertion:", round(t_ins * 1e6, 1), "μs
    ↪ merge:",
          round(t_mrg * 1e6, 1), "μs  →", winner)
```

Predict before running: §2 said merge wins every count — does it win every clock? Then read the table. On most machines a *neighborhood* appears at small n where insertion wins the clock while losing the count: the plodder is a simple creature with almost no overhead, and at a dozen items, overhead is most of the bill. Find your machine’s crossover point (the n where the winner flips) and write it down — it is data for Problem 2.1, and it is the constant factor, located. The count predicted the shapes; the constants own the neighborhood.

5. The first plot

Six chapters of tables; time for the picture. `matplotlib` draws it — three moves for now: `plt.plot` draws a curve from lists of `x` and `y` values, `plt.loglog` does the same with both axes multiplying (so power laws become straight lines), and `plt.show()` reveals.

```
import matplotlib.pyplot as plt

ns = [8, 32, 128, 512, 2048]
ins_counts = []
mrg_counts = []
for n in ns:
    worst = list(range(n, 0, -1))
    ins_counts.append(insertion_sort_count(worst)[1])
    mrg_counts.append(merge_sort_count(worst)[1])

plt.loglog(ns, ins_counts, "o-", label="insertion (counted)")
plt.loglog(ns, mrg_counts, "s-", label="merge (counted)")
plt.xlabel("n")
plt.ylabel("steps")
plt.legend()
plt.show()
```

Two nearly straight lines, diverging — and their *slopes* are the exponents: the insertion line climbs twice as steeply (n^2 against $n \cdot \log n$'s almost-1). You are looking at Θ notation before it had a name: the plot ignores constants (they only shift a line up or down on these axes) and shows shape, which is exactly the deal the notation offers. Then plot the clocked data from §4 the same way and find the crossover as a *crossing of curves* — the picture of the constants' neighborhood.

6. The humbling

One more clock. Python ships with `sorted()` — Timsort, the deep water’s engineered descendant of exactly the two algorithms on your screen. Predict the ratio before running: your merge sort versus the professionals, $n = 2,048$, shuffled. Twice as fast? Five times?

```
items = list(range(2048))
random.shuffle(items)
t_mine = clock(merge_sort, items, 11)
t_builtin = clock(sorted, items, 11)
print("mine:", round(t_mine * 1e3, 2), "ms   built-in:",
      round(t_builtin * 1e3, 2), "ms   ratio:", round(t_mine /
      ↪ t_builtin))
```

The ratio is humbling — and the humility has *structure*, so spend it wisely. The built-in is the same $\Theta(n \log n)$ as yours; the fence guarantees it cannot be a different shape. The entire margin is constants: decades of engineering, plus the quiet fact that `sorted` runs in C beneath Python’s floor. Your algorithm and theirs will scale *identically* — double n and both double-and-a-bit — which you can verify by clocking both at 4,096 and checking the two ratios-to-before. The shapes are yours; the constants are an industry; and you now own the instrument that tells those two stories apart.

Break it

Predict first. Run second. Explain any gap third.

1. Feed §2’s counted race *already-sorted* inputs instead of reversed: `list(range(n))` in both places. Predict both columns before running. One algorithm’s column collapses to almost nothing; which, why, and what is its exact new count? Connect to the chapter’s recorded virtue — and to why “insertion sort is $\Theta(n^2)$ ” needed the words *worst case* attached.

2. Sabotage the timing protocol: change `clock` so all repeats share one list instead of sorting fresh copies (`copy = items` instead of `list(items)`). Predict what happens to insertion sort's clocked numbers and *why* — then run §4's race with the corrupted clock and watch the standings change. Which algorithm benefited from the corruption, and what does Break-it 1 have to do with it? (Measurement bugs that flatter one contestant are the most dangerous kind; you have just built one.)

3. Time a *single* run instead of a median of 51 (set repeats to 1) and run the $n = 64$ comparison ten times in a loop, printing the winner each time. Predict: same winner all ten times? Report the tally. The median protocol exists because of what you just witnessed; state the lesson in one sentence about believing any single measurement, anywhere, ever.

4. Build the chapter's counting sort for exam scores (whole numbers 0–100): 101 buckets, one pass in, read out in order. Count its touches at $n = 1,000,000$ (simulate with random scores) and clock it against sorted on the same data. Predict the winner first — you are betting a linear algorithm against the best-engineered $n \log n$ on Earth. Then betray it: change the keys from scores to `random.random()` decimals and explain, in the theorem-and-gate language of the chapter, why your algorithm doesn't just slow down but *stops making sense*.

Chapter 7: The Algorithm That Flips Coins

★ ★ ★

*In which an adversary is defeated by dice, and an average earns
the right to be trusted*

The adversary

Here is a sorting algorithm with an old and excellent idea — **quick-sort**. Pick one item from the list and call it the **pivot**. Walk the list once, dealing everything smaller than the pivot to its left and everything bigger to its right (one comparison per item: cost $n - 1$). The pivot now sits exactly where it belongs, forever. Sort the two sides — *somehow*, and by Chapter 3 you know what fills in “somehow.”

The bill depends entirely on the pivot’s quality, and you have already met both fates — they are Chapter 3’s two trees. A pivot near the **middle** splits the list in half: halving recursion, n work per level, $\log n$ levels, the $n \log n$ discount. A pivot at the **extreme** peels off one item and leaves $n - 1$: the lopsided vine you built in Lab 3’s Break-it 3, n levels, the n^2 bill, and (you watched this happen) a recursion n calls deep that the machine’s stack refuses.

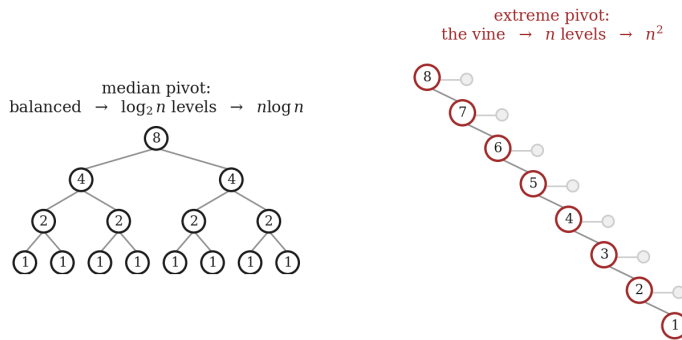


Figure 7.3 — Two fates of a pivot: median pivots build Chapter 3's balanced tree; extreme pivots grow Lab 3's vine.

Figure 7.3 — Two fates of a pivot: median pivots build Chapter 3's balanced tree; extreme pivots grow Lab 3's vine.

So everything rides on choosing pivots well — and here the trouble starts. Suppose the rule is simple and fixed: *always take the first item*. Now imagine an **adversary**: someone who knows your rule and supplies your input. Against first-item pivots, the adversary hands you an already-sorted list — the first item is the minimum, the worst pivot possible, at *every level of the recursion*. Your clever algorithm runs its full n^2 on the most innocent-looking input in the world, and in Lab 7 you will watch it die of exactly this. Change the rule — last item, middle item, some fixed recipe — and the adversary recomputes and ruins you again. This is the iron law of fixed strategies: **any pivot rule the adversary can read, the adversary can defeat**. The worst case isn't an accident of one rule; it is a consequence of being predictable.

The move

So stop being predictable. **Choose the pivot by flipping coins** — uniformly at random from the current list, fresh randomness at every level.

Feel precisely what changed, because it is a shift in the logic, not the code. Before: the adversary picks the input *after* reading your rule, so there exists a bad input. After: the adversary still picks any input it likes — sorted, reversed, crafted with infinite malice — but your coins are flipped *afterward*, and the coins do not read lists. Against every input, most pivots are decent (half the items split off at least a quarter of the list — mediocrity is abundant). There is no longer such a thing as a **bad input**; there is only **bad luck**, and bad luck, unlike a bad input, cannot be arranged by anyone.

The worst case did not vanish — be honest about this. The coins could come up cruel every time, and the n^2 bill would arrive; Problem 3.1 makes you stare at that possibility. What changed is its *status*: from “the adversary’s certainty” to “an improbability on every input, of the same kind as shuffling a deck into perfect order.” We have traded a guarantee for a gamble. The rest of the chapter is about why this particular gamble deserves your trust — which requires knowing, honestly, what “fast on average” means.

What an average promises

Expected value is the average of the outcomes weighted by their probabilities — the long-run average if you played forever. A die’s expected roll is $(1+2+3+4+5+6)/6 = 3.5$: a number the die never shows, and yet the most useful single fact about it.

The tool that makes expectation *compute* is one fact, modest-looking and inexhaustible — **linearity**: the expectation of a sum is the sum of the expectations, *always*, even when the pieces influence each other. No independence required, no fine print. Watch it demolish a problem that looks hopeless. The **hat-check story**: n people check their hats; the clerk loses the tickets and returns hats at random. On average, how many people get their own hat back?

The full analysis looks dreadful — whether you get your hat affects whether I get mine. Linearity ignores the entanglement. Per-

son i gets their own hat with probability $1/n$ (one right hat among n , symmetry). The number of lucky people is a sum of n such person-by-person events, so its expectation is the sum of the pieces: $n \times (1/n) = 1$. One lucky person on average — for ten hats, for a million hats, for any n whatsoever. The interactions, which terrified us, were never asked to testify. That is linearity's standing offer: *decompose the quantity into small events, price each alone, add*. It is the table method's spirit — refuse the whole, price the pieces — applied to chance, and it is the workhorse of every average in this subject.

Applied to quicksort (the full derivation is the deep water; here is its shape): decompose the total comparisons into pair-by-pair events — do items i and j ever get compared? They do exactly when one of them is the *first pivot chosen* from the stretch of items between them, which for distant pairs is unlikely; price each pair, add them all, and the sum comes out near $2n \ln n$ — the $n \log n$ shape, with a particular constant, as a theorem about the average. For $n = 256$ that is a few thousand comparisons against a worst case of 32,640.

But an average alone can mislead (the hat-check average is 1 whether the outcomes are steady or wild), so the lab does the thing this book always does: it *measures the whole distribution*. Two thousand independent runs of randomized quicksort on the same input, every comparison counted, all of it poured into your first **histogram** — and the picture is the argument: a tight hill near the mean, thin tails, and the worst case so far to the right it cannot share the page. The gamble is not a coin flip between fast and slow; it is fast with overwhelming concentration, and you will have seen it with your own instrument rather than taken it on faith.

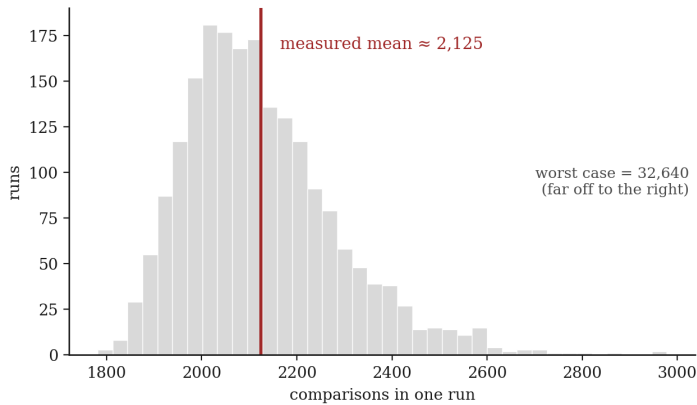


Figure 7.1 — Two thousand gambles, one hill: randomized quicksort's comparison counts at $n = 256$, the mean at the measured value, and the worst case so far right it cannot share the page.

Figure 7.1 — Two thousand gambles, one hill: randomized quicksort's comparison counts at $n = 256$, the mean at the measured 2,125, and the worst case so far right it cannot share the page.

A human moment

Computing with deliberate randomness was born in convalescence, over a card game. In 1946 Stanisław Ulam — recovering from a severe illness, playing endless rounds of solitaire — asked himself what fraction of solitaire deals are winnable. The honest combinatorics was hopeless. Then the lazier, deeper thought: *deal a hundred random games and count*. Back at Los Alamos, Ulam and John von Neumann (his second appearance in two chapters — the man is unavoidable) realized the trick was general: problems too tangled to solve exactly — neutrons rattling through a bomb core, integrals with no formula — could be *sampled* instead, on the new electronic computers, with manufactured randomness. The approach needed a code name, and their colleague Nicholas Metropolis supplied one from family gossip: Ulam had an uncle who was forever borrowing money to gamble at **Monte Carlo**. The name stuck to the method, then to a whole way of computing. Within it sits this chapter's lesson in miniature: when exact

analysis closes its door, an honest die and a fast counter open a window — and the window, used carefully, can be made as trustworthy as the door.

The trick under the notebook

One debt from Chapter 4 remains unpaid. The notebook — Python’s dictionary — answers “is this question already written down?” essentially *instantly*, on notebooks of any size. Lists can’t do that (finding a thing in an unsorted list is Chapter 1’s linear search). What is the dictionary doing?

It is gambling, and winning the same way quicksort does. The scheme is called **hashing**: keep k buckets; put each key through a fixed scrambling function — the **hash function** — that turns the key into a bucket number, spreading keys across buckets like a fair die; to look a key up, recompute its bucket number and look *only there*. One arithmetic step to find the bucket, and then a search within it — fast exactly when buckets stay small.

So everything turns on **collisions**: different keys landing in the same bucket. How soon do collisions appear? That question has a famous, disguised twin. A class of k students, 365 possible birthdays: how likely is it that *some two students share one*? Intuition says you’d need a hundred-plus students for an even chance. The truth — compute it, then simulate it in the lab until belief arrives — is **23**. With 23 people, the probability of a shared birthday already exceeds one half (0.507, exactly); at 50 people it is 0.97. The reason intuition fails by a factor of five: you instinctively count *people*, but collisions are about *pairs*, and 23 people contain 253 pairs — each pair an unlikely coincidence, but 253 lottery tickets add up. (The Diamond has you prove the threshold of 23 with one inequality from Book 2 and no calculator, via a numerical coincidence so clean it feels illegal.)

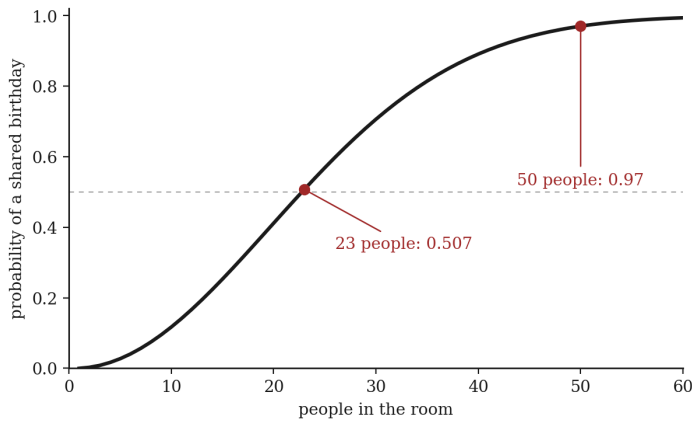


Figure 7.2 — The birthday curve: probability of a shared birthday against group size, crossing one-half at 23 and saturating by 50.

Figure 7.2 — The birthday curve: probability of a shared birthday against group size, crossing one-half at 23 and saturating by 50.

The birthday arithmetic delivers hashing’s honest verdict, both halves. The pessimistic half: collisions arrive *shockingly early* — around $\sqrt{k}$ keys into k buckets — so no hash table outruns collisions, and the design must survive them (each bucket holds a little list; lookups search it). The optimistic half: early collisions are not *many* collisions. Keep roughly as many buckets as keys, and the *average* bucket holds about one item — so the average lookup is an arithmetic step plus a glance: **constant time, on average**, by the same logic that priced the hats. Chapter 4’s miracle, retroactively explained: the notebook was fast because it was gambling on a fair scramble, and the gamble concentrates just like quicksort’s. The two halves of this chapter are one idea wearing two costumes — randomness, honestly priced, converts worst-case dread into average-case contracts you can measure, trust, and build on.

Deeper water: the average, exactly

Quicksort’s expected comparisons, by linearity. Sort distinct items; name them by rank, 1 through n . Decompose: for each pair $i < j$, let the pair’s contribution be 1 if i and j are ever com-

pared, else 0; the total comparisons is the sum over all pairs.

When are ranks i and j compared? Only when one of them is chosen as a pivot while both are still in the same sublist — and the sublist containing both always contains the whole stretch $i, i+1, \dots, j$ (nothing between them can leave for the other side before they're separated). Watch that stretch of $j - i + 1$ items: the *first* of them to be picked as a pivot decides everything. If it's i or j , they are compared (the pivot meets everyone present); if it's anything strictly between, they are sent to opposite sides, never to meet. By the fairness of the coins, each of the $j - i + 1$ candidates is equally likely to be first, so

$$P(i, j \text{ compared}) = 2 / (j - i + 1).$$

Adjacent ranks are always compared ($2/2 = 1$ — someone must separate them, and only they themselves can); the extreme pair almost never ($2/n$). Sum over all pairs, group by distance $d = j - i$, and the sum telescopes into harmonic numbers: $E = 2(n+1)H_n - 4n$, where $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \approx \ln n$. Hence $E \approx 2n \ln n$ for large n — with the honest small-print this book owes you: at $n = 256$ the exact formula gives $\approx 2,125$ while the asymptotic slogan says 2,839, a third too high. Your histogram's hill will sit on the exact number, not the slogan. Asymptotics are shadows; at toy sizes, measure.

Deeper water: two kinds of gambler

Quicksort's gamble never risks the *answer* — every run sorts correctly; only the *time* is random. Algorithms of this temperament are called **Las Vegas**. The other temperament — fixed time, but a small probability of a wrong answer — is called **Monte Carlo**, after the vignette's method, and it is not a compromise for the desperate: the fastest known ways to test whether a 600-digit number is prime are Monte Carlo, wrong with probability less than one in 2^{100} per run — and runs can be repeated, squaring the doubt away, until the error probability is dwarfed by the chance of the computer being struck by a

meteor mid-calculation. At that point, philosophy must decide what “knowing” ever meant; engineering, sensibly, ships it. (Your phone performed such a test, several times, the last time it opened a secure connection.)

Where this leaves us

Predictability was the vulnerability: any fixed rule, read by an adversary, has a killer input. Coins removed the vulnerability by removing the readability — bad inputs became bad luck, and bad luck, priced by linearity and verified by histogram, turned out to be a tight hill far from the cliff. The same gamble runs the dictionary: scramble fairly, collide early but thinly, answer in constant time on average. And “average” itself was promoted from a vague comfort to a computed, concentrated, instrument-checked quantity.

Hold that promotion carefully, because the book’s second act now begins, and it runs on nothing else. Chapter 8 asks what it means for a machine to *learn* — and the answer, it will turn out, is built from this chapter’s materials: a quantity averaged over examples, minimized by a greedy walk, trusted not because any single answer is guaranteed but because the average is honest and the distribution concentrates. “Approximately correct, on average” sounds like a retreat from Chapter 1’s two questions. It is actually their final form.

Problem Set 7

★ ★ ★

Lab 7 (The Honest Die) is the Computational Lab tier; the Diamond's part (b) and problem 2.4 lean on its simulations.

Fluency

1.1 Compute the expected value of: one roll of a fair die; the sum of two dice; the larger of two dice (enumerate the 36 outcomes — no shortcut yet). Which of the three did linearity make trivial, and which made you work?

1.2 Hat-check with $n = 3$, by hand: list all 6 ways three hats can return, count the lucky people in each, and average. Confirm the theorem's answer of 1, and note which single outcome did most of the lifting.

1.3 Run one quicksort partition by hand: list 5, 9, 2, 8, 1, 6 with pivot 6 (chosen by a generous coin). Show the left pile, right pile, the pivot's final position, and the comparison count.

1.4 For randomized quicksort on $n = 8$ distinct items, compute $P(\text{ranks } i, j \text{ compared})$ for the pairs (3, 4), (2, 5), and (1, 8). Which pair's fate is certain, and why does the proof say so?

1.5 Birthday product, small: compute by hand the probability that 4 people all have different birthdays, and from it $p(\text{some pair shares})$. Keep four decimal places; the smallness is the point — and the Diamond explains how 0.016 becomes 0.507 by $k = 23$.

1.6 Verify by direct enumeration that randomized quicksort on

3 distinct items makes $8/3$ expected comparisons: the first pivot is each rank with probability $1/3$ — trace each case's total and average. Then check $8/3$ against the deep water's exact formula $2(n+1)H_n - 4n$ at $n = 3$.

Practice

2.1 Expected rolls until a die first shows a six: call the answer E and notice that after one roll you have either finished (probability $1/6$) or are *exactly where you started* (probability $5/6$) — write the one-line equation this self-similarity implies and solve it. (Recursion's ghost, haunting probability.)

2.2 A cereal brand packs one of 4 equally likely toys per box. Expected boxes to collect all 4? Decompose by linearity into four waiting times — already have c distinct toys, the next new one is a “six” on a die with what success probability? — and sum. Then simulate in the lab and compare.

2.3 Your school has 730 lockers and assigns them by hashing student names. With 300 students, estimate via the birthday lens: is a collision likely? Roughly how many students before collisions are *expected* (the $\sqrt{k}$ scale)? And the design question: what must the locker system do about collisions that the no-collision fantasy ignores?

2.4 *Sizing the gamble.* You will store $m = 1,000$ keys and want the probability of *any* collision below $1/2$ (a luxury — explain first why hash tables don't actually need this). Using the Diamond's inequality machinery in reverse, estimate how many buckets k you'd need, and compare k to m . Then the honest engineering question: with the standard $k \approx m$ instead, what is the *expected number* of collisions (linearity over pairs: $m(m-1)/2$ pairs, each colliding with probability $1/k$), and why is that number livable?

2.5 The fixed-person birthday: how many people must be in the room before someone likely shares *your* birthday? Set up the prod-

uct, show the answer is near 253, and write two sentences for a friend on why 23 and 253 answer different questions — naming the exact phrase in each question that changes the count of lottery tickets.

Concept & Counterexample

3.1 Claim: “randomized quicksort has no worst case.” Exhibit, for $n = 4$, an explicit sequence of pivot choices under which the algorithm performs its full quadratic bill — then compute the probability of that sequence and state the claim correctly. Improbable and impossible are different words; the chapter’s trade was one for the other.

3.2 Claim: “the adversary can still defeat randomized quicksort: just supply sorted input, which was the killer before.” Explain the quantifier shift that breaks this — *when* are the coins flipped relative to the input choice? Then find the loophole that resurrects the adversary in practice: your lab seeds its generator with a fixed number. If the adversary knows the seed, what can it reconstruct, and what does this say about the word “random” in “pseudo-random”? (This is a real attack class; the repair — secret, unpredictable seeds — is a real industry.)

3.3 Claim: “the hat-check average is 1, so usually about one person gets their hat.” For $n = 3$, your 1.2 enumeration has the distribution — is the claim roughly honest there? Now invent a different game whose average is also 1 but where “usually about 1” is badly false (two outcomes suffice). State the moral: what does an expectation alone *not* tell you, and which instrument from the lab fills the gap?

3.4 Claim: “a hash function that spreads keys evenly is all you need.” The adversary disagrees: if the function is *public and fixed*, what can an adversary who wants to hurt you construct, and what happens to your “constant time on average” when every key lands in

one bucket? Connect this to the chapter’s iron law of fixed strategies — hashing’s repair is the same as quicksort’s; say what it is.

3.5 Claim: “366 people guarantee a shared birthday, so the birthday ‘paradox’ is just the pigeonhole principle in slow motion.” Half right. State exactly what pigeonhole proves at 366, what it proves at 23 (nothing!), and what kind of statement — in the vocabulary of this chapter — the 23 result is instead. Two true theorems, two different species; name both species.

Diamond

4.1 Twenty-three, by inequality alone. Let $q(k)$ be the probability that k people have k different birthdays (365 days, all equally likely). (a) Derive $q(k) = (1 - 1/365)(1 - 2/365)\cdots(1 - (k-1)/365)$, arguing each factor as a conditional step: given no collision so far, the next person must dodge how many taken days? (b) Book 2’s inequality: $1 - x \leq e^{-x}$ for all x . Apply it factor by factor to get $q(k) \leq e^{-k(k-1)/730}$, showing where the sum $1 + 2 + \dots + (k-1)$ enters. (c) Conclude: a shared birthday has probability at least $\frac{1}{2}$ as soon as $k(k-1) \geq 730 \ln 2$. Compute $730 \ln 2 \approx 505.97$ and $23 \times 22 = 506$, and savor the margin of 0.03 — then state the theorem you have proved with no calculator beyond one logarithm: at $k = 23$, $p \geq 1 - e^{-506/730} > \frac{1}{2}$. (d) The general moral: for k keys into N buckets the same argument gives the collision threshold at $k \approx \sqrt{2N \ln 2} \approx 1.18\sqrt{N}$. Verify it predicts 23 for $N = 365$, and use it to answer: how many random 8-digit ID numbers before a collision is more likely than not? (e) One sentence: the inequality in (b) points the *useful* direction — it upper-bounds q , hence lower-bounds the collision probability. If it pointed the other way, what exactly would your “proof” of (c) be worth?

Computational Lab

5.1 Lab 7 in full, predictions before runs. Your histogram and its mean are evidence in problems 3.3 and the Diamond's part (b) check; export the numbers.

Teach It

6.1 Run the birthday bet live: gather any group of 23+ (a class, a team roster, a family reunion — or a list of 23 historical mathematicians' birthdays pulled from a shelf) and check for a shared birthday, *after* recording everyone's prediction. Whatever happens, explain the 253 pairs. Three sentences: did the demonstration or the pairs-argument move them more — and which *should* it have been, by this book's own standards?

6.2 Explain to someone why a casino is calm. They face thousands of gamblers, any one of whom might win big tonight — yet the house sleeps well. Use this chapter's words: whose worst case is whose bad luck, what does linearity promise the house per bet, and what does concentration add that the average alone doesn't? Then the turn: explain that randomized quicksort *is* the house. Report whether they bought it.

Lab 7: The Honest Die

★ ★ ★

Accompanies Chapter 7. One instrument arrives — manufactured randomness — and with it the chapter's whole case gets tested on your machine: the adversary kills the predictable quicksort in front of you, the coins rescue it, two thousand runs pour into your first histogram, and the birthday paradox is simulated until belief arrives. Throughout, one discipline: predict the average before the die rolls.

1. The die

Python's randomness lives in the `random` module. Four moves, each with its debt paid immediately:

```
import random

print(random.randrange(6))          # a fair choice from
↪ 0,1,2,3,4,5
print(random.choice(["heads", "tails"]))
items = [1, 2, 3, 4, 5]
random.shuffle(items)              # an honest shuffle, in
↪ place
print(items)
```

First experiment: a die's expected roll is 3.5 — a number the die never shows. Watch the long run converge on it:

```
total = 0
for count in [10, 100, 1000, 10000, 100000]:
    total = 0
    for _ in range(count):
        total = total + random.randrange(1, 7)
    print(count, "rolls, average:", total / count)
```

(`_` is the polite name for a loop variable nobody uses.) Read the column: wobbly at 10, respectable at 1,000, locked on at 100,000. The expectation is not a prophecy about one roll; it is a contract about many — and you have just watched the contract enforce itself. The enforcement has a name, the *law of large numbers*, and this little loop is the only proof of it most working scientists ever need.

2. The hat-check, simulated

The chapter priced the hat-check at exactly 1 lucky person, for any n , by linearity. Audit the claim — a random return of hats is a shuffle:

```
def lucky_people(n):
    hats = list(range(n))
    random.shuffle(hats)
    lucky = 0
    for person in range(n):
        if hats[person] == person:
            lucky = lucky + 1
    return lucky

for n in [3, 10, 100, 1000]:
    total = 0
    for _ in range(10000):
        total = total + lucky_people(n)
    print(n, "people, average lucky:", total / 10000)
```

Four sizes, one answer, hovering at 1.0 — the n -independence that looked too clean on paper, holding in the wild. (Run it twice; the third decimal changes. Hold that thought for §5.)

3. The adversary's kill, then the coins' res-cue

Quicksort with the *predictable* rule — first item as pivot — and a comparison counter riding along:

```
def quicksort_first(items):
    if len(items) <= 1:
        return [items, 0]
    pivot = items[0]
    smaller = []
    bigger = []
    for x in items[1:]:                # items[1:]: everything
        ↪ after the pivot
        if x < pivot:
            smaller.append(x)
        else:
            bigger.append(x)
    left = quicksort_first(smaller)
    right = quicksort_first(bigger)
    return [left[0] + [pivot] + right[0], left[1] + right[1] +
        ↪ len(items) - 1]

shuffled = list(range(200))
random.shuffle(shuffled)
print("shuffled input:", quicksort_first(shuffled)[1],
    ↪ "comparisons")
```

On shuffled input: respectable, $n \log n$ -ish. Now be the adversary — you know the rule, so supply the killer:

```
print("sorted input:", quicksort_first(list(range(200)))[1],
      ↪ "comparisons")
```

Quadratic: $19,900 = 200 \cdot 199 / 2$, the full bill, on the most innocent input there is. Then push the adversary's advantage to its conclusion: try `list(range(5000))` and read the error to its last line — Lab 3's old enemy, because the lopsided recursion is n deep. The adversary didn't just slow your algorithm; it *crashed* it, with sorted data.

The rescue is one line — the pivot by coin:

```
def quicksort_random(items):
    if len(items) <= 1:
        return [items, 0]
    pivot = random.choice(items)
    smaller = []
    bigger = []
    for x in items:
        if x < pivot:
            smaller.append(x)
        elif x > pivot:
            bigger.append(x)          # equal-to-pivot items
            ↪ sit out: they're placed
    left = quicksort_random(smaller)
    right = quicksort_random(bigger)
    return [left[0] + [pivot] + right[0], left[1] + right[1] +
           ↪ len(items) - 1]

print(quicksort_random(list(range(5000)))[1], "comparisons on
      ↪ the killer input")
```

The input that crashed the predictable version is now unremarkable. Run that cell five times: five *different* comparison counts, all in the same healthy neighborhood. The cost has become a random quantity — which is exactly why the next section needs a new in-

strument.

4. The distribution, drawn

A random cost is not one number; it is a *shape*, and the instrument for shapes is the **histogram**: chop the number line into bins, count how many runs land in each, draw the counts as bars. One new plotting move, `plt.hist`:

```
import matplotlib.pyplot as plt

n = 256
counts = []
for _ in range(2000):
    counts.append(quicksort_random(list(range(n)))[1])

mean = sum(counts) / len(counts)
print("mean:", round(mean, 1), "    smallest:", min(counts), "
↪   largest:", max(counts))

plt.hist(counts, bins=40)
plt.xlabel("comparisons (n = 256)")
plt.ylabel("how many of the 2000 runs")
plt.show()
```

Read your hill. The mean sits near 2,125 — the deep water's exact $2(n+1)H_n - 4n$, *not* the asymptotic slogan $2n \ln n \approx 2,839$; check both against your printout and note which one your data votes for. Then the question the picture answers and the mean cannot: where is the worst case? It is $n(n-1)/2 = 32,640$ — find the largest of your 2,000 runs and see how breathtakingly far short of the cliff even your *unluckiest* run fell. Two thousand genuine gambles, and the catastrophe wasn't merely avoided; it was never in the room. That concentration — not the average alone — is what the chapter

meant by trusting the gamble.

5. The birthday machine

The chapter claims 23 people make a shared birthday more likely than not. Exact arithmetic first:

```
def p_shared(k):
    q = 1.0
    for i in range(k):
        q = q * (365 - i) / 365
    return 1 - q

for k in [5, 10, 23, 50]:
    print(k, "people:", round(p_shared(k), 4))
```

There it is: 0.5073 at 23. But this lab's standard is belief through simulation — so build the room:

```
def room_has_collision(k):
    seen = set()
    for _ in range(k):
        day = random.randrange(365)
        if day in seen:
            return True
        seen.add(day)
    return False

for k in [10, 23, 50]:
    hits = 0
    for _ in range(10000):
        if room_has_collision(k):
            hits = hits + 1
    print(k, "people: shared birthday in", hits, "of 10000
    ↪ rooms")
```

Ten thousand rooms of 23, and right around 5,073 of them have the coincidence — exact formula and honest die agreeing to three figures. (The seen set doing collision-detection here is a hash table detecting hash collisions — the lab is quietly eating its own cooking.)

Break it

Predict first. Run second. Explain any gap third.

1. *The seed.* Add `random.seed(42)` at the top of a cell, run \$4's histogram experiment with only 20 runs, and print the 20 counts. Now rerun the identical cell. Predict before the rerun: same 20 “random” numbers, or different? The result tells you what the word *pseudo* is doing in “pseudo-random” — state it plainly. Then the consequence, which Problem 3.2 weaponizes: if randomness is reproducible by anyone who knows the seed, what has happened, exactly, to the chapter's claim that “the coins do not read lists”? When is reproducibility a virtue (think: this book's figures, science itself) and when is it a vulnerability?

2. *The biased die.* Build a crooked die: `random.choice([1, 2, 3, 4, 5, 6, 6])` — the six appears twice in the list. Predict its expected roll exactly (it's a weighted average; compute before running), then detect the bias by simulation: how many rolls did *you* need before the empirical average stood clearly apart from 3.5? A casual glance at twenty rolls would never catch it; state in one sentence what the long run does to small dishonesties. (Casinos, auditors, and particle physicists all live by your sentence.)

3. *Rigged dice, both directions.* In `quicksort_random`, replace the honest coin with two rigged ones and measure each against \$4's baseline (400 runs suffice). First the saboteur: `pivot = the minimum of five` random picks — a die heavily biased toward terrible pivots. Predict the hill's move, then measure (the chapter's wisdom says lopsided splits cost; expect the mean up by roughly a quarter

and the worst runs worse). Then the surprise: pivot = the **median of three** random picks (draw three with `random.choice`, take the middle one — two ifs will do it). Predict first; then watch the hill land *left* of the honest die's, around 13% cheaper. Real libraries ship exactly this rig ("median-of-three"); you have just rediscovered why. Now the question both rigs were for: if unfair coins can beat fair ones, what were the fair coins ever *for*? Answer precisely — what property do the honest die and the median-of-three rig share that `pivot = items[0]` lacks, and which §3 catastrophe does that property, alone, rule out? (Hint: it is a sentence about the adversary, not about speed. A rigged die the adversary cannot read is still a die.) One curiosity to keep, free of charge: a *mild* low-bias rig — minimum of just two picks — measurably doesn't hurt the mean at all; if you have the patience, confirm it, and notice you have reached the edge of what this chapter's tools explain. The full account needs integrals this book doesn't assume. Edges are good places to know the location of.

4. *Certainty returns*. Run the birthday simulation at $k = 366$. Predict the count out of 10,000 rooms before running — and predict what kind of fact your prediction is. The chapter's Problem 3.5 distinguishes two species of theorem; after this cell you will have simulated one species and *proved* the other, and you should be able to say which is which in one sentence each.

Chapter 8: Learning as an Algorithm

★ ★ ★

In which the two questions soften without surrendering, and an old wound reopens on a landscape

A different kind of problem

Every problem in this book so far came with a definition. “Sorted” has a definition; “shortest path” has a definition; an algorithm could be *proved* correct because correct *meant* something exact. Now consider the problems your devices answer all day: is this email spam? Is this photo of a cat? Given the words so far, what word comes next? Try to write the definition of “cat photo” precise enough to check a procedure against. There isn’t one. Nobody has one. And yet the problems get solved, billions of times a day, to a standard everyone lives with.

The escape is a different deal with the problem. Instead of a definition, you are handed **examples** — ten thousand photos labeled cat/not-cat, a million emails labeled spam/not-spam — and the task becomes: *produce a procedure whose answers agree with the examples, and keep agreeing on examples you haven’t seen*. An algorithm fitted to examples rather than derived from a definition is called **learning**, and this chapter’s claim is that it is not a new subject. It is this book’s

same two questions, in their final form:

- **Does it work?** softens to: *is it approximately correct, on average, over the examples the world will actually send?* — and Chapter 7 spent itself making “on average” a phrase you can compute, concentrate, and trust. That promotion is cashed here, immediately and forever.
- **What does it cost?** doesn’t soften at all. We still count.

The simplest learner

Here is a complete learning algorithm, four lines long, with no training step whatsoever. The examples are points — say, each point is (hours of sleep, hours of practice) for a student, labeled by whether they passed the audition. To predict a new student’s fate: **find the example nearest to them, and copy its label.**

That’s the whole algorithm — **nearest neighbor** — and “learning” here is just *storage*: the learner memorizes the examples and does nothing else until asked. All the cost lands at answer time, and the cost is an old friend wearing new clothes: finding the nearest point in a pile of n examples is *search* — Chapter 1’s linear scan, distance instead of equality, n steps per question. (And Chapter 1’s sequel question returns with it: could structure make the search cheap, the way sortedness did? For points in space the answer is yes, with lovely data structures this book only waves at — but the *instinct* to ask is the chapter’s point.)

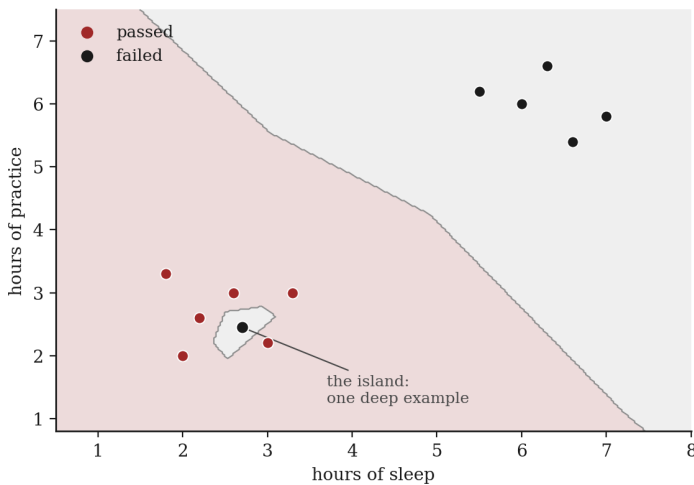


Figure 8.1 — The nearest-neighbor map: twelve memorized examples and the territory each one governs. Note the island around the lone deep point.

Figure 8.1 — The nearest-neighbor map: twelve memorized examples and the territory each one governs. Note the island around the lone deep point — memory, faithfully serving one example, possibly an error.

Simple as it is, nearest neighbor already contains the field's central tension, visible in its map. Each memorized example governs the territory nearer to it than to anything else — and one example deep in foreign territory (a mislabeled point, an odd duck) carves out a little island of its influence. The learner serves that island as faithfully as everything else, because the learner *cannot tell data from noise*: it memorizes both. Whether faithful memory is wisdom or gullibility is the question the chapter ends on. File the island; we return to it.

A number called wrong

Classification copies labels; the other half of learning **predicts numbers** — tomorrow's temperature, a house's price, a test score from hours studied. Here the examples are pairs (x, y) , and the learner is a formula with adjustable knobs. Take the simplest: predict y as $w \cdot x$, one knob, the slope w . Which w is *best*?

The move that unlocks everything: define wrongness as a **number**. For a candidate w , measure each example's miss — prediction minus truth — square it, and average over all examples:

$$L(w) = \text{average of } (w \cdot x_i - y_i)^2$$

This is a **loss function**, and pause on what just happened, because it is the hinge of the whole subject: the fuzzy demand “fit the data well” became a single number $L(w)$ attached to every candidate knob-setting, and *learning became minimization* — find the w making one number small. Everything after this is optimization. (Why *squared* misses, rather than absolute ones or fourth powers? Three honest reasons: squaring punishes large misses more than proportionally, which is usually what you want; it treats high and low misses symmetrically; and — decisive for the next section — the squared loss is *smooth*, a polished bowl Book 2's calculus can get its hands on. There is also a fourth reason, which is that the choice has a track record older than electricity.)

A human moment

The loss function found a planet. On the first night of 1801, the astronomer Giuseppe Piazzi spotted a faint new object — Ceres, the first asteroid — and tracked it for six weeks before it slid behind the sun's glare, carrying only a thin arc of noisy observations with it. The astronomers of Europe could not agree where it would re-emerge; the discovery was about to be lost. A 24-year-old named Carl Friedrich Gauss — yes, the school-boy of Lab 0, third appearance, last one — attacked the problem with a then-private method: choose the orbit whose predictions make the **sum of squared misses** against the observations as small as possible, and trust the noisy data *collectively* though no single observation deserved trust alone. His predicted position was nowhere near anyone else's. In December 1801, almost a year after its loss, Ceres was found again — essentially where Gauss's minimization said to look — and an obscure young man

became the most famous mathematician in Europe overnight. (Honesty about the credit: Legendre *published* the method of least squares first, in 1805, and the priority quarrel between them was real and bitter. The idea, as usual, was ready before either man.) Every loss function minimized in every training run today is the direct descendant of that orbit: wrongness made into one number, and the number made small.

Going downhill

So minimize $L(w)$. And immediately, every tool in this book's first act fails at once, for one reason: **w is continuous**. Try-everything is dead — there is no “everything”; between any two candidate slopes live infinitely many more. The table method is dead with it — Chapter 4's notebook needs *questions you can list*, and a continuum cannot be listed. Six chapters of machinery, and the doorknob comes off in your hand.

What survives is *local* information. Stand at some w and ask Book 2's question: what is the slope of L *here* — the derivative $L'(w)$? It points uphill; its size says how steep. You cannot see the whole landscape, but you can feel the ground under your feet — so take a small step downhill and ask again:

$$w \leftarrow w - \eta \cdot L'(w)$$

— move against the slope, by a step proportional to steepness, scaled by a small number η (the **learning rate**, the size of your stride). Repeat. That is **gradient descent**, the algorithm under essentially all modern machine learning, and it fits in one line because it is one idea — and the idea has a name from Chapter 2. *At every step, take the option that looks best from where you stand, commit, and never look back*. Gradient descent is **greedy**, in continuous space, and you already know exactly what that means: sometimes provably perfect, sometimes confidently wrong, and the difference will be a property of the landscape.

Run it by hand first — three steps, the bowl $f(x) = x^2$, slope $f'(x) = 2x$, stride $\eta = 0.25$, starting at $x_0 = 2$:

- $x_1 = 2 - 0.25 \cdot (4) = 1$
- $x_2 = 1 - 0.25 \cdot (2) = 0.5$
- $x_3 = 0.5 - 0.25 \cdot (1) = 0.25$

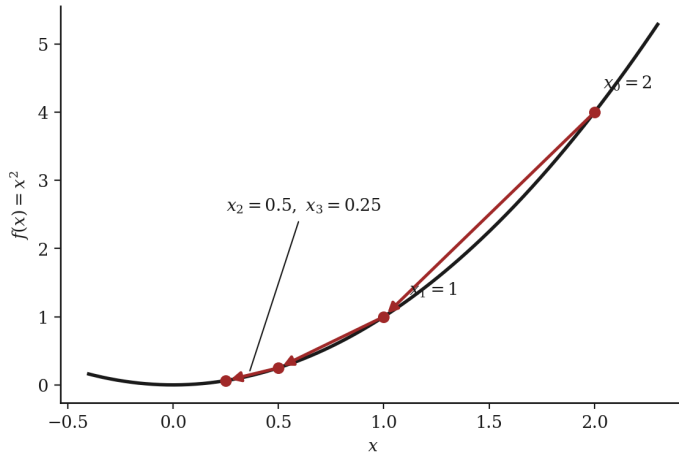


Figure 8.2 — Three steps down the bowl: each stride is proportional to the slope, so the steps shrink as the floor flattens — the walk slows itself near the answer.

Figure 8.2 — Three steps down the bowl: each stride is proportional to the slope, so the steps shrink as the floor flattens — the walk slows itself near the answer.

Halved, every step — and notice the built-in grace: nobody told the algorithm to slow down near the bottom, but the slope is smaller there, so the strides shrink on their own. The walk brakes automatically as it arrives. On this bowl that's not a happy accident but an exact law: each step multiplies the position by $(1 - 2\eta)$, and the Diamond has you turn that observation into the book's first *convergence proof* — including the full strange taxonomy of strides: $\eta = 0.5$ lands exactly at the bottom in **one step**; η slightly under 1 converges while *overshooting the bottom every single step*, ringing like a struck bell; $\eta = 1$ bounces between 2 and -2 forever; η beyond 1 climbs *out* of the bowl, each step worse than the last. The learning rate is not a detail. It is the difference between arriving, ringing, and exploding, and in Lab 8 you will produce all three on purpose.

The two valleys

On a bowl, greedy-in-continuous-space is provably perfect: one valley, every downhill road leads there. Now tilt the world. The lab's landscape $f(x) = x^4 - 3x^2 + x$ has **two valleys** — a deep one near $x = -1.30$ (floor at -3.51) and a shallow one near $x = 1.13$ (floor at -1.07) — separated by a hilltop near $x = 0.17$.

Start the descent at $x_0 = -0.5$: it walks left, settles at -1.30 , the true bottom. Start at $x_0 = +0.5$ — a coin's width across the hilltop — and it walks right, settles into the shallow valley, and *stays there at -1.07 , forever*, reporting success. Every local test the algorithm can perform says it is finished: the ground is flat, every direction is uphill. It cannot see the better valley two units to the left, because it has never had anything but the ground under its feet.

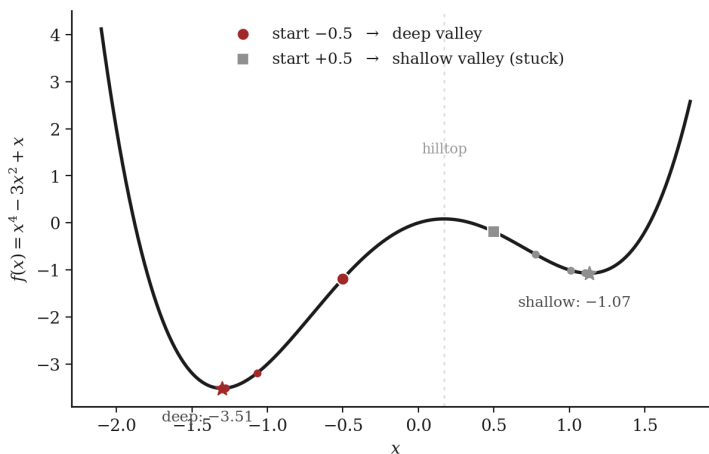


Figure 8.3 — Two valleys, two destinies: from -0.5 the walk finds the deep floor; from $+0.5$ it commits into the shallow valley and pronounces itself done.

Figure 8.3 — Two valleys, two destinies: from -0.5 the walk finds the deep floor; from $+0.5$ it commits into the shallow valley and pronounces itself done. Start decides everything; the walker never knows.

You have seen this exact failure before, and the recognition is the point of the whole book's architecture. The shallow valley is **the grab-the-10 valley**. The deep one is **the 7 + 7 valley**. Gradient descent at $+0.5$ is Sevenia's cashier: the locally best move, committed,

mortgaging a better future it never priced. Chapter 2 diagnosed the disease; Chapter 4 cured it — *price every branch, then choose* — and here is the bitter pill: **the cure is unavailable**. The table method needs listable questions, and a landscape’s points cannot be listed. The notebook that healed Sevenia cannot be opened on a continuum. So modern practice lives with greedy and manages its valleys the way you’d guess once you say it out loud: descend from *several random starts* and keep the best (Chapter 7’s move — when cleverness is unavailable, honest dice); jitter the steps with noise so shallow valleys can be stumbled out of; and lean on a strange empirical mercy — in the absurdly high-dimensional landscapes of real models, valleys seem to behave differently than our 1-D pictures suggest, in ways that are genuinely, actively, *not understood*. That last clause is not a textbook hedge. It is the honest edge of the field, and the deep water says more.

The whole loop, honestly

You now hold every part of the modern machine, so here is what “training an AI” is, in one paragraph, with nothing hidden. A model is a formula with knobs — not one knob *w* but billions, the same mathematics with the derivative grown into a *gradient* (a slope per knob; the deep water below). The loss is a number measuring wrongness averaged over examples — for a language model, roughly: across mountains of text, how badly did you predict each next word? Training is the loop you ran by hand on the bowl: compute the loss’s gradient, step every knob a little downhill, repeat — not three steps but trillions, on hardware built for exactly this loop and nothing else. That is the entire ritual. The gradient of a billion-knob formula sounds unaffordable; the algorithm that makes it cheap is a beautiful machine built from this book’s own parts — the chain rule riding a graph — and Chapter 9 does not describe it. Chapter 9 *builds it*, and then builds the model, and then

trains it, on your machine, until it speaks.

The real question

One question stands above all of it, and nearest neighbor — remember the island — asked it first. A learner is fitted to examples; agreeing with *those examples* is therefore no evidence at all. Nearest neighbor agrees with its own examples *perfectly, always, by construction* — ask it about a memorized point and it finds that very point at distance zero and copies the label. A parrot scores 100% on the questions it was trained on. The only honest exam is one the student hasn't seen.

So the field's entire experimental discipline is one move: **split the examples**. Fit on one part (the *training set*); measure on the part the learner never saw (the *test set*); report only the second number. The gap between the two numbers is the gap between *memorizing* and *learning* — and in Lab 8 you will measure that gap on nearest neighbor itself, watch a single poisoned example widen it, and hold in your hands the distinction this entire subject turns on. When the test score is good, something genuinely transferred from the examples to the world. Why transfer happens as well as it does for enormous models — models with vastly more knobs than examples, which by every classical right *should* be parrots — is, as of this book's writing, an open research problem. People are working on it. The author of this book is one of them. The honest sentence is: we can build it, we can train it, we can measure that it generalizes, and we cannot yet fully say why. You are not reading a finished subject. You are reading one with the paint wet, and the last brushes are within reach of people who, like you, started by counting steps.

Deeper water: many knobs, and the shape of high places

With knobs $w_1, \dots, w_d$, the loss is a surface over d -

dimensional space, and “slope” becomes the **gradient**: the vector of partial derivatives $(\partial L / \partial w_1, \dots, \partial L / \partial w_d)$ — each entry answering “if I turn only this knob, how does the loss tilt?” The gradient points in the steepest uphill direction (a lovely fact, provable with Book 2’s tools plus one inequality), so the update $w \leftarrow w - \eta \cdot \nabla L$ steps each knob against its own slope, all knobs at once: one line, any dimension. Everything in this chapter survives the generalization untouched — bowls (now called *convex* functions) still come with guarantees, valleys still trap.

But dimension does something strange to the *zoology* of flat places. In one dimension a flat point is a floor or a hilltop. In a million dimensions it is almost always neither: a **saddle** — downhill some directions, uphill others — and a walker pausing there can usually find an exit, where a true 1-D hilltop offers none. Some of the empirical mercy of deep learning (greedy works far better than the two-valleys picture predicts) is suspected to live here, in the geometry of very high places; the suspicion is not a theorem, the geometry of real loss landscapes is an active research frontier, and the tools being thrown at it include some you would recognize — eigenvalues, phase transitions, the spectra of large matrices. The mountains are real; the cartography has barely begun.

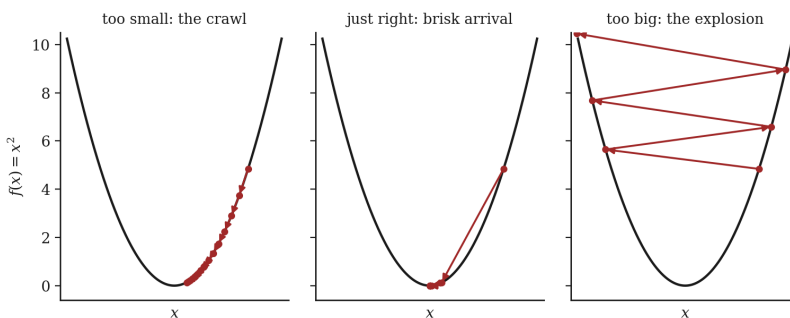


Figure 8.4 — Three temperaments of the stride: the crawl, the brisk arrival, and the explosion. One dial, every fate.

Figure 8.4 — Three temperaments of the stride: the crawl, the brisk arrival, and the

explosion. One dial, every fate.

Deeper water: the stride, exactly

On the bowl, the Diamond’s law $|x_{\{k+1\}}| = |1 - 2\eta| \cdot |x_k|$ classifies every stride with one number, $|1 - 2\eta|$, and it is worth tabulating the taxonomy because every behavior in it occurs daily in real training: η tiny $\rightarrow$ factor near 1, convergence at a crawl (real name: “undertrained”); $\eta = 0.5 \rightarrow$ factor 0, the floor in one step (possible only because this bowl is perfectly known — never in real life); $\eta \in (0.5, 1) \rightarrow$ negative factor, *oscillating* convergence, the loss curve ringing downward (you will see this ringing in Chapter 9’s training runs and now you know its name); $\eta \geq 1 \rightarrow$ factor at or beyond 1, ringing forever or exploding (real name, seen in every practitioner’s logs: “loss went to NaN”). One honest limitation, which the Diamond’s part (e) makes you locate precisely: the *exactness* of this law leans on the bowl’s slope being exactly linear. General smooth bowls keep the qualitative story — small strides converge, oversized strides explode — but the clean factor and the magic $\eta = 0.5$ are this bowl’s private property. The idea generalizes; the arithmetic does not. Knowing which part travels is most of mathematical maturity.

Where this leaves us

The two questions held. “Correct” softened to *approximately correct on average over unseen examples* — measured by the train/test discipline, trusted by Chapter 7’s concentration. “Cost” never softened: nearest neighbor pays at answer time, descent pays per step times steps, and we counted. The new machinery is small and yours: examples instead of definitions; loss — wrongness as one number; descent — greedy on a landscape, with greedy’s old guarantee on

bowls and greedy's old disease among valleys, now incurable by the notebook and managed by dice. And above it all, the field's one honest exam: never grade a learner on questions it has memorized.

What remains is to stop gesturing at “a formula with billions of knobs” and build one — the knobs, the chain rule that prices them all at once, the loop, the model, the training run, the first words it speaks. Everything needed is already in this book, scattered through eight chapters like parts on a workbench. Chapter 9 is assembly.

Problem Set 8

★ ★ ★

Lab 8 (Downhill) is the Computational Lab tier; problems 2.4, 3.3, and the Diamond's part (d) use its runs as evidence.

Fluency

1.1 Run gradient descent by hand on $f(x) = x^2$, $f'(x) = 2x$, with $\eta = 0.1$ from $x_0 = 5$, for four steps, keeping three decimals. What constant factor multiplies x each step? Check it against $1 - 2\eta$.

1.2 For the loss $L(w) = \text{average of } (w \cdot x_i - y_i)^2$ on the two examples (1, 2) and (2, 5): compute $L(0)$, $L(2)$, and $L(3)$ by hand. Which of the three candidate slopes is best, and what is each prediction actually missing by?

1.3 Differentiate the one-example loss $\ell(w) = (w \cdot x - y)^2$ with respect to w (Book 2's chain rule, one application). Then state the descent update for w in terms of the miss $(w \cdot x - y)$ — and read it aloud as a sentence about blame: which examples move the knob hardest, and in which direction?

1.4 Nearest neighbor by hand: examples at (1, 1) PASS, (2, 4) PASS, (4, 1) FAIL, (5, 4) FAIL. Classify the points (3, 1), (3, 4), and (3, 2.5) — distances computed explicitly (squared distances suffice; why may you skip the square root when only *comparing*?). One of the three is a genuine coin flip; which?

1.5 On the two-valley landscape $f(x) = x^4 - 3x^2 + x$, evaluate f and f' at $x = -0.5$ and $x = +0.5$, and state — from the signs alone,

before any descent — which direction each walker’s first step takes and which valley therefore claims it.

1.6 The stride taxonomy: for $\eta = 0.05, 0.4, 0.5, 0.75, 1.0, 1.2$ on the bowl, compute the factor $1 - 2\eta$ and classify each stride’s fate in one word (crawl / converge / one-step / ring-inward / ring-forever / explode).

Practice

2.1 Hours studied x , test score y , three students: $(2, 65), (4, 72), (6, 91)$. Fit $y \approx w \cdot x$ by hand-cranked descent: start $w = 0$, $\eta = 0.01$, run four steps with the gradient from 1.3 (calculator encouraged, three decimals). Then sanity-check your w against the crude estimate “average of y/x .” Is descent heading there?

2.2 A delivery company predicts delivery time as $w \cdot (\text{distance})$. Their loss is squared misses. A rival argues for absolute misses, $|w \cdot x - y|$, because “a 20-minute miss is exactly twice as bad as a 10-minute miss, not four times.” Both positions are defensible — give one realistic scenario favoring each loss, then state which one this chapter’s *machinery* prefers and point to the exact sentence of the descent method that explains why (what does the update need to exist at every point?).

2.3 Nearest neighbor for music: each song is a point (loudness, tempo), labeled “morning” or “night” by a user’s history. Design six example songs (your numbers) such that one plausible new song gets a label you’d call *wrong* despite the algorithm working perfectly. What feature is missing from your points, and what does this say about where a learner’s competence actually lives — the algorithm, or the representation?

2.4 *The several-starts gamble*. On the two-valley landscape, suppose starting points are drawn uniformly from $[-2, 2]$, and (from your lab runs or 1.5’s reasoning) starts below the hilltop at ≈ 0.17 reach the deep valley. What is the probability one random start finds the

deep floor? How many independent starts before the probability of *missing* it drops below 1%? (Chapter 7’s machinery: the misses must all happen together.) State the general trade this strategy makes, in the currency-language of this book.

2.5 The parrot exam: a learner is graded 98% on its training examples and 71% on held-out examples. A second learner scores 84% and 82%. Which would you deploy, and what *single number* — define it — summarizes why? Then construct the cautionary scenario: describe circumstances under which even the honest test score misleads (think: where did the test examples come from, and where will tomorrow’s questions come from?).

Concept & Counterexample

3.1 Claim: “gradient descent finds the minimum of any function if you run it long enough.” Refute three ways with this chapter’s own objects: a landscape where it provably converges to the wrong place; a starting point on the two-valley landscape where it never moves at all (find it exactly — what equation does that point satisfy, and is it a minimum?); and a stride for which it diverges on even the friendliest bowl.

3.2 Claim: “smaller learning rates are always safer, so when in doubt, shrink η .” Half true. State the half that’s true (what catastrophe does shrinking rule out, by the Diamond’s law?) and refute the “always safer” half with a budget argument: with 100 steps allowed on the bowl from $x_0 = 2$, compare where $\eta = 0.001$ and $\eta = 0.25$ each end up. Safety from explosion is not safety from failure; name the failure mode the timid stride commits.

3.3 Claim: “nearest neighbor doesn’t really learn — it just memorizes.” Defend *both* sides with one lab measurement each: the sense in which the claim is exactly right (which score is always perfect, and why is that score therefore worthless?), and the sense in which it’s wrong (which score can nonetheless be genuinely good, and what

does that prove transferred?). Conclude with the sentence this chapter would write on the matter.

3.4 Claim: “with squared loss, one wildly wrong example among a thousand barely matters — it’s just one term in an average.” Compute the refutation: examples (1, 1), (2, 2), (3, 3) plus the typo (1, 100). Find the best slope with and without the typo (use 1.3’s gradient set to zero, or argue directly), and report how far one liar dragged the line. Then connect: which of the *three honest reasons for squaring* from the chapter is also this vulnerability’s cause, and what does the trade-off teach about choosing losses?

3.5 Claim: “the train/test split is bureaucracy — if the model fits all available data well, it has learned the pattern.” Build the counterexample in miniature: invent a learner (nearest neighbor will do) and a dataset where the training score is perfect, the test score is coin-flip, and *you* can say exactly what the learner actually “learned.” (Hint: label your points by a fair coin and see what perfect memorization of noise looks like under honest examination.)

Diamond

4.1 The first convergence proof. Gradient descent on $f(x) = x^2$, update $x_{k+1} = x_k - \eta \cdot 2x_k$. (a) Show $x_{k+1} = (1 - 2\eta) \cdot x_k$ exactly, and unroll: $x_k = (1 - 2\eta)^k \cdot x_0$. (b) Prove: if $0 < \eta < 1$ then $|1 - 2\eta| < 1$ and $x_k \rightarrow 0$ — convergence, geometric, from *every* starting point. (Where in this argument does Chapter 7’s “geometric series ruled by its ratio” — Chapter 3’s, originally — make its return?) (c) Classify the boundary and beyond: $\eta = 0.5$, $\eta = 1$, $\eta > 1$ — state each fate and prove it from the unrolled formula. (d) Reconcile with your lab: match each η you ran to its predicted fate, and identify the run whose loss curve “rang.” What sign pattern in x_k is the ringing? (e) The honest perimeter: every step above leaned on $f'(x) = 2x$ being *exactly linear*. Point to the precise line of (a) that fails for a general bowl like $f(x) = x^2 + x^4$ — then state, without proof, which

conclusion you'd still bet on and which (the magic of $\eta = 0.5$) you'd abandon. Knowing what survives generalization is the skill this part grades.

Computational Lab

5.1 Lab 8 in full, predictions before runs. Export three artifacts for the problem set: your η -fate table (Diamond d), your two-valley endpoints (2.4), and your train/test gap before and after the poisoned label (3.3).

Teach It

6.1 Blindfold someone (or have them close their eyes) on a gentle slope — a yard, a wheelchair ramp, a hallway with a tilted board — and ask them to find the lowest point using only the feeling under their feet, narrating their rule aloud. Then tell them that's the algorithm training every AI they've heard of, and let them object. Their best objection is the chapter's content; write down which one they found (valleys? stride size? "what if the ground is flat?") and how far they got unprompted.

6.2 Explain the parrot exam to a younger student using actual flashcards: train them on ten cards, test on the same ten, then on five new ones, and discuss the two scores. Then the reveal: every AI benchmark they've seen a headline about is the five new cards — and the entire trustworthiness of the field hangs on those cards being genuinely new. Report their reaction, and your own honest answer to the question they will ask: "how does anyone *know* the test cards weren't in the training pile?" (Welcome to a real and current controversy.)

Lab 8: Downhill

★ ★ ★

Accompanies Chapter 8. Gradient descent gets built in ten lines and then systematically mistreated: the stride cranked through every fate the Diamond classifies, the two valleys claiming walkers by their birthplace, a line fitted to real points, and nearest neighbor sitting the parrot exam — scoring perfectly on what it memorized and honestly on what it never saw. One quiet arrival: numbers with decimal points, and their one sharp edge.

1. Floats, and their edge

Until now this book's numbers were whole. Landscapes need **floats** — numbers with decimal points — and they mostly behave like numbers. Mostly. Run the classic:

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
```

Not a bug — a confession. Floats are *approximations*: the machine stores numbers in binary, and 0.1 in binary is an infinite repeating fraction the machine must cut short, the way $1/3 = 0.3333\dots$ must be cut short in decimal. The error is tiny (look at it: seventeen places out) and almost never matters — except in one habit you must never form: **testing floats with ==**. Ask instead whether two floats are *close*: `abs(a - b) < 0.000001`. Adopt the habit now; today's algorithms converge *toward* answers without ever exactly arriving, and `==` would wait forever for an arrival that isn't coming.

2. The descent, in ten lines

The bowl $f(x) = x^2$, slope $2x$, the chapter's hand-run made executable:

```
def descend(slope, x, eta, steps):
    path = [x]
    for _ in range(steps):
        x = x - eta * slope(x)
        path.append(x)
    return path

def bowl_slope(x):
    return 2 * x

print(descend(bowl_slope, 2.0, 0.25, 6))
```

Check the first three steps against the chapter: 2, 1, 0.5, 0.25 — then watch the next three continue the halving. The function takes the *slope*, not the landscape: the walker never sees the terrain, only the tilt underfoot. That blindness is not an implementation shortcut. It is the algorithm's entire epistemology, and everything that goes wrong today goes wrong because of it.

3. Every fate of the stride

The Diamond claims one number, $1 - 2\eta$, decides everything on this bowl. Produce the full taxonomy:

```
for eta in [0.05, 0.25, 0.5, 0.75, 1.0, 1.05]:
    path = descend(bowl_slope, 2.0, eta, 20)
    print("eta =", eta, " factor =", 1 - 2 * eta,
          " x after 20 steps =", round(path[20], 6))
```

Predict each row before running, from the factor alone. Then the

pictures — plot $|x|$ against step for a chosen few (a log scale makes geometric behavior into straight lines; `plt.semilogy` is `plt.plot` with a logarithmic y-axis):

```
import matplotlib.pyplot as plt

for eta in [0.05, 0.25, 0.75, 1.05]:
    path = descend(bowl_slope, 2.0, eta, 20)
    sizes = [abs(x) + 1e-12 for x in path]      # +tiny: log
    ↪ of zero is forbidden
    plt.semilogy(sizes, label="eta = " + str(eta))
plt.xlabel("step")
plt.ylabel("|x| (log scale)")
plt.legend()
plt.show()
```

Four straight lines, four slopes: crawl, brisk convergence, *ringing* convergence (print that path and look at the signs — the walker overshoots the bottom every step and lands closer anyway), and explosion. Find $\eta = 0.5$'s line by adding it: where did it go, and why does the `+1e-12` in the code suddenly matter? One dial, every fate, exactly as the factor foretold.

4. The two valleys

The chapter's tilted world, $f(x) = x^4 - 3x^2 + x$, slope $4x^3 - 6x + 1$:

```
def hills_slope(x):
    return 4 * x**3 - 6 * x + 1

def hills(x):
    return x**4 - 3 * x**2 + x

for start in [-2.0, -0.5, 0.5, 2.0]:
```



```
path = descend(hills_slope, start, 0.02, 500)
end = path[500]
print("start", start, " -> settles at", round(end, 4),
      " floor value", round(hills(end), 4))
```

Predict the four destinies first (Fluency 1.5 did the two middle ones by sign alone). Then read the results: two walkers in the deep valley at -1.3008 (floor -3.514 , the 7+7 valley), two in the shallow one at 1.1309 (floor -1.070 , the grab-the-10 valley) — and the border between fates runs along the hilltop near 0.17 , not at zero. Each walker, asked, would report complete success: slope zero, every direction uphill, work done. Print `hills_slope(path[500])` for a shallow-valley walker and see its honest, useless certificate.

Now the chapter's managed retreat, dice against valleys — many random birthplaces, keep the best:

```
import random
random.seed(8)

best_end = None
for _ in range(10):
    start = random.uniform(-2, 2)
    end = descend(hills_slope, start, 0.02, 500)[500]
    if best_end is None or hills(end) < hills(best_end):
        best_end = end
print("best of 10 random starts:", round(best_end, 4), "
↳ floor:", round(hills(best_end), 4))
```

Ten dice throws against a landscape the walker cannot see. Compute (Problem 2.4) what each throw's chance of the deep valley is, and how rarely all ten can miss.

5. Learning a line

Now the real thing: descent applied to a *loss*. Hours studied and test scores, six students:

```

hours = [1, 2, 3, 4, 5, 6]
scores = [16, 31, 47, 60, 78, 89]

def loss(w):
    total = 0
    for x, y in zip(hours, scores):      # zip: walk two
        ↪ lists in step
        total = total + (w * x - y) ** 2
    return total / len(hours)

def loss_slope(w):                      # the gradient from
    ↪ Problem 1.3
    total = 0
    for x, y in zip(hours, scores):
        total = total + 2 * (w * x - y) * x
    return total / len(hours)

path = descend(loss_slope, 0.0, 0.01, 60)
print("learned slope:", round(path[60], 3))

plt.plot([loss(w) for w in path])
plt.xlabel("step")
plt.ylabel("loss")
plt.show()

```

The loss curve — the signature picture of this entire field; every training run you will ever see reported is this plot, grown up — falls steeply, then levels as the strides self-brake near the floor. The learned slope says each hour of study buys about 15 points; check it crudely against the data by eye. Then two experiments of your own:

re-run with $\eta = 0.05$ and watch the curve ring (you know its name now), and add one student who studied 2 hours and scored 95 — a prodigy or a typo — and report what one outlier does to the learned line (Problem 3.4 predicted this; here is the measurement).

6. The parrot exam

Nearest neighbor, four lines as promised, on points with two features:

```
def nearest_label(examples, p):
    best = None
    for q, label in examples:
        d = (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2
        if best is None or d < best[0]:
            best = [d, label]
    return best[1]
```

(Chapter 1's linear search, distance for equality — and squared distance suffices, since comparing squares compares distances.) Build a little world. Each song is [loudness, tempo]; in this user's library, morning and night songs come at *every* loudness — the signal lives entirely in the tempo, slow mornings and fast nights, with honest scatter at the border:

```
def make_song(kind):
    if kind == "morning":
        return [[random.uniform(1, 9), random.uniform(60,
            ↪ 110)], "morning"]
    else:
        return [[random.uniform(1, 9), random.uniform(105,
            ↪ 160)], "night"]

random.seed(88)
```

```
training = [make_song("morning") for _ in range(20)] +  
↳ [make_song("night") for _ in range(20)]  
testing = [make_song("morning") for _ in range(50)] +  
↳ [make_song("night") for _ in range(50)]
```

Now the two exams — the one it memorized, and the one it never saw:

```
def score(examples, exam):  
    right = 0  
    for p, truth in exam:  
        if nearest_label(examples, p) == truth:  
            right = right + 1  
    return right / len(exam)  
  
print("training score:", score(training, training))  
print("test score:    ", score(training, testing))
```

Predict both before running. The first is 1.0 — *necessarily*, eternally, by construction (each exam point finds itself at distance zero); write one sentence on why that score is therefore worth nothing. The second is the honest number — high here, because the world really is learnable and the features really do carry the signal — and the *gap* between the two numbers is the chapter’s last section in miniature. You have administered the parrot exam, and this learner passed it. Keep the grader; Chapter 9’s learner takes the same exam.

Break it

Predict first. Run second. Explain any gap third.

1. *The walker on the hilltop.* The two-valley landscape’s hill crests where the slope is exactly zero — solve nothing, just start a walker at 0.16993844331159128 (the lab hands you the cursed coordinate) and descend. Predict its journey. Then nudge it: start at 0.1699384 (seven digits). Predict again, run, and explain the difference in fates

using §1’s confession about floats. A zero-slope point is a trap of measure zero — but what does “measure zero” feel like in a world of seventeen decimal places, and which of the two runs is the one real training jobs experience?

2. *Convergence vs. arrival.* On the bowl with $\eta = 0.25$, descend from 2.0 and test `path[k] == 0.0` for $k = 20, 50, 200$. Predict the three answers. Then test `abs(path[k]) < 1e-6` instead. State the rule you’ll follow for the rest of your life about asking whether an iterative method has “finished” — and connect it to why §1 banned `==` before you’d written a single algorithm that needed the ban.

3. *The poisoned example.* In §6, flip exactly one training label (pick a morning song, call it “night”). Predict both scores’ movements before running: which is *guaranteed* not to drop, and why? Then measure the test score’s change, and — using `nearest_label` as a probe — map the poisoned point’s island: classify a small grid of points around it and find the territory one lie governs. The chapter’s Figure 8.1 island, grown in your own lab; finish with one sentence on what a learner that cannot doubt its examples does with a lie.

4. *Features are destiny.* Rebuild §6’s world with the tempo feature corrupted: multiply every tempo by 0.001 (a units botch — somebody filed kilo-beats-per-minute). The labels are untouched, the algorithm is untouched, and *the information is all still present* — every tempo is exactly recoverable. Predict the test score anyway. Then run it and explain the collapse to a coin flip: in `nearest_label`’s distance sum, who is doing all the talking now, who has been whispered into silence, and why does the only feature that knows anything no longer get a vote? Try 0.01 too and watch the score land *between* coin-flip and honest — silencing comes in degrees. Then repair it *without touching the data*: rescale inside the distance until both features speak at comparable volume. The lesson — a learner’s competence lives as much in the representation as in the algorithm — is Problem 2.3’s, now measured; it is also, one chapter early, the reason Chapter 9 will spend real care on how text becomes numbers before any learning begins.

Chapter 9: Building the Learner

★ ★ ★

In which eight chapters return as parts on a workbench, and the machine is assembled, and it speaks

The one missing piece

Chapter 8 left you holding the whole idea of learning and one unpaid bill. A model is a formula with knobs; training steps every knob downhill on the loss; the step direction is the gradient, a slope per knob. On the bowl, one knob, you computed that slope by hand — $f'(x) = 2x$. But a real model has not one knob but thousands, even billions, wired through layers of multiplication and squashing, and the loss is a single number sitting at the far end of that wiring. The bill: *how do you get the slope with respect to every knob at once, when the knobs are buried a dozen operations deep?*

Computing each slope separately is hopeless — billions of knobs, each needing its own derivative through the whole tangle. There has to be a way to get them all in one sweep. There is, and it is the most important algorithm in this book that you have not yet met: **backpropagation**. By the end of this chapter you will not have read about it. You will have *built* it — sixty lines, from nothing — and then used it to assemble a working learner that generates names of

mathematicians who never existed but plausibly could have. The parts are all already on your workbench, made in earlier chapters. This chapter is assembly.

And the first part requires the one Python tool this book has deliberately withheld for nine chapters.

A number that remembers how it was made

Here is the problem that forces the new tool. To push blame backward through the wiring, each number in the computation must remember three things: its *value*, the *other numbers it was built from*, and *how to pass blame to them*. A plain Python number — 3.0 — remembers only its value. It has forgotten its parents and its history. We need a number that doesn't forget.

A thing that bundles data together with behavior — a value, its parents, and a rule — is an **object**, and the tool for minting objects is `class`. This is why classes waited: not because they are advanced, but because nothing earlier *needed* them, and a tool introduced before its need is a tool half-understood. Backpropagation needs them. Here is the whole tool, learned on the smallest possible example:

```
class Value:
    def __init__(self, data, parents=()):
        self.data = data          # the number itself
        self.grad = 0.0          # blame, filled in later by
        ↪ backprop
        self._parents = set(parents)
        self._backward = lambda: None # how to pass blame to
        ↪ parents

    def __add__(self, other):      # teaches Python what a + b
        ↪ means for Values
        out = Value(self.data + other.data, (self, other))
        def _backward():
```

```

        self.grad += out.grad      # addition passes blame
        ↪ through unchanged
        other.grad += out.grad
    out._backward = _backward
    return out

```

Read `class` as a *factory*. `Value(3.0)` builds one number-that-remembers; `self` is the object being built, and `self.data`, `self.grad`, `self._parents` are the three things it carries. The strange-looking `__add__` is Python’s hook for the `+` symbol: define it, and `a + b` for two `Values` produces a *new* `Value` that remembers its two parents — so the act of computing silently records its own history. (You met this idea of a process recording its history once before: Chapter 3’s call stack, the machine remembering paused work. Here the computation remembers its *construction*.)

The line that matters most is inside `_backward`: when addition’s output receives blame (`out.grad`), it passes that blame to *both* parents unchanged. Why unchanged? Because if `a + b` nudges up by a hair, both `a` and `b` contributed exactly that hair — Book 2’s derivative of a sum. **Every operation knows its own blame-rule**, and the rule is just its derivative. Multiplication’s rule:

```

def __mul__(self, other):
    out = Value(self.data * other.data, (self, other))
    def _backward():
        self.grad += other.data * out.grad    # blame
        ↪ scaled by the OTHER factor
        other.grad += self.data * out.grad
    out._backward = _backward
    return out

```

Multiplication passes blame *scaled by the other factor* — because if `a·b` and you wiggle `a`, the output moves by `b` times the wiggle (Book 2 again: the derivative of `a·b` with respect to `a` is `b`). That is the entire secret of backpropagation, hiding in plain sight: each opera-

tion, when built, tucks away a tiny function that knows how to pass blame to its parents, and that function is nothing but the operation's derivative. Add a few more operations the same way — powers, tanh for squashing, exp and log for probabilities — each three lines, each just its derivative, and the engine is nearly done.

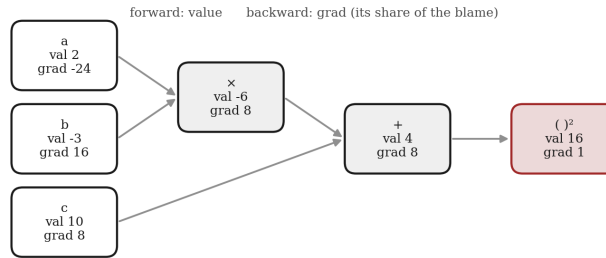


Figure 9.1 — A number that remembers how it was made: the graph of $L = (a \cdot b + c)^2$, each node carrying its forward value and its share of the blame.

Figure 9.1 — A number that remembers how it was made: the graph of $L = (a \cdot b + c)^2$, each node carrying its forward value and, after the backward sweep, its share of the blame. Chapter 5's graph, now the substrate of calculus.

The sweep that prices everything

The operations record history as the computation runs forward. One sweep then prices every knob backward — and the sweep is built entirely from earlier chapters. Watch the curriculum reassemble:

The recorded history is a **graph** — Chapter 5 — with each Value a vertex and an edge to each parent. To pass blame correctly, you must process each node only *after* everything that depends on it, which means visiting the graph in reverse of a careful ordering: a depth-first traversal — Chapter 5's thread, which was Chapter 3's recursion — laying the nodes in dependency order, then walked backward.

```

def backward(self):
    order = []
    visited = set()
    def build(v):
        # depth-first, Chapter
        ↪ 5 exactly
        if v not in visited:
            visited.add(v)
            for parent in v._parents:
                build(parent)
            order.append(v)
    build(self)

    self.grad = 1.0
    # the loss blames
    ↪ itself fully
    for v in reversed(order):
        # walk backward; each
        ↪ node blames its parents
        v._backward()

```

That is backpropagation, complete. Seed the final output with blame 1.0 (the loss is fully responsible for itself), then walk the recorded graph backward; at each node, fire the little blame-rule that operation tucked away, and watch blame flow from the single loss number out to every knob that touched it. One forward computation records the graph; one backward walk prices the entire graph. The billion-knob bill from Chapter 8 is paid in a single sweep — and the sweep is just DFS over a graph of remembered operations, every piece of it built in a chapter you have already done.

There is even a ghost of Chapter 4 here, if you look: when one Value feeds *several* later operations, blame arrives from each, and the $+=$ in every rule *accumulates* it — the multivariable chain rule, which says a knob’s total influence is the sum of its influences along every path it affects. Summing a quantity over all the paths that contribute to it: that is linearity, Chapter 7’s workhorse, doing the bookkeeping one final time.

A human moment

Backpropagation was discovered, forgotten, and rediscovered for two decades before the world noticed. The core idea — reverse-mode differentiation — appears in a Finnish master’s thesis by Seppo Linnainmaa in 1970, with no mention of neural networks; it surfaces in Paul Werbos’s 1974 Harvard PhD thesis as a way to train them, and is largely ignored. Only in 1986, when David Rumelhart, Geoffrey Hinton, and Ronald Williams published it with compelling experiments, did the field finally absorb that this one algorithm made training deep networks *practical* — and even then, a long winter followed before the data and the machines caught up to the idea. Hinton would share the 2018 Turing Award and, in 2024, a Nobel Prize in Physics. The pattern is Chapter 5’s Dijkstra and Chapter 2’s Borůvka once more, sharpened to a point: the algorithm that now consumes a measurable fraction of the world’s electricity sat in plain sight, in published theses, for sixteen years, because the *idea* was ready before the world that needed it. You are building, in an afternoon, the thing three fields walked past for a generation.

From engine to learner

The engine prices knobs; a learner needs knobs arranged into something that computes. Three small classes, each a few lines, each built from the Value engine — and the design idea is the one classes exist to express: **composition**, small pieces that snap together.

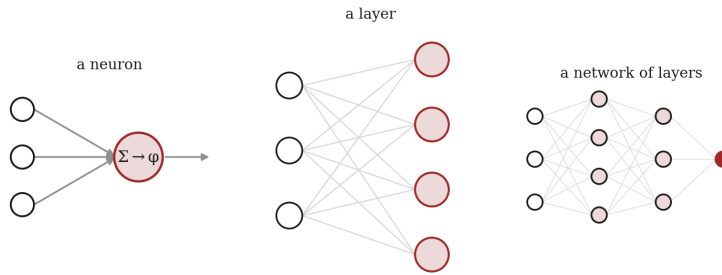


Figure 9.2 — Composition at three zoom levels: a neuron, a layer of neurons, a network of layers — the same shape repeated.

Figure 9.2 — Composition at three zoom levels: a neuron, a layer of neurons, a network of layers — the same shape repeated, which is the whole reason classes exist.

A **neuron** holds a weight per input and a bias (all Values, all knobs), multiplies each input by its weight, sums, adds the bias, squashes with $\tanh$. A **layer** is a row of neurons sharing inputs. A **network** is layers chained, each feeding the next — the same shape at three zoom levels, which is exactly what composition buys. Because every operation inside is a Value operation, the whole network — however deep — builds one giant graph as it runs forward, and the single `backward()` you wrote prices every weight in it. You do not write new calculus for a bigger network. The engine already handles arbitrary graphs; a deeper network is just a bigger graph.

That is the entire principle behind every deep learning framework in the world. The ones running in data centers differ from yours in three ways and *only* three: they store numbers in big arrays instead of one-at-a-time (the next section's instrument), they run on hardware built for exactly this graph, and they have a decade of engineering in the constants — Chapter 6's lesson, that the shape is yours and the constants are an industry, returning for its final bow. The *idea* on your screen is complete.

The instrument, and the model

To train on real data at any speed, one concession — the same kind Chapter 6 made for clocks and Chapter 7 for dice: an instrument earned by need. Pricing Values one at a time is correct but slow; a network’s thousands of multiply-adds want to happen *together*. **NumPy** is Python’s tool for holding numbers in arrays and operating on whole arrays at once — and it is to this chapter what the stopwatch was to Lab 6: adopted for scale, framed honestly, the same mathematics at a thousandfold the speed. The pure-Value engine stays on the workbench as the thing that *explains*; numpy becomes the thing that *runs*. (Lab 9 keeps both: it trains a tiny model with your own Value engine, end to end, to prove the engine is real — then rebuilds the same idea in numpy to train at a size that can actually speak.)

The model itself is the simplest one that *generates*: it reads the last few characters and predicts the next, then feeds its own prediction back as input and predicts again, spinning out a name one letter at a time. The training data is this book’s own cast — a list of mathematicians’ names. (The example lists that have seeded this book since Lab 1 — Ada, Erdős, Gauss — were always meant to arrive here.) The loss is the one Chapter 8 built, averaged over every next-letter prediction across every name: how surprised was the model by the letter that actually came next? Train by the loop you ran on the bowl — gradient down, repeat — and the model’s surprise falls, its letters grow wordlike, and eventually it produces names like *Riemanujan*, *Dantor*, *Wilevin*: nobody’s name, everybody’s plausible mathematician. The machine learned the *shape* of the names without memorizing the names, which is the whole of what learning is.

And here the parrot exam from Chapter 8 returns to keep everyone honest, because it has teeth in this chapter. As the model trains, its *training* loss keeps falling — but its loss on held-out names it

never saw will, past a point, start to *rise*. That gap is overfitting, live, in your own run: the model beginning to memorize its specific training names rather than learn the general music of names. Lab 9 makes you watch both curves and name the moment they part ways. The most expensive lesson in modern AI, the one Chapter 8 said no one fully understands at scale, is reproducible on your laptop in under a minute — and seeing it small is the beginning of taking it seriously large.

Weighted looking-back

One idea separates the toy above from the architecture — the **transformer** — behind the systems you actually use, and it is simple enough to state plainly and honest enough to deserve stating. The toy looks back at a *fixed* few characters. The transformer looks back at *all* of them, but learns *how much to weigh each one* for the prediction at hand. Guessing the letter after “riema—”, the model has learned to weigh the “r” and the “m” heavily and the earlier letters less; guessing after a different stem, it weighs differently. Each position asks every earlier position “how much do you matter to what comes next?”, and the weights — which sum to one, the softmax of Chapter 8 turning raw scores into proportions — are not hand-set but *learned*, by the same descent, priced by the same backward sweep.

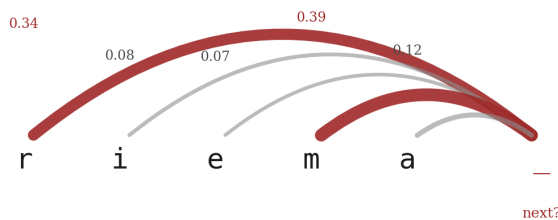


Figure 9.3 — Attention as weighted looking-back: arcs from the current position to each earlier letter; thickness the learned weight. (Weights illustrative.)

Figure 9.3 — Attention as weighted looking-back: arcs from the current position to each earlier letter, thickness the learned weight — the model’s own opinion about which letters matter for what comes next.

That is **attention**: weighted looking-back, with the weights learned. Stack a few layers of it, train on enough text instead of just names, and the same machine you built for mathematicians becomes the same *species* of machine — differing in size and data, not in kind — that completes your sentences and answers your questions. There is no further secret. There is more engineering than any one person holds, and vastly more data and electricity than any one person should casually invoke, but the idea on your screen is the idea. You have built the thing, and you understand the thing, and those are not the same achievement — the chapter’s last section is about the gap between them.

The transformer, with its formulas

The toy reads a fixed window of previous characters, flattens them, and pushes them through one hidden layer. It works, but its memory is a fixed-size lump: it cannot treat “the third letter back” differently depending on what that letter is. The transformer fixes exactly this, with a mechanism precise enough to write in five equations — and since you have linear algebra, we write them. Everything below is one **self-attention head**, the indivisible unit of the architecture, and it is implemented in full in Lab 9, gradient-checked against finite differences to six decimals.

Setup: characters become vectors. Fix a model dimension d (the lab uses $d = 32$). Two learned matrices turn symbols and positions into vectors: a token embedding $W_{tok} \in \mathbb{R}^{(V \times d)}$, one row per possible character, and a positional embedding $W_{pos} \in \mathbb{R}^{(T \times d)}$, one row per slot in the sequence (position matters — “n” after “riema” is not “n” starting a word). For an input sequence of T characters, stack their vectors into

$X = \text{token rows} + \text{position rows} \in \mathbb{R}^{T \times d}$

Row t of X is everything the model currently knows about position t : which letter, and where.

Query, key, value. Here is the move that is attention. Each position produces three different vectors from its X -row, via three learned matrices W_Q, W_K, W_V :

$Q = X W_Q, K = X W_K, V = X W_V$

The metaphor is a library lookup, and it is exact, not loose. Position t 's **query** q_t is what it is looking for ("I'm a consonant mid-word; I want the vowel I follow"). Each position's **key** k_s advertises what it offers ("I am the vowel 'e' at position 3"). The **value** v_s is the information that position hands over if attended to. Query asks, key answers, value delivers.

The attention weights. How much should position t attend to position s ? Take the dot product of t 's query with s 's key — large when they align, the geometric meaning of dot product you know — scale it, and softmax across all s so the weights form a distribution:

$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) \in \mathbb{R}^{T \times T}$

Entry $A_{t,s}$ is how much position t draws from position s , and each row sums to 1 — Chapter 8's softmax, doing the exact job it was built for. The $\sqrt{d_k}$ in the denominator is not decoration: without it, dot products in high dimension grow large, softmax saturates into a near-spike, and gradients vanish — Problem 3.4 has you feel why the scaling keeps learning alive. This A is a real, inspectable object; Lab 9 prints it, and you watch a trained model decide, for the name "riemann," that the second "n" should look hard at the first.

The causal mask. A model predicting the *next* letter must not peek at it. So before the softmax, every entry with $s > t$ — every "look ahead" — is set to $-\infty$, which softmax sends to weight zero:

$\text{scores}_{t,s} \leftarrow -\infty \text{ for } s > t$

This is why the attention matrix you print is lower-triangular:

position 3 attends to positions 0, 1, 2, 3 and nothing later. The mask is the one-line difference between a model that *generates* (each letter from its past) and a model that merely *fills in blanks*.

Output. Each position's new vector is the attention-weighted sum of the values — exactly the matrix product —

$$\text{context} = A V \in \mathbb{R}^{T \times d},$$

and a final learned matrix $W_{out} \in \mathbb{R}^{d \times V}$ turns each context vector into scores over the next character, which softmax-and-cross-entropy (Chapter 8) train as before. That is the whole head: embed, project to $Q/K/V$, score, mask, weight, combine, read out. Six matrix multiplies, one softmax, one mask — and every entry of every matrix is a knob your Chapter-9 backward sweep prices, because each operation here (matrix multiply, softmax, the scaling) is built from the Value operations you wrote, just batched into arrays by numpy. The lab implements this head from these equations and trains it on the names; its gradients agree with finite differences, which is to say *these formulas and your sixty-line engine are the same mathematics*.

From head to transformer, in three honest sentences. Real transformers do three things more, each a multiplier on what you built, none a new idea. They run several heads in *parallel* (each with its own W_Q, W_K, W_V , learning to attend for different reasons — one tracks vowels, another tracks word-endings) and concatenate the results: *multi-head attention*. They follow each attention layer with an ordinary per-position neural network (the layer from earlier in this chapter), and *stack* the attention-plus-network block dozens of times, each block refining the last. And they train on oceans of text rather than a hundred names. None of that changes the unit on your screen; it scales it. The architecture behind the systems of your era is this head, run in parallel, stacked deep, and fed the written world.

Deeper water: why softmax and cross-entropy are made for each other

The model's last step turns raw scores into probabilities with softmax, and the loss measures surprise with the negative log of the true letter's probability — *cross-entropy*. These two are not an arbitrary pair; they are built for each other, and the proof is a single clean gradient. Let the scores be $s_1, \dots, s_V$, let p be their softmax, and let the true next letter be class t . Then the loss is $L = -\log(p_t)$, and the gradient with respect to each score is astonishingly simple:

$\partial L / \partial s_i = p_i - [i = t]$ — the predicted probability, minus 1 at the true letter, and just the probability everywhere else.

Read what that *says*: the blame on each score is exactly *how wrong the probability was*. The model said the true letter had probability p_t ; the blame pushes that score up by $(1 - p_t)$ — hard when the model was confident and wrong, gently when it was nearly right — and pushes every wrong letter's score down by its own probability. The learning signal is the error itself, naked, with no extra machinery. This is why every classifier in the world pairs these two: the Diamond has you derive this gradient, and when you do, your Value engine will agree with your algebra to the last decimal, because it computed the very same derivatives, one operation at a time.

Deeper water: the size of the thing you did not build

Your name model has a few thousand knobs — about eight thousand, when Lab 9 counts them — and the public figure for a frontier model is the order of a *trillion*. The gap is eight powers of ten, and almost everything strange and unsettled about modern AI lives in that gap. The training loop is identical; the architecture is the same attention you built, stacked; the loss is the same cross-entropy. What changes across twelve orders of magnitude is not the algorithm but the *behavior* — capabilities appear that no one designed and few predicted, and the question of why a trillion-knob next-letter-guesser, trained on enough text, does what it does is not a settled engineering mat-

ter but an open scientific one, pursued right now by people using tools you would recognize from this book: the spectra of large matrices, the geometry of high-dimensional landscapes, the search for phase transitions in training. The author of this book is among them. You have built the small version completely. The large version is built completely too — and understood by no one completely, which is the most honest and most exciting sentence this book can end its technical chapters on.

Where this leaves us

The bill from Chapter 8 is paid. Backpropagation — one forward pass recording a graph, one backward pass pricing it — gets the slope of every knob at once, and you built it from sixty lines that were really just Chapters 3, 5, 7, and Book 2’s derivatives, assembled. Around it, three small classes compose knobs into a network; numpy scales the network to where it can speak; attention upgrades it to the architecture of the present age. And the parrot exam, throughout, kept the enterprise honest: a model that memorizes is not a model that learned, and the gap between the two is visible on your own screen in a one-minute run.

You set out, nine chapters ago, to learn the usual algorithms — greedy, recursive, the ones in a college book — and some foundations of the machines that learn. You have them, and you have something the usual treatment rarely conveys: every one of these ideas, from binary search to backpropagation, is a particular way of *refusing to do most of the work*, and the refusals were invented by people, sat unnoticed for years as often as not, and are still being invented now, by a community whose tools you have just spent a book acquiring. There remains one short chapter, and it is not about a method. It is about the edges of the whole enterprise — the work that cannot be refused, the problems no algorithm will ever solve, and the questions that are open not because this book is elementary but because

no one on Earth yet knows the answers. The map has edges, and standing at them is the best view in the subject.

Problem Set 9

★ ★ ★

Lab 9 (The Engine) is the Computational Lab tier — the chapter’s claims are not real until the engine trains in your hands. The Diamond’s derivation is checked against your own engine’s output.

Fluency

1.1 By hand, build the Value graph for $L = (a + b) \cdot c$ with $a = 2$, $b = 3$, $c = 4$: draw each node, its value, and its parents. Then run backprop on paper — seed the output with grad 1, apply each operation’s blame-rule — and find $\partial L / \partial a$, $\partial L / \partial b$, $\partial L / \partial c$. (Check: $\partial L / \partial c$ should be $a + b = 5$.)

1.2 For the addition rule, explain in one sentence why blame passes to both parents *unchanged*; for the multiplication rule, why blame is scaled by the *other* factor. Tie each to a derivative from Book 2.

1.3 Trace the backward method’s depth-first build on the graph of 1.1: in what order are the nodes appended, and why must the output node be processed *first* in the reversed walk? Connect to Chapter 5’s claim that this traversal is Chapter 3’s recursion.

1.4 A single neuron with two inputs has how many knobs? A layer of 4 such neurons? A two-layer network, 2 inputs $\rightarrow$ 4 hidden $\rightarrow$ 1 output? (Count weights and biases separately, then total — this is the skill Lab 9 uses to size the real model.)

1.5 Softmax by hand: scores $[1, 2, 0] \rightarrow$ probabilities (use e^1 ,

e^2 , e^0 ; normalize). Then compute the cross-entropy loss if the true class is the middle one, and—using the deep water’s formula—the gradient on all three scores. Confirm they sum to zero, and say in one sentence why they must.

1.6 “The model feeds its own output back as input to generate.” Walk through generating a 4-letter name by hand given a model that always predicts the most likely next letter from a tiny table you invent — show the context sliding forward one letter at a time, including the start and stop markers.

Practice

2.1 Extend the Value class with a subtraction rule (`__sub__`) and state its blame-rule, deriving it from addition and negation. Then a division rule — harder; use $a \cdot b^{-1}$ and the power rule, and write the blame-rule for b^{-1} explicitly. (Lab 9 has you test both against finite differences.)

2.2 A model’s training loss after each epoch: 3.1, 1.9, 1.3, 1.1, 1.05, 1.02. Its held-out loss: 3.0, 2.0, 1.8, 2.1, 2.6, 3.2. Plot both by hand (rough axes), mark the epoch where you would *stop training*, and explain in the chapter’s language what is happening to the model after that point. What is it learning instead of the pattern?

2.3 Temperature in generation: sampling divides scores by a temperature T before softmax. Predict, with reasoning, what the generated names look like as $T \rightarrow 0$ (very small) and as $T \rightarrow \text{large}$. (Lab 9 measures both; commit to predictions first.) Connect T to a knob you have already met — what does dividing all scores by a constant do to their softmax spread?

2.4 Attention as weighted looking-back: a model guessing the letter after “albe—” has learned attention weights over the five prior letters. Invent plausible weights (summing to 1) that would help predict “r”, and explain which letters earn the weight and why. Then change the stem to “albea—” and argue how the weights should

shift. (You are reasoning about what the figure’s arcs encode.)

2.5 Parameter counting at scale: your name model has $\sim Q$ knobs (use your 1.4 method on Lab 9’s actual dimensions — you should land near eight thousand). A frontier model has $\sim 10^{12}$. Compute the ratio as a power of ten, and then a humbling comparison: if your knobs were seconds, how long is your model’s training; if theirs were seconds, how long is theirs? (The gap between “an afternoon” and “an age” is the gap the last deep water is about.)

2.6 Query, key, value by hand. A single attention head has $d = 4$, and at some position the query is $q = (1, 0, 1, 0)$. Three earlier positions have keys $k_1 = (1, 0, 0, 0)$, $k_2 = (1, 0, 1, 0)$, $k_3 = (0, 1, 0, 0)$. Compute the three dot products $q \cdot k_s$, divide by $\sqrt{d}$, and apply softmax to get the attention weights. Which position does this query attend to most, and why does the *direction* of the key vectors — not their labels — decide it?

2.7 The mask, concretely. Write the 4×4 matrix of raw scores for a length-4 sequence as a general grid, then show exactly which entries the causal mask sets to $-\infty$ and what the matrix looks like after. After softmax, why is the resulting attention matrix lower-triangular, and what would a model that *skipped* the mask be able to do that makes its low training loss worthless? (Lab 9 Break-it 5 is this problem, executed.)

2.8 Dimensions audit. For the lab’s head ($d = 32$, $d_k = 32$, vocabulary V , sequence length T), give the shape of every matrix in the five equations: X , Q , K , V , QK^T , A , the context AV , and the logits. Confirm the chain of shapes is consistent (each product’s inner dimensions match), and identify which single matrix’s row-count is “one per possible character” rather than “one per position.” (This shape-checking is most of what catches bugs in real model code.)

Concept & Counterexample

3.1 Claim: “you need calculus to know each operation’s blame-rule, so backprop can’t be elementary.” Half true. Show that *each individual rule* is a single Book 2 derivative (list the rules for $+$, $\cdot$, x^n , $\exp$, $\log$), and that the *algorithm* combining them needs no calculus at all — only graph traversal. State precisely where the mathematics lives and where the computer science does.

3.2 Claim: “a deeper network needs a more complicated backprop.” Refute using the chapter’s central structural fact: what does `backward()` actually operate on, and why is it blind to how deep or wide the network is? Construct the one-sentence argument that the *same sixty lines* train a 2-layer and a 200-layer network.

3.3 Claim: “the model generates real mathematician names, so it has memorized the list.” Distinguish, with the parrot exam: what would *perfect memorization* look like in the generated samples (and in the two loss curves)? What does the appearance of *novel plausible* names prove instead? Then the subtle case from the lab: if the model *does* emit some exact training names alongside novel ones, has it memorized? What single experiment settles it?

3.4 Claim: “softmax and cross-entropy are just two reasonable choices among many.” The deep water shows they are *matched*. Pair softmax instead with a different loss — squared error on the probabilities — and argue qualitatively why the gradient is messier and the learning signal less clean. (You need not compute it fully; identify what the clean “ $p - 1$ ” structure depended on.)

3.5 Claim: “attention is fundamentally different from the simple model — a new kind of thing.” Argue the opposite from the chapter: name everything attention *shares* with the toy (the loss, the descent, the backward sweep, the softmax), and isolate the single thing it adds (learned, input-dependent weights over all prior positions). Is the transformer a new *idea* or a new *arrangement* of old ideas? Defend your answer.

3.6 Claim: “the $\sqrt{d_k}$ scaling in attention is a minor numerical tweak.” Refute it with the mechanism: when query and key entries are independent with unit variance, what is the typical size of their dot product over d_k dimensions (think about summing d_k independent products — Chapter 7’s linearity of expectation, and what happens to the *spread*)? Then argue what an unscaled, large-magnitude score does to softmax (where do its outputs go?) and to the gradient through that softmax. The scaling is the difference between a head that learns and a head that is frozen at birth; say precisely why.

Diamond

4.1 The matched pair, derived. Prove the deep water’s claim: for softmax probabilities p over scores $s_1, \dots, s_V$ with true class t and loss $L = -\log(p_t)$, $\partial L / \partial s_i = p_i - [i = t]$. (a) Write $p_t = e^{s_t} / \sum_j e^{s_j}$ and $L = -s_t + \log(\sum_j e^{s_j})$. (b) Differentiate the two terms of L with respect to a general score s_i , carefully handling the case $i = t$ versus $i \neq t$ in the first term, and using the chain rule on the log-sum in the second. (The derivative of the log-sum is where p_i is born — show it.) (c) Combine to get $p_i - [i = t]$, and interpret the result as a sentence about blame: confident-and-wrong versus nearly-right. (d) The verification that closes the loop: explain why your hand-derived gradient *must* match what the Value engine computes when you build this same loss out of $\exp$, $\log$, and division — and what it would mean about your algebra (or the engine) if they disagreed. This is the chapter’s deepest point: the engine and the calculus are the same thing, one done by machine and one by hand.

4.2 (Alternative Diamond — attention.) For a single attention head, derive the gradient of the loss with respect to the query matrix W_Q , at least through the first link of the chain. (a) Write the output context vector for position t as a weighted sum of value vectors with weights $A_{\{t,s\}}$, and A as the softmax of scaled scores $QK^T/\sqrt{d_k}$.

(b) Using the softmax gradient you derived in 4.1 (the same $p - [i=t]$ structure, now per attention row), express $\partial(\text{loss})/\partial(\text{scores})$ in terms of the incoming gradient on the context. (c) Push one more link: $\partial(\text{scores})/\partial Q$ is linear in K , so assemble $\partial(\text{loss})/\partial W_Q$ via the chain rule and X . You need not simplify fully — the point is to *see* that attention’s gradient is the same softmax-and-chain machinery as the classifier’s, which is why your Value engine trains it without knowing it is “attention.” (d) State, in one sentence, what this shared machinery says about the unity of everything in this chapter.

Computational Lab

5.1 Lab 9 in full. The engine is not real until (a) it matches finite-difference gradients on a messy expression, (b) it trains a tiny model end-to-end by your own backward sweep, and (c) the numpy model generates novel names and shows you the train/test gap opening. Export your parameter count (Problem 2.5) and your two loss curves (Problem 2.2).

Teach It

6.1 Explain backpropagation to someone using the word “blame” and no calculus: a final mistake is blamed on the things that caused it, each cause blamed in proportion to how much it mattered, blame flowing backward through the steps until every knob knows its share. Use a concrete chain (the bowl, or a two-step calculation). Then show them the actual backward method and point to where “blame flows backward” is the reversed loop. Report which half — the metaphor or the code — made it click.

6.2 Show someone the model generating names, then tell them the honest two-part truth this book ends its technical arc on: *we built exactly this, and we understand exactly this; the trillion-knob version is built exactly too, and understood by no one exactly.* Discuss the gap

between building and understanding. Report their reaction to the idea that the most powerful technology of their era is, at bottom, this — counted steps, downhill walks, blame flowing backward — scaled twelve orders of magnitude past where anyone can fully follow it.

Lab 9: The Engine

★ ★ ★

The capstone. You build backpropagation from nothing — sixty lines — verify it against the calculus by hand and against finite differences by machine, then train a tiny model end to end with your own engine to prove it is real. Then numpy arrives as the scale instrument, the name model trains until it speaks, and the parrot exam from Chapter 8 returns with teeth: you watch the training loss fall while the held-out loss begins to climb, and you name the moment they part. Everything in nine chapters has been parts on a workbench. This is the build.

1. The number that remembers

A plain number forgets its history. Build one that doesn't — the `Value`, learned in pieces. Start with the skeleton and the two core operations:

```
import math, random

class Value:
    def __init__(self, data, parents=()):
        self.data = data
        self.grad = 0.0
        self._parents = set(parents)
        self._backward = lambda: None

    def __add__(self, other):
```

```
other = other if isinstance(other, Value) else
    ↪ Value(other)
out = Value(self.data + other.data, (self, other))
def _backward():
    self.grad += out.grad
    other.grad += out.grad
out._backward = _backward
return out

def __mul__(self, other):
    other = other if isinstance(other, Value) else
    ↪ Value(other)
    out = Value(self.data * other.data, (self, other))
    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad
    out._backward = _backward
    return out

a = Value(2.0)
b = Value(-3.0)
c = a * b
print(c.data, c._parents)
```

`Value(2.0)` builds a number-that-remembers; `c = a * b` builds a *new* one that remembers `a` and `b` as parents. The `isinstance` guard lets you write `a + 1` (it wraps the bare `1` into a `Value`). Each operation's `_backward` is a tiny function holding that operation's *derivative* — addition passes blame whole, multiplication scales it by the other factor. Confirm `c` knows its parents before moving on.

2. The backward sweep

History recorded forward; now price it backward. The sweep is Chapter 5's depth-first traversal, walked in reverse:

```
def backward(self):
    order = []
    visited = set()
    def build(v):
        if v not in visited:
            visited.add(v)
            for parent in v._parents:
                build(parent)
            order.append(v)
    build(self)
    self.grad = 1.0
    for v in reversed(order):
        v._backward()
```

Add this method inside the `Value` class (same indentation as `__add__`). Then the moment of truth — build a small expression, sweep, and check against calculus you can do by hand:

```
a = Value(2.0)
b = Value(-3.0)
c = Value(7.0)
L = (a * b + c) * (a * b + c)      # L = (ab + c)^2
L.backward()
print("L    =", L.data, " (expect 1)")
print("a.grad =", a.grad, " (expect -6)")
print("b.grad =", b.grad, " (expect 4)")
print("c.grad =", c.grad, " (expect 2)")
```

By hand: $ab + c = 1$, so $L = 1$; the outer square contributes $2 \cdot (ab + c) = 2$; then $\partial L / \partial a = 2 \cdot b = -6$, $\partial L / \partial b = 2 \cdot a = 4$, $\partial L / \partial c = 2$. The engine should agree exactly. It does — and you have just watched blame flow from one number out to three, through a graph the computation recorded itself.

3. More operations, tested against reality

The model needs squashing and probabilities. Add these inside the class, each three lines, each just its derivative:

```
def __pow__(self, p):
    out = Value(self.data ** p, (self,))
    def _backward():
        self.grad += p * (self.data ** (p - 1)) * out.grad
    out._backward = _backward
    return out

def tanh(self):
    t = math.tanh(self.data)
    out = Value(t, (self,))
    def _backward():
        self.grad += (1 - t * t) * out.grad
    out._backward = _backward
    return out

def exp(self):
    e = math.exp(self.data)
    out = Value(e, (self,))
    def _backward():
        self.grad += e * out.grad
    out._backward = _backward
    return out

def log(self):
    out = Value(math.log(self.data), (self,))
    def _backward():
        self.grad += (1 / self.data) * out.grad
    out._backward = _backward
    return out
```

```
def __truediv__(self, other):
    other = other if isinstance(other, Value) else
    ↪ Value(other)
    return self * other ** -1

def __neg__(self): return self * -1
def __sub__(self, other):
    other = other if isinstance(other, Value) else
    ↪ Value(other)
    return self + (-other)
def __radd__(self, other): return self + other
def __rmul__(self, other): return self * other
```

Now verify the *whole engine* the way professionals verify gradients: against **finite differences**. Calculus says the derivative is the limit of $(f(x+h) - f(x-h)) / 2h$; compute that numerically and check the engine matches:

```
def f(av, bv, cv):
    return (math.tanh(av * bv) + cv ** 2) * (av + cv)

a = Value(0.5); b = Value(1.3); c = Value(-0.7)
out = ((a * b).tanh() + c ** 2) * (a + c)
out.backward()

h = 1e-6
ga = (f(0.5 + h, 1.3, -0.7) - f(0.5 - h, 1.3, -0.7)) / (2 * h)
print("a.grad engine:", round(a.grad, 5), "  finite diff:",
    ↪ round(ga, 5))
```

They agree to five decimals — your sixty lines compute the same derivatives the limit does, on an expression you'd never want to differentiate by hand. This is the test that makes the engine *trustworthy*: not a proof, but the check every practitioner actually runs.

4. A tiny model, trained by your own engine

Before numpy, prove the engine can *learn* — train a tiny next-letter model with nothing but Values and your backward sweep. Small enough to be pure Value, real enough to work:

```

random.seed(0)
names = ["ada", "gauss", "euler", "abel", "nash", "hardy",
  ↪ "klein", "peano", "boole", "bayes"]
chars = sorted(set("".join(names))) + ["."]
stoi = {c: i for i, c in enumerate(chars)}
V = len(chars)

# knobs: a score for each (previous letter -> next letter)
↪ pair
W = [[Value(random.gauss(0, 0.1)) for _ in range(V)] for _ in
  ↪ range(V)]

data = []
for name in names:
    cs = ["."] + list(name) + ["."]
    for prev, nxt in zip(cs, cs[1:]):
        data.append((stoi[prev], stoi[nxt]))

for step in range(40):
    loss = Value(0.0)
    for prev, nxt in data:
        scores = W[prev]
        m = max(s.data for s in scores)                # for
        ↪ numerical stability
        exps = [(s - m).exp() for s in scores]
        Z = exps[0]
        for e in exps[1:]:
            Z = Z + e
        p = exps[nxt] / Z

```

```
    loss = loss + (-(p.log()))                #
    ↪ cross-entropy, Chapter 8
    loss = loss * (1.0 / len(data))

    for row in W:
        for w in row:
            w.grad = 0.0                      # reset
            ↪ blame each step
        loss.backward()                       # YOUR
        ↪ sweep prices every knob
    for row in W:
        for w in row:
            w.data = w.data - 1.0 * w.grad    #
            ↪ gradient step, Chapter 8

    if step % 10 == 0:
        print("step", step, " loss", round(loss.data, 3))
```

The loss falls — and every gradient driving it down came from the sweep *you wrote*, firing each operation’s blame-rule across a graph the computation built itself. This is the entire chapter, working, in pure Python. (Why reset `w.grad = 0.0` each step? Predict what the `+=` in every blame-rule does across steps if you don’t — then hold it for Break-it 1.)

5. NumPy, and a model that can speak

Pure Value is for understanding; **numpy** is for scale — arrays of numbers operated on all at once, the same mathematics a thousand times faster. The full name model (a 3-letter-context network) is provided as one block; read it for the shape you already know — embedding, hidden layer with tanh, scores, softmax, cross-entropy — and run it:

```
import numpy as np
rng = np.random.default_rng(1)

names = ""ada gauss euler noether hilbert euclid fermat
↳ cantor godel turing lovelace ramanujan
poincare riemann hardy erdos klein hopper church kolmogorov
↳ galois abel jacobi lagrange laplace
fourier cauchy leibniz newton pascal bernoulli dirichlet
↳ weierstrass dedekind peano banach markov
bayes nash hamilton cayley boole russell minkowski hausdorff
↳ lebesgue hermite chebyshev legendre
maxwell dirac feynman fermi bohr planck einstein pauli wiener
↳ shannon hamming huffman dijkstra
hoare knuth karp cook tarski kleene curry milner tarjan floyd
↳ kruskal dantzig weyl cartan
grothendieck serre deligne tate wiles taylor mazur selberg
↳ renyi turan lovasz tao gowers"".split()
names = list(dict.fromkeys(names))

chars = sorted(set("".join(names))) + ["."]
stoi = {c: i for i, c in enumerate(chars)}; itos = {i: c for
↳ c, i in stoi.items()}
V = len(chars); CONTEXT = 3

# hold out 15% of names as a test set the model never trains
↳ on
perm = rng.permutation(len(names)); split = int(0.85 *
↳ len(names))
train_names = [names[i] for i in perm[:split]]
test_names = [names[i] for i in perm[split:]]

def make(nms):
    X, Y = [], []
    for name in nms:
        cs = ["."] * CONTEXT + list(name) + ["."]
```

```

        for i in range(len(cs) - CONTEXT):
            X.append([stoi[c] for c in cs[i:i+CONTEXT]]);
            ↪ Y.append(stoi[cs[i+CONTEXT]])
        return np.array(X), np.array(Y)
X, Y = make(train_names); Xte, Yte = make(test_names)

E, H = 12, 128
C = rng.standard_normal((V, E)) * 0.1
W1 = rng.standard_normal((CONTEXT * E, H)) * 0.1; b1 =
    ↪ np.zeros(H)
W2 = rng.standard_normal((H, V)) * 0.1;          b2 =
    ↪ np.zeros(V)

def forward(Xb):
    emb = C[Xb].reshape(len(Xb), -1)
    h = np.tanh(emb @ W1 + b1)
    return emb, h, h @ W2 + b2

def loss_on(Xb, Yb):
    _, _, logits = forward(Xb); n = len(Xb)
    logits = logits - logits.max(1, keepdims=True)
    p = np.exp(logits); p /= p.sum(1, keepdims=True)
    return -np.log(p[np.arange(n), Yb] + 1e-9).mean()

train_curve, test_curve = [], []
for step in range(8000):
    emb, h, logits = forward(X); n = len(X)
    logits = logits - logits.max(1, keepdims=True)
    p = np.exp(logits); p /= p.sum(1, keepdims=True)
    # backprop, by hand in numpy – the same blame-rules,
    ↪ batched
    dlogits = p.copy(); dlogits[np.arange(n), Y] -= 1; dlogits
    ↪ /= n
    dW2 = h.T @ dlogits + 0.001 * 2 * W2; db2 = dlogits.sum(0)
    dh = dlogits @ W2.T; dhraw = (1 - h**2) * dh

```

```

dW1 = emb.T @ dhraw + 0.001 * 2 * W1; db1 = dhraw.sum(0)
demb = (dhraw @ W1.T).reshape(n, CONTEXT, E)
dC = np.zeros_like(C); np.add.at(dC, X.reshape(-1),
    ↪ demb.reshape(-1, E))
for param, grad in [(C, dC), (W1, dW1), (b1, db1), (W2,
    ↪ dW2), (b2, db2)]:
    param -= 0.3 * grad
if step % 400 == 0:
    tr, te = loss_on(X, Y), loss_on(Xte, Yte)
    train_curve.append(tr); test_curve.append(te)
    print(f"step {step:4d}  train {tr:.3f}  held-out
    ↪ {te:.3f}")

```

Watch the two columns. Early on both fall. Then they *part*: train keeps dropping while held-out bottoms out and turns upward. That divergence is the parrot exam catching the model beginning to memorize — and you are seeing, on your laptop in under a minute, the single most consequential phenomenon in modern machine learning. Plot it:

```

import matplotlib.pyplot as plt
steps = [400 * i for i in range(len(train_curve))]
plt.plot(steps, train_curve, label="training loss")
plt.plot(steps, test_curve, label="held-out loss")
plt.xlabel("step"); plt.ylabel("loss"); plt.legend();
    ↪ plt.show()

```

6. The machine speaks

Generate names — feed the model a start, take its prediction, feed it back, repeat:

```

def sample(temp=1.0):
    ctx = [stoi["."]] * CONTEXT; out = []

```

```

for _ in range(20):
    emb = C[np.array([ctx])].reshape(1, -1)
    h = np.tanh(emb @ W1 + b1)
    logits = (h @ W2 + b2)[0] / temp
    logits -= logits.max(); pr = np.exp(logits); pr /=
    ↪ pr.sum()
    ix = rng.choice(V, p=pr)
    if itos[ix] == ".": break
    out.append(itos[ix]); ctx = ctx[1:] + [ix]
return "".join(out)

known = set(names)
print("generated names (★ = novel):")
for _ in range(25):
    name = sample(1.0)
    print(f"    {name:<16} {'★ never existed' if name not in
    ↪ known else '(in training)'}")

```

Names like *Riemanujan*, *Dantor*, *Wilevin* — nobody’s, everybody’s plausible. The model learned the *music* of the names without copying them, which is learning itself. Some exact training names may appear too; Problem 3.3 asks what that does and doesn’t prove. Then run §6 again with `temp=0.4` and `temp=1.6` (Problem 2.3’s prediction): cold sampling plays it safe and wordlike, hot sampling gets adventurous and strange.

7. A real attention head

The toy model read a flat window. Now build the unit behind the transformer — one self-attention head, from the chapter’s five equations, in numpy. This is the architecture of your era, small enough to read whole.

The pieces, dimensioned (D = model dimension, V = vocabulary, `BLOCK` = longest sequence):

```
import numpy as np
rng = np.random.default_rng(0)

# reuse names / chars / stoi / itos / V from §5; set up
↳ dimensions
BLOCK = 16
D = 32; Dk = 32
Wtok = rng.standard_normal((V, D)) * 0.1      # character ->
↳ vector
Wpos = rng.standard_normal((BLOCK, D)) * 0.1 # position ->
↳ vector
WQ = rng.standard_normal((D, Dk)) * 0.1      # query
↳ projection
WK = rng.standard_normal((D, Dk)) * 0.1      # key projection
WV = rng.standard_normal((D, D)) * 0.1      # value
↳ projection
Wout = rng.standard_normal((D, V)) * 0.1     # context ->
↳ next-char scores

def softmax_rows(M):
    M = M - M.max(axis=-1, keepdims=True)
    e = np.exp(M)
    return e / e.sum(axis=-1, keepdims=True)
```

The forward pass is the five equations, in order — embed, project to Q/K/V, score, mask, weight, combine, read out:

```
def forward(seq):
    T = len(seq)
    X = Wtok[seq] + Wpos[:T]                # (T,D) symbols
    ↳ + positions
    Q = X @ WQ; K = X @ WK; Vv = X @ WV    # the three
    ↳ projections
    scores = Q @ K.T / np.sqrt(Dk)          # (T,T) query ·
    ↳ key, scaled
```

```

mask = np.triu(np.ones((T, T)), k=1).astype(bool)
scores = np.where(mask, -1e9, scores)      # causal: no
↳ peeking ahead
A = softmax_rows(scores)                  # (T,T)
↳ attention weights
context = A @ Vv                          # (T,D)
↳ weighted values
logits = context @ Wout                   # (T,V)
↳ next-char scores
return logits, A

seq = [stoi[c] for c in "." + "riemann"]
logits, A = forward(seq)
print("attention matrix shape:", A.shape, " (should be square,
↳ one row per position)")

```

Read A and confirm it is **lower-triangular** — each position attends only to itself and earlier, the mask at work. That triangle is the difference between generating and blank-filling.

Training is the same loop (the full gradient code is provided in the notebook — it is the chapter’s backward sweep applied to each matrix, batched). Run it, then **inspect what the model learned to look at**:

```

# after training (provided), print the attention matrix for a
↳ name
seq = [stoi[c] for c in "." + "riemann"]
_, A = forward(seq)
labels = list("." + "riemann")
print("      " + " ".join(f"{c:>5}" for c in labels))
for i, row in enumerate(A):
    print(f"{labels[i]:>4} " + " ".join(f"{x:5.2f}" for x in
↳ row))

```

You will see real structure: a consonant attending to the vowel it follows, the second “n” looking back at the first. These weights

were not set by anyone — descent found them, your backward sweep priced them, and now you can read the model’s own opinion about which letters matter for what comes next. This is Figure 9.3, computed in your hands.

Break it

Predict first. Run second. Explain any gap third.

1. *Blame that never resets.* In §4, delete the `w.grad = 0.0` reset loop and run. Predict what happens to the loss before running — converges faster? slower? diverges? wildly? Then read the result and explain it through the `+=` in every blame-rule: across steps, what is each knob’s grad accumulating, and why does that poison the descent? (This exact bug — forgotten gradient reset — is the single most common mistake in real deep-learning code; you have now made it deliberately and watched it bite.)

2. *The engine versus the calculus.* Build the softmax-plus-cross-entropy loss for three scores out of `exp`, `log`, and `division`, on `Values`, and run backward. Separately, compute the gradient by the deep water’s formula $p - [i = t]$ by hand. Predict they match; verify they do to several decimals. This is the Diamond made executable — and the deepest point of the chapter. State in one sentence what the agreement proves about the relationship between your engine and Book 2.

3. *Watching it memorize.* In §5, shrink the training set hard — keep only the first 15 names — and retrain. Predict both curves before running: where will the held-out loss go, and how early will it turn up? Then generate from this starved model and report what fraction of samples are exact copies. Connect all of it to one sentence: what does a model do when it has more knobs than pattern to learn? (You are reproducing, deliberately, the overfitting the chapter says no one fully understands at scale.)

4. *Temperature to the limits.* Generate 10 names each at `temp = 0.1`,

temp = 1.0, and temp = 3.0. Predict the character of each batch first. At very low temperature the model becomes nearly deterministic — predict what it converges toward (is it always the *same* name? why?). At very high temperature, predict whether the output is still namelike at all. Then connect temperature to Chapter 8: dividing all scores by T before softmax does what, geometrically, to the gap between the likeliest letter and the rest?

5. *Breaking the mask.* In §7's forward, delete the two masking lines (the mask = ... and the scores = np.where(...)). The model can now attend to *future* characters — including the very one it is trying to predict. Predict what happens to the training loss before running: does it improve, and is that improvement real or fraudulent? Then check by trying to *generate* from the unmasked model and explaining why generation falls apart even though the loss looked wonderful. State the one-sentence moral about a model that is allowed to see the answer it is being graded on. (This connects directly to Chapter 8's parrot exam — what is leakage of the test answer into the training input?)

Epilogue: The Edge of the Map

★ ★ ★

No problems. No lab. Written to be over your head now and exactly right later.

Nine chapters asked two questions of one algorithm after another — *is it right? what does it cost?* — and answered them, every time. This last chapter is about the questions that do not get answered. Not because the book is elementary, though it is, but because **no one on Earth yet knows the answers**, or because the answers are *no* — there are problems no algorithm will ever solve, and the proof that they exist is one of the most beautiful things a human mind has produced. The map you have been drawing has edges. Standing at them, looking out, is the best view in the subject.

Work that cannot be refused

The whole book was a campaign of refusal: an algorithm is a way of *not doing most of the work*, and chapter after chapter found the work that could be skipped. So a fair question — the kind that ends some inquiries and opens others — is whether refusal has a limit. Is there work that *cannot* be skipped, a problem so hard that no clever trick, no idea however deep, no machine however fast, will ever solve it?

There is, and the first example is not exotic. It is a question about programs themselves.

You met it in Chapter 3, and were told to hold it. Checking that *your* recursion terminates was easy — it shrank toward a floor. But could there be a single algorithm — call it the **halt-checker** — that reads *any* program, together with its input, and correctly answers one yes-or-no question: *does this program eventually stop, or does it run forever?* Such a thing would be priceless. It would catch every infinite loop before it ran, prove software correct, settle open mathematical conjectures by running a search and asking whether it ever halts. Surely, with enough cleverness, it can be built.

It cannot. And here is the proof — the **halting problem**, proved by Alan Turing in 1936, before there were computers to run the programs it is about.

The barber who shaves himself

The argument is a descendant of an old paradox. A town has one barber, who shaves *exactly those men who do not shave themselves*. Does the barber shave himself? If he does, then he is a man who shaves himself, so by the rule the barber does *not* shave him — contradiction. If he does not, then he is a man who does not shave himself, so by the rule the barber *does* shave him — contradiction. There is no consistent answer. The town, as described, cannot exist.

Turing built that barber out of programs. Suppose, for contradiction, the halt-checker exists: a program `HALTS(P, x)` that always stops and correctly returns *yes* if program `P` halts on input `x`, *no* if it loops forever. From it, build a second program — call it `TROUBLE` — that takes a single program `P` as input and does this:

ask `HALTS(P, P)` — does `P` halt when fed *its own code* as input? if the answer is *yes*, deliberately loop forever; if the answer is *no*, immediately stop.

`TROUBLE` does the opposite of whatever the halt-checker predicts.

Now ask the question that destroys the town. **Run TROUBLE on its own code.** Does `TROUBLE(TROUBLE)` halt?

- Suppose it halts. It halts only if, inside, `HALTS(TROUBLE, TROUBLE)` returned *no* — i.e. the halt-checker predicted it runs forever. The checker was wrong.
- Suppose it runs forever. It loops only if `HALTS(TROUBLE, TROUBLE)` returned *yes* — i.e. the checker predicted it halts. The checker was wrong.

Either way the halt-checker gives the wrong answer about `TROUBLE(TROUBLE)`. But the halt-checker was assumed *never* to be wrong. The assumption cannot stand. **No halt-checker exists** — not “none has been found,” not “none is fast enough,” but none is *possible*, ever, on any machine, by any mind. The work cannot be refused because it cannot be done.

Deeper water: this is Chapter 6’s fence, grown infinite

Look at the shape of what just happened. Chapter 6 proved no comparison sort beats $n \log n$ by counting: $n!$ distinct situations, too few questions to tell them apart. Chapter 1 fenced the guessing game the same way. This is that argument’s furthest reach — a *diagonal* argument, the technique Georg Cantor invented in 1891 to prove some infinities are larger than others. Imagine an infinite table: one row per program, one column per input, each cell marked “halts” or “loops.” `TROUBLE` is built to *disagree with the table’s diagonal* — to differ from program 1 on input 1, program 2 on input 2, program k on input k , forever. So `TROUBLE` cannot be any row of the table; it differs from each at the diagonal. Yet `TROUBLE` is a perfectly good program — it should be *some* row. Contradiction, built by walking down the diagonal and flipping every entry. The same diagonal flip proves there are more real numbers than whole numbers, more functions than programs to compute them, more truths than proofs.

It is the single most powerful idea about the limits of systems, and you have now seen it draw a fence around computation itself.

Easy to check, hard to find

The halting problem is a wall: solving it is *impossible*. The second edge of the map is stranger, because no one knows whether it is a wall or a door. It is the most important open question in this subject, and one of the deepest in all of mathematics, and you can understand exactly what it asks.

Some problems are hard to *solve* but easy to *check*. A finished jigsaw is obviously finished — one glance. Finding the arrangement took hours; verifying it takes a second. A sudoku solution is checked in a moment; finding it is work. This gap — between *finding* an answer and *recognizing* a correct one — has a precise form, and a name.

Consider a concrete puzzle, the granddaddy of them all. You are given a logical formula: a string of true/false variables joined by ANDs and ORs, like $(a \text{ or } b \text{ or not-}c) \text{ and } (\text{not-}a \text{ or } c \text{ or } d) \text{ and } \dots$, with hundreds of variables and clauses. The question: is there any setting of the variables that makes the whole thing true? This is **satisfiability**, and notice its two faces. If someone *hands* you a setting that works, you check it in an eyeblink — plug in the values, evaluate each clause, done, a few hundred steps. But *finding* a setting, with no one to hand you one? The obvious method is Chapter 1's enemy: try all 2^n settings, an exponential wall, hopeless past a few dozen variables. Centuries of cleverness have not found a fast method, and satisfiability is not alone — thousands of problems, from scheduling to protein folding to optimal routing, share its exact character: *a solution is checkable in polynomial time (the “easy” growth shapes of this book — n , n^2 , $n \log n$); a solution seems to require exponential time to find.*

Call the easy-to-solve problems **P** (solvable in polynomial time —

everything this book built fast algorithms for). Call the easy-to-check problems **NP** (a proposed solution is *verifiable* in polynomial time — satisfiability, and its thousands of cousins). Every **P** problem is obviously in **NP**: if you can solve it fast, you can check a solution fast by solving it yourself. The question — the whole question — is whether it goes the other way:

Is **P equal to **NP**?** Is every problem whose solutions are easy to *check* also a problem whose solutions are easy to *find*?

If **P** = **NP**, then finding is secretly no harder than checking — there is a fast algorithm for satisfiability, for thousands of problems we believe are hard, and almost every optimization humanity cares about falls at once. Much of modern cryptography, which rests on certain problems being hard to *solve* but easy to *check*, would dissolve; so would much of the difficulty of science. If **P** $\neq$ **NP**, then some problems are genuinely, permanently hard to solve despite being trivial to verify — finding really is harder than recognizing, and the exponential wall around satisfiability is real and forever.

Almost everyone who studies it believes **P** $\neq$ **NP**. No one has proved it. The problem was crisply posed around 1971, by Stephen Cook and Leonid Levin — independently, on two sides of a closed border, the book’s recurring pattern one final time. It carries a million-dollar prize, unclaimed. It is, very possibly, the deepest question this book could point at, and it remains exactly as open today as the day it was asked. The honest sentence, the one this whole book has been teaching you to say: *we do not know, and the not-knowing is not a gap in your education — it is the live edge of human knowledge, and you are now standing on it.*

Deeper water: the strange unity of the hard problems

One fact makes **P** vs **NP** sharper than “some problems seem hard.” In 1971 Cook proved that satisfiability is **NP-complete**: every single problem in **NP** can be efficiently *translated into* a satisfiability question. Then Richard Karp, in 1972, showed

twenty-one classic problems — scheduling, graph coloring, the traveling salesman among them — are NP-complete too, and the list has since grown to thousands. They are, in a precise sense, *the same problem in different clothes*: a fast algorithm for any one of them would yield a fast algorithm for *all* of them, and would prove $P = NP$. They stand or fall together. This is why the question is not a curiosity about one puzzle but a hinge under all of computation: thousands of independent walls turn out to be one wall, and no one knows if it has a door. (The translation between them, by the way, is itself an algorithm — efficient, constructive, the book’s own machinery turned to map one problem onto another. Even the study of impossibility is done with algorithms.)

The schoolroom, again

Return, one last time, to where this began — a classroom, a boy, a long column of numbers to add. The teacher meant the sum of 1 through 100 as busywork; the boy, Gauss, refused to do most of the work, paired the numbers off, and wrote a single line. Every chapter of this book was a variation on that morning. Binary search refused to examine all-but-one of a sorted list. The notebook refused to recompute what it had already computed. The ripple refused to wander a graph twice. Backpropagation refused to find a billion derivatives separately. Each is a particular, hard-won way of *not doing the work* — and each was invented by a person, often unnoticed for years, sometimes by two people on opposite sides of a border who never met.

You set out to learn the usual algorithms, the ones in a college book, and some foundations of the machines that learn. You have them. But you have also, I hope, learned the deeper thing the algorithms are made of: the conviction that effort and intelligence are different quantities, that the second can dramatically reduce the

first, and that the reduction is itself a thing you can study, prove, measure, and improve. That conviction is the whole of the subject, and it does not run out. There is no last refusal. The interlude's other exponential — the one made of ideas rather than transistors — is still climbing, and it climbs through people, and it has just acquired one more: you have the tools now.

The map has edges. Two of them you have just seen — the wall that cannot be passed, and the door no one can find. There are others, and there are regions with no edges drawn yet at all, blank space where the next refusals are waiting to be invented. Some of them are waiting for you.

The work that remains is not busywork. Go find the line that does it in one stroke.

End of the technical arc. What follows in a complete edition — solutions to selected Diamond problems, a guide to the labs, and notes for the teacher — is scaffolding. The book is the nine chapters, the ten labs, and the two questions. Everything else is commentary.

Going Deeper

★ ★ ★

A short, opinionated shelf — where to go when this book runs out. It is arranged the way the book was: the friendly first steps, then the standard references, then the machines that learn, then the edge of the map. Nothing here is required, and none of it is a textbook you must finish. All of it is an invitation.

Read next — and now

- **Brian Christian and Tom Griffiths, *Algorithms to Live By* (2016).** The gentlest possible sequel: the very ideas of this book — searching, sorting, when to stop looking, how to guess well — turned on everyday life, from packing a closet to choosing an apartment. No code, no equations, all intuition. Read it the month you finish this one.
- **Grant Sanderson, *3Blue1Brown* (videos, free online).** The *Essence of Linear Algebra* and *Neural Networks* series are the clearest moving pictures of the mathematics under Chapters 8 and 9. Watch the neural-network series the week you finish Chapter 9, and the descent of Chapter 8 will turn from words into a thing you can see.

The standard references — for the reread at 17

- **Thomas Cormen, Charles Leiserson, Ronald Rivest, and Clifford Stein, *Introduction to Algorithms* (4th edition, MIT Press, 2022).** The book the whole field calls *CLRS* — the comprehensive reference for everything in Chapters 1 through 7, with every proof in full. This is *the* MIT algorithms book; when a university course says “the algorithms textbook,” it means this one. We kept ours to a first taste on purpose. This is the full meal, and you are now ready to sit down to it.
- **Steven Skiena, *The Algorithm Design Manual* (3rd edition, 2020).** Warmer and more practical than *CLRS*, full of “war stories” of real algorithms meeting real problems. The best companion for someone who wants to *use* this material, not only prove it.
- **Charles Petzold, *The Annotated Turing* (2008).** Walks line by line through Turing’s 1936 paper — the halting-problem argument of the epilogue, in the original. Demanding, and worth it once the epilogue has made you curious about where the wall came from.

The machines that learn — further into Chapter 9

- **Andrej Karpathy, *Neural Networks: Zero to Hero* (video course, free online).** Builds, from nothing, exactly what Chapter 9 builds: a tiny automatic-differentiation engine (*micrograd*) and then a small language model (*makemore*, then *nanoGPT*). If Chapter 9 was your favorite chapter, this is the next thing to do with your own hands.

- **Michael Nielsen, *Neural Networks and Deep Learning* (free online book).** Derives backpropagation from first principles, with the clearest explanations of *why* it works that exist anywhere.

The edge of the map — years from now

- **Avi Wigderson, *Mathematics and Computation* (Princeton University Press, 2019; free PDF from the author).** Wigderson — winner of both the Abel Prize and the Turing Award — is among the deepest living authorities on the P-versus-NP question and on the role of randomness (Chapter 7's coins) in computation. The book is written for mathematicians; treat it as a mountain to keep in view, not one to climb yet. That the epilogue's open question has a living literature this deep is itself the point.
- **The P vs NP Problem, Clay Mathematics Institute.** The official statement, by Stephen Cook, of the epilogue's great open question — one of seven Millennium Prize Problems, with a million-dollar prize unclaimed. Short, precise, and exactly as unsolved as the day you read it.
- **Scott Aaronson, *Quantum Computing Since Democritus* (2013), and the blog *Shtetl-Optimized*.** The most candid, funny, and deep writing anywhere on what computers can and cannot do. Start with the blog; read the book when the mood for the deep end strikes.

One closing word, the book's own. Every title on this shelf was written by someone who refused to do most of the work and then proved the refusal correct — and who left the door open behind them. Go in.